The Impact of AI Coding Assistants on Software Engineering

Annie Vella

A thesis submitted in fulfilment of the requirements for the degree of
Master of Engineering in Software Engineering,
The University of Auckland, 2026.

Abstract

The field of software engineering has transformed many times since its origins, but the impact of AI coding assistants may be the most significant yet. This study explores how engineers experience that impact and how their perceptions evolve over time. Through a longitudinal mixed-methods investigation, we examined how professional software engineers perceive the effects of AI coding assistants across four areas: task focus, developer experience and productivity, skill development and self-efficacy, and prompt engineering as an emerging competency. Two questionnaires were administered six months apart, yielding 158 eligible participants at the first time point, 101 at the second, and a matched longitudinal cohort of 95. Quantitative measures were complemented by reflexive thematic analysis of qualitative accounts.

Participants reported spending less time on most development tasks, with 82% reporting less on writing code. Reviewing code was the exception, increasing only modestly. While this suggests a broader shift in focus from creation to verification activities, this alone does not account for all the perceived reductions. To explain this gap, we propose a new category of work we term *supervisory engineering work*, encompassing the direction, evaluation, and correction of AI output. We identified a productivity-experience paradox: productivity perceptions held stable, with 84% reporting improvement at both time points, yet the proportion reporting worsened experience in at least one dimension nearly doubled from 14% to 27%, with flow state and cognitive load eroding while feedback loops improved. Most participants felt more capable when using AI tools, with 85% reporting higher self-efficacy. Views on skill development were more divided, with some describing a speed-depth trade-off. Prompt engineering perceptions converged toward substitution of traditional coding skills, though its status as a genuinely new competency remains contested. Finally, we found that mindset mattered more than circumstance, with self-efficacy predicting experiential outcomes—productivity, developer experience,

and skill development—more strongly than gender, experience level, or company size.

These findings suggest that, from the perspective of practising engineers, AI coding assistants are impacting both the nature of software engineering work and how engineers experience it, with implications for practitioners, organisations, educators, and researchers navigating this transition.

Acknowledgements

Firstly, I would like to thank my supervisor, Dr Kelly Blincoe, for her guidance, patience, and expertise throughout this project. Her knowledge of empirical software engineering research has been invaluable, and her feedback always pushed me to sharpen my thinking. I am also grateful to Dr Robert Biddle, my Honours supervisor at Victoria University of Wellington, for his encouragement over the past two decades and for introducing me to Kelly.

I wish to thank Sarah D'Angelo for the thoughtful conversations about my research topic and her feedback on the questionnaire design. I also wish to thank the members of the Human Aspects of Software Engineering Lab (HASEL) for their constructive feedback during my presentations of early findings, and in particular Dr Valerio Terragni for inviting me to contribute to a co-authored paper on AI-driven software engineering.

Finally, I wish to thank my employer, Westpac New Zealand, for supporting my postgraduate studies, and my parents for always encouraging me to pursue my dreams. Last but not least, I am eternally grateful to my wonderful husband for his unwavering belief in me and for making this journey possible.

Contents

Contents **6**

1 Introduction **9**
- 1.1 Research Questions . . . 10
- 1.2 Researcher Positionality . . . 10
- 1.3 Contributions . . . 10
- 1.4 Thesis Structure . . . 11

2 Background and Motivation **13**
- 2.1 The Changing Landscape of Software Engineering . . . 13
- 2.2 The Rapid Emergence of AI Coding Assistants . . . 16
- 2.3 Industry Pressure and Organisational Transformation . . . 17
- 2.4 What's at Stake for Software Engineers . . . 18
- 2.5 Motivation for This Study . . . 19

3 Literature Review **21**
- 3.1 Productivity and Workflow . . . 21
- 3.2 Developer Experience . . . 24
- 3.3 Skills, Learning, and Professional Development . . . 27
- 3.4 Emerging Competencies: Prompt Engineering . . . 28
- 3.5 Gaps in Current Understanding . . . 29

4 Methodology **33**
- 4.1 Research Paradigm . . . 33
- 4.2 Study Design . . . 33
- 4.3 Participants . . . 38
- 4.4 Analytical Structure . . . 40

5 Results **51**

5.1 Study Context 51
5.2 RQ1: Task Focus Shifts 56
5.3 RQ2: Developer Experience and Productivity 71
5.4 RQ3: Skill Development and Self-Efficacy 90
5.5 RQ4: Prompt Engineering 107

6 Discussion **123**
6.1 RQ1: Task Focus Shifts 123
6.2 RQ2: Developer Experience and Productivity 126
6.3 RQ3: Skill Development and Self-Efficacy 129
6.4 RQ4: Prompt Engineering 132
6.5 Cross-Cutting Patterns 135
6.6 Threats to Validity 137
6.7 Implications 139
6.8 Future Research Directions 141

7 Conclusion **143**

A Study Instrument **145**
A.1 Questionnaire Items 145
A.2 Response Scales 151
A.3 Construct Definitions Provided to Participants 151
A.4 Key Differences Between Questionnaires 152

B Standardised Variable Groupings **153**

C Software and Packages **155**

D AI Coding Assistants **157**

Works Cited **159**

Chapter 1

Introduction

AI coding assistants are changing the way software engineers work. Tools such as GitHub Copilot and Cursor, along with general-purpose AI systems like ChatGPT and Claude, have become embedded in everyday development work in just a few years. Promising reports of productivity gains [74, 25] have driven rapid adoption, both by organisations seeking efficiency and by individuals seeking support across a wide range of development tasks.

It is easy to view these tools primarily as productivity aids, but their impact may go deeper. AI coding assistants are changing where developers spend their time, how they experience their work, and how they see their own capabilities. There are early signs that they are transforming not just how software gets built, but what it *means* to be a software engineer.

Much of the early research in this area has centred on evaluating the technical capabilities of AI coding assistants: the correctness, security, and maintainability of generated code [100, 70, 66, 75]. Other studies have examined how engineers use these tools in practice [67, 72, 88]. A smaller body of work explores developers' perceptions, covering trust, usability, and impacts on learning [57, 51], though often with students rather than professional practitioners [94]. What remains largely missing is an understanding of how professional software engineers experience these tools over time: how their perceptions evolve as they gain experience, and what this means for their work, their skills, and their profession.

We address this gap through a longitudinal mixed-methods study of professional software engineers using AI coding assistants at work, on personal projects, or both. Participants completed two questionnaires six months apart, with 158 eligible respondents initially, 101 at follow-up, and 95 completing both phases to form the matched cohort. By prioritising lived experience over tool performance,

this study offers a practice-oriented view of how AI coding assistants are influencing software engineering work.

1.1 Research Questions

This study is guided by four research questions:

- **RQ1:** How does the introduction of AI coding assistants shift software engineers' perceived focus across various development tasks, such as designing, writing, refactoring, testing, debugging, and reviewing code?
- **RQ2:** How does the use of AI coding assistants impact software engineers' perceptions of their developer experience and productivity?
- **RQ3:** How do software engineers perceive the impact of AI coding assistants on their skill development and self-efficacy when building software?
- **RQ4:** How do software engineers perceive the role of prompt engineering as a new core competency in the AI-assisted era of software development?

These questions examine different aspects of the AI-assisted experience: observational perceptions of how focus shifts across tasks (RQ1), experiential perceptions of productivity, developer experience, skill development, and capability (RQ2–RQ3), and predictive beliefs about the profession's future (RQ4).

1.2 Researcher Positionality

This research was carried out by a part-time postgraduate student with over two decades of professional experience in software engineering. Rather than setting this experience aside, we take a reflexive interpretivist stance, using practitioner insight as a lens to understand how these tools are experienced in practice. We discuss how this position shaped the analysis in Section 4.4.2, and how we mitigated its potential influence on the questionnaire design in Section 6.6.

1.3 Contributions

This study offers the following contributions:

- Evidence of a *creation-to-verification shift*: most participants perceived spending less time on the development tasks we measured, with writing code showing the steepest decline. The balance tilted modestly toward verification activities over the study period, though not toward high-level design work as one might expect.
- A proposed new work category, *supervisory engineering work*, encompassing the effort required to direct AI, evaluate its output, and correct its errors. This construct, which emerged from convergent quantitative and qualitative patterns, may help explain where perceived time savings from traditional tasks are being reallocated.
- Evidence of a *productivity-experience paradox*: consistent productivity gains coexisting with declining developer experience, especially in flow state and cognitive load. This pattern suggests the established relationship between developer experience and productivity may operate differently when AI is embedded in workflows.
- Characterisation of *additive and substitutive belief profiles* regarding prompt engineering, with views shifting toward substitution over the study period, though whether it constitutes a genuinely new competency, or an extension of existing practices, remains contested.
- An indication that *mindset matters more than circumstance*: how engineers engaged with AI tools and perceived their own abilities were more strongly associated with outcomes than experience level, gender, or organisational context. This is encouraging because these are factors individuals can more readily influence.

Together, these findings offer a grounded perspective on how AI coding assistants are transforming professional practice, and what that may signal for the future of software engineering.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- Chapter 2 provides background and motivation, tracing the evolution of software engineering practices and the rapid emergence of AI coding assistants.

- Chapter 3 reviews relevant literature on AI coding assistants' impacts on workflows, productivity, developer experience, skill development, and emerging competencies.
- Chapter 4 describes the research methodology, including the longitudinal design, participant recruitment, data collection instruments, and mixed-methods analysis approach.
- Chapter 5 presents results structured by research question, complementing quantitative findings with themes from participants' qualitative accounts.
- Chapter 6 interprets the findings in relation to prior research, integrates patterns across research questions, and examines implications along with limitations and directions for future work.
- Chapter 7 concludes by summarising key contributions and reflecting on what these findings suggest about the evolving nature of software engineering work.

Chapter 2

Background and Motivation

This chapter traces the evolution of software engineering as a discipline shaped by persistent pressures and recurring attempts at transformation. It positions AI coding assistants as the latest, and arguably most significant, response to challenges that have defined the field since its origins. The chapter concludes by establishing the motivation for this study.

2.1 The Changing Landscape of Software Engineering

For decades, changes in the field of software engineering have been driven by a need to deliver software faster, better and cheaper. As society's dependence on software has increased and systems have grown in complexity and scale, these pressures have only intensified. The field has had to adapt by developing new tools, methodologies and practices to keep pace. Yet, no matter how much progress has been made, the demand for larger, more complex systems has frequently outpaced the available workforce to build them.

AI coding assistants appear to offer substantial potential for automating and optimising how software is designed, created, tested and maintained across the entire software development lifecycle. If these tools deliver on their promise, they could help close the gap between supply and demand that has long defined the field. However, with any change this significant, there are also uncertainties and risks to consider. Frederick P. Brooks Jr.'s "No Silver Bullet" essay [13] reminds us that the inherent complexity of software development has historically resisted dramatic improvements through technology alone. While there is considerable

hope that AI coding assistants will prove different, history suggests we must be mindful of the unintended consequences that such a transformation may have on the software engineering profession.

2.1.1 Historical Roots and Early Challenges

The origins of software engineering are often traced to the late 1960s, when growing problems in large software projects prompted concern among industry and academic leaders. As the increasing complexity of software systems revealed substantial flaws in early development practices, projects frequently faced delays, cost overruns, and system reliability issues. In 1968, the NATO Software Engineering Conference in Garmisch, Germany brought together experts to address these challenges. At the conference, attendees openly acknowledged a *"widening gap between ambitions and achievements in software engineering"* [69], and the term "software crisis" emerged to characterise the state of the field. Indeed, the very title of the conference was deliberately chosen as a provocation: "software engineering" expressed an aspiration rather than a reality, a call for software development to adopt the rigour of traditional engineering fields [79]. This conference is widely regarded as the birth of software engineering as a distinct professional discipline.

These early issues established the foundations for subsequent evolutions in the field, with many attempts to address these limitations through improved methodologies and better tooling. Yet the software crisis was never truly resolved; it simply became the normal state of affairs in an industry perpetually striving to close the gap between ambition and achievement [39].

2.1.2 Methodological Shifts

Each wave of methodological change not only reshaped processes but also redefined how engineers work. Structured programming in the 1970s and 1980s introduced rigour through clear control structures and modular design [30], and the waterfall model formalised development into distinct phases with document-based handoffs [84]. When sequential approaches proved too rigid for evolving requirements, iterative and incremental methods [55] and later Agile methodologies shifted the emphasis toward adaptability, frequent feedback, and cross-functional collaboration [34]. More recently, DevOps extended this integration across the software lifecycle, breaking down barriers between development and operations

through shared responsibility and automated deployment pipelines. Yet each shift often traded one form of overhead for another, improving delivery without fully resolving the underlying complexity.

2.1.3 Advances in Tooling and Automation

Alongside methodological change, tooling has steadily evolved to support software engineers. From high-level languages and compilers (early examples of what was then called "automatic programming" [2]) that raised the level of abstraction, to Integrated Development Environments (IDEs) that unified editing, compiling, and debugging into a single interface, each generation of tools has aimed to reduce accidental complexity and improve productivity. Version control systems enabled distributed collaboration, package managers simplified dependency management and code sharing, automated testing frameworks and static analysis tools provided earlier feedback on code quality, and CI/CD pipelines automated the build, test, and deployment process, enabling the rapid release cycles that modern methodologies demand [87].

Despite these advances, the fundamental tension between time, cost, and quality—the so-called *"Iron Triangle"* of project management [77]—has persisted. Each wave of tooling has delivered incremental improvements, yet the essential complexity of software development remains. As systems have grown more distributed and codebases more fragmented, the cognitive load on engineers has increased even as their tools have become more sophisticated [71]. This persistent gap between ambition and achievement has driven continued interest in technologies that might offer step-changes in productivity, and in recent years, Artificial Intelligence (AI) has emerged as the most promising candidate.

2.1.4 Early AI Efforts in Software Development

AI-driven approaches to software engineering are not new. Researchers began applying AI to development tasks in the 1970s, starting with expert systems that used knowledge derived from human experts to simulate reasoning and assist with complex problem-solving [29]. Program synthesis pursued a more ambitious goal: generating code automatically from formal specifications [59], while inductive programming explored learning programs from examples [9].

The vision was ambitious, but sceptics cautioned that expectations were often based on an idealised view of reality. Rich and Waters noted that a "cocktail

party" description of these efforts would promise to eliminate programming altogether: end users, who only needed to know about the application domain, would write brief requirements, and automatic systems would produce efficient programs satisfying them [82]. Although attractive, this description rested on faulty assumptions. Systems could be end-user oriented, general-purpose, or fully automatic, but not all three at once. Narrow-domain systems showed real promise, yet the grander vision of eliminating programming altogether remained out of reach.

2.1.5 Modern AI Coding Assistants as a New Wave

Today, with the emergence of generative AI and large language models (LLMs) trained on vast corpora of code and natural language, we are hearing remarkably similar promises. This time, however, many practitioners argue that the technology feels qualitatively different. Rather than requiring formal specifications or narrow domain knowledge, developers interact with modern AI coding assistants through natural language and code context. These systems can read and explain code, debug errors, generate implementations, write tests and documentation, and even execute commands on the developer's behalf. They take many forms: IDE plugins like GitHub Copilot[1], conversational interfaces like ChatGPT[2], terminal-based agents like Claude Code[3], and purpose-built agentic IDEs like Cursor[4]. Proponents argue that generative AI can greatly enhance software productivity by automating routine tasks such as code generation and testing, freeing developers to concentrate on more complex and creative aspects of their work [35]. Yet familiar concerns persist: questions about code quality, security risks, intellectual property, and the non-deterministic nature of these systems mean that careful validation remains essential. Whether this wave of AI will finally deliver on the long-standing promise of its predecessors, or encounter the same limits, remains an open question.

2.2 The Rapid Emergence of AI Coding Assistants

The adoption of AI coding assistants has been remarkably fast. GitHub Copilot became generally available in mid-2022 and ChatGPT launched later that year.

[1] `https://github.com/copilot`
[2] `https://chatgpt.com`
[3] `https://claude.com/product/claude-code`
[4] `https://cursor.com`

Since then, the market has expanded rapidly: as of December 2025, one industry directory lists over 600 AI development tools[5], and new entrants continue to appear regularly. The Stack Overflow Developer Survey reported that the proportion of developers using or planning to use AI tools rose from 76% in 2024 to 84% in 2025, with 51% now using them daily [90, 89]. Other surveys report even higher figures [31], but these sample different populations and frame adoption differently. The overall trend is consistent: AI coding assistants have moved from novelty to mainstream practice in just a few years.

The breadth of use cases may help explain this rapid uptake. Unlike earlier AI tools that targeted narrow domains, modern AI coding assistants support tasks across the development lifecycle and work with virtually any programming language. Common use cases include writing boilerplate code, fixing bugs, generating tests, and refactoring [47]. This generality, combined with ease of access through familiar interfaces, has lowered the barrier to adoption considerably.

Yet adoption does not imply uncritical acceptance. The 2025 Stack Overflow survey found that while 60% of developers view AI tools favourably, this represents a decline from over 70% in previous years [89]. More developers now distrust the accuracy of AI output than trust it, and a majority report frustration with solutions that are "almost right, but not quite". This mixed sentiment suggests that widespread use coexists with significant reservations.

2.3 Industry Pressure and Organisational Transformation

The rapid adoption of AI coding assistants is not driven by developers alone. Organisations increasingly view these tools as a means to boost productivity and reduce costs. Technical leaders now rank generative AI adoption among the top measures for improving developer productivity, alongside internal training and process improvements [47]. In some cases, organisations have gone further, embedding AI tools into standard workflows or expecting their use. In this climate, there is a growing expectation that teams can deliver more with less time and, potentially, fewer people. These expectations are being shaped by bold predictions from industry leaders. In 2024, Google CEO Sundar Pichai reported that more than a quarter of Google's new code was already generated by AI [32], and

[5] `https://ainativedev.io/landscape`

predictions grew bolder through 2025, with Anthropic CEO Dario Amodei predicting that AI would be writing 90% of code within three to six months, and essentially all of it within a year [15]. Microsoft CTO Kevin Scott predicted that within five years, 95% of code would be AI-generated [16]. For practitioners, such predictions create pressure to adapt, regardless of whether they prove accurate.

Software engineering has become an early focus for AI adoption because the domain lends itself to it. Programming languages have strict syntax, and outputs can be verified through automated tests. The methodologies, tooling, and verification practices that the software industry has developed over decades provide a natural framework for integrating AI-generated contributions. Source control, code review, continuous integration, and automated testing all serve as checkpoints where AI output can be validated before it reaches production. In this way, software engineering can be seen as a canary in the AI coal mine: an early proving ground where the promises and pitfalls of AI-assisted work are being tested before broader adoption across other professions.

Yet this organisational enthusiasm does not always align with developer experience. As noted earlier, many developers remain sceptical of AI output, and research suggests that adoption is shaped by both individual and organisational motivations and challenges [56]. The pressure to adopt these tools, combined with uncertainty about their long-term effects on skills and roles, places software engineers in a complex position: expected to embrace tools that promise productivity gains, while navigating concerns about quality, autonomy, and professional development.

2.4 What's at Stake for Software Engineers

Some may argue that the introduction of AI coding assistants is just another step in the evolution of software engineering, a field accustomed to change. This time, however, may be different. Unlike previous shifts, these tools are general-purpose, rapidly adopted, and capable of performing tasks that have long been central to the craft. If they deliver on their promise, they may finally help close the persistent gap between ambition and achievement, but as Brooks reminded us, we must also be mindful of the unintended consequences that such a transformation may have on the profession.

For instance, as AI tools become integrated into daily workflows, they may be altering how engineers allocate their time, potentially increasing the time spent

on higher-level activities like designing and reviewing code while reducing the time spent on writing it. Do engineers perceive these shifts as beneficial to their productivity and day-to-day experience, or do they bring new frustrations and challenges?

There are also questions about skill development and self-efficacy. These tools may boost confidence by acting as an ever-present collaborator, but does that support become a crutch that affects engineers' ability to learn and improve their coding skills independently? New competencies may also be emerging; the ability to craft effective prompts for AI tools is increasingly discussed as a skill in its own right, raising questions about whether it will complement or replace traditional coding competencies.

For a global community of professionals who have built careers around software engineering, these are not abstract questions. They affect how engineers approach their work, invest in their development, and think about their futures. The way engineers perceive these tools matters because perceptions drive behaviour: those who see clear benefit integrate AI into their core workflows; those who do not may limit their use or find workarounds. Perceptions shape which tasks get delegated to AI, how much effort engineers invest in adapting their practices, and whether adoption is sustained or superficial. As Meyer et al. observed, understanding how developers perceive their work can inform how productivity "is defined, measured, assessed and supported by tools and best practices" [62]. Adoption statistics and productivity metrics alone cannot capture this. To understand what these tools mean for software engineers, we must ask them directly.

2.5 Motivation for This Study

Against this backdrop, our study investigates how professional software engineers perceive the impact of AI coding assistants on their work. Recognising that some of the most significant changes cannot be captured through productivity metrics or system telemetry alone, we focus on how engineers themselves experience these tools in their day-to-day practice, across the four areas identified by our research questions.

By capturing these perceptions across two time points six months apart, this study provides a longitudinal view of how professional software engineers are adapting to AI coding assistants. The findings aim to inform educators shaping the next generation of software engineers, industry leaders defining roles and ex-

pectations, and organisations seeking to support their engineering teams through this period of rapid change.

Chapter 3

Literature Review

This chapter examines existing research on the impact of AI coding assistants on software engineering. The literature reviewed spans controlled experiments measuring productivity gains, surveys capturing developer perceptions, and qualitative studies examining workflow changes. We cover four main areas: productivity and how work gets done; the subjective experience of using these tools; what happens to skills, learning, and professional confidence; and whether prompt engineering is becoming a genuine competency. We conclude by identifying gaps in current understanding, particularly the absence of longitudinal, perception-focused research with professional developers, and explain how our study addresses them.

3.1 Productivity and Workflow

Developer productivity has been a primary focus of AI coding assistant research. A number of controlled experiments have been conducted, and one of the earliest, by Peng et al. in 2023, found that developers using GitHub Copilot completed a JavaScript task 55.8% faster than those without access [74]. Benchmark evaluations confirmed the underlying capability: Yetistiren et al. found that ChatGPT produced correct solutions 65% of the time on the HumanEval benchmark, with GitHub Copilot at 46%, and newer versions of these tools showed marked improvement, suggesting rapid advancement [100]. Weber et al. observed similar productivity improvements with both autocomplete and conversational interfaces, though each encouraged different usage patterns [97].

Enterprise studies reinforced these findings. A six-week experiment at ANZ Bank reported a 42% reduction in task completion time on predefined coding challenges [18]. Research at Cisco found time savings of 30–50% for specific tasks like

documentation and autocompletion, with smaller but meaningful gains for unit testing and debugging [72]. By 2025, large-scale field experiments involving 4,867 developers across Microsoft, Accenture, and a Fortune 100 company showed a 26% increase in completed tasks, with less experienced developers showing greater gains [25].

However, a more nuanced picture emerged as research expanded beyond isolated coding tasks. When Jimenez et al. introduced SWE-bench in late 2023, a benchmark based on real GitHub issues requiring changes across multiple files, even the best-performing models at the time could resolve fewer than 5% of issues [48]. Other evaluations showed a similar pattern: Kuhail et al. found that ChatGPT 3.5 solved easy and medium LeetCode problems well but struggled with hard ones [51]. Correctness also varied by programming language; Nguyen and Nadi found that GitHub Copilot's suggestions were correct 57% of the time for Java but only 27% for JavaScript [70]. These findings highlighted that productivity benefits depend heavily on task complexity and context. An IBM study found that while AI coding assistants often provided net productivity increases, these benefits were not experienced uniformly, and some developers saw no gains at all [98].

A large-scale usability survey helped explain why benefits vary. Liang et al. surveyed 410 developers and found that the most common reasons for not using AI coding assistants were that generated code failed to satisfy functional or non-functional requirements, and that developers found it difficult to steer the tools toward producing desired output [57]. Among those who did use these tools, 28% reported often giving up on incorporating generated code entirely. Even when AI suggestions appear helpful, they may introduce subtle problems: DePalma et al. found that ChatGPT's code refactoring improved naming conventions and formatting while introducing semantic inconsistencies that broke functional correctness [28]. These findings suggest that productivity limitations stem not only from model capability but also from the difficulty of evaluating whether suggestions are truly correct.

Industry surveys consistently find developers perceive AI coding assistants as productivity-enhancing: 81% in the 2024 Stack Overflow survey identified increased productivity as the primary benefit [90], while over 80% in the 2025 DORA report reported productivity gains [31]. A 2025 study of over 500 developers found this rose to 90% among regular users [42]. However, a more provocative finding emerged in 2025 when Becker et al. conducted a randomised controlled trial with

16 experienced open-source developers completing 246 tasks on mature codebases and found that AI tool usage had actually *increased* task completion time by 19% even though developers estimated AI had reduced their completion time by 20% [5]. Methodological considerations—including the small sample, substantial task heterogeneity, and retrospective estimation procedures—suggest caution in generalising this specific finding. Nevertheless, earlier security research shows a similar pattern: Pearce et al. found that AI tools frequently generated vulnerable code [73], while Perry et al. found that developers using AI assistants were more likely to believe their code was secure when it was not [75]. These studies, though context-specific, suggest that developer perceptions may not always reflect actual outcomes.

Given the evidence that these tools accelerate specific software engineering tasks, one would expect that embedding them into daily practice will shift where developers focus their effort. A survey of 481 developers found a negative correlation between how much they enjoy an activity and their willingness to delegate it to AI: writing tests and documentation were rated least enjoyable and most likely to be delegated, while implementing new features was most enjoyable and least likely to be delegated [88]. This aligns with broader findings that developers want AI to reduce toil, freeing them to focus on complex problem-solving [26]. Creativity in software engineering, often expressed through refactoring, designing reusable code, and learning, is something developers want to preserve rather than delegate [44].

Research has begun to clarify how this effort is reallocated. An early user study found that while developers preferred using GitHub Copilot, they did not always complete tasks faster due to time spent understanding, editing, and debugging generated suggestions [94]. Bird et al. observed similar patterns and predicted that as AI tools become more integrated, developer roles will shift toward assessing suggestions rather than performing tasks directly [10]. A controlled study by Mozannar et al. measured how developers allocate time when coding with GitHub Copilot and found that new tasks introduced by AI assistance (verifying suggestions, editing generated code, crafting prompts, and waiting for responses) consumed over half of total coding time on average [67]. Verification alone accounted for 22% of task time, more than writing new code. Survey data reinforced this pattern: while 70% of developers reported that ChatGPT reduced coding time, 21% reported increased code review time, with some participants noting that review effort could absorb the initial savings [93]. At a broader level of task allocation,

a two-year study of open-source developers found that access to GitHub Copilot shifted effort toward core coding activities and away from peripheral project management tasks, with effects strongest among less experienced developers [41].

These studies indicate that AI coding assistants redistribute engineers' focus and effort, though several questions remain open. Controlled studies capture behaviour on predefined tasks, but how these patterns manifest in the varied contexts of day-to-day professional work is less well understood. Furthermore, both the underlying AI models and the tools built on them are evolving rapidly as adoption continues to grow. Understanding how engineers' focus shifts as these tools mature, and how engineers perceive those shifts, would benefit from research that tracks practitioners over time.

3.2 Developer Experience

Research consistently finds that developer experience and productivity are intertwined, each shaping the other. Studies use varied terminology (some focus on "satisfaction", others on "happiness" or "experience") but the underlying question is the same: how do developers feel about their work, and does it matter for outcomes? The evidence suggests it does. A survey of 622 developers across three companies found that non-technical factors, including job enthusiasm, useful feedback, and peer support, predicted self-rated productivity more strongly than technical factors [68]. Another study of 465 developers at Microsoft proposed a bi-directional relationship: satisfied developers tend to be more productive, and productive developers tend to be more satisfied, with factors such as impactful work, autonomy, and engineering tools mediating this relationship [91]. Given this relationship, understanding how AI coding assistants affect developer experience becomes essential for evaluating their true impact.

Early evidence is mixed. Chen found that developers using AI pair programming reported significantly higher satisfaction, enjoyment, and engagement than those working without AI assistance [20]. However, the same study identified qualitative concerns about "autonomy-losing" and "motivation-losing", suggesting that aggregate satisfaction scores may obscure important experiential trade-offs. A randomised controlled trial with a three-week diary study told a similar story: sustained use significantly increased developers' perceptions of AI tools as useful and enjoyable, with 84% reporting positive changes in daily work practices; however, views on trustworthiness remained unchanged even after extended

use, suggesting that familiarity may breed appreciation but not necessarily confidence in AI-generated output [17]. These experiential trade-offs may help explain the broader cooling of sentiment noted in Section 2.2. The question is whether productivity gains come at an experiential cost.

We use the Developer Experience (DevEx) framework proposed by Noda et al. to structure our investigation [71]. It identifies three dimensions: feedback loops (the speed at which developers receive information about their work), cognitive load (the mental effort required to complete tasks), and flow state (the ability to concentrate without interruption). This framework has gained traction in industry, likely because its three dimensions are easier to operationalise than more comprehensive taxonomies.

3.2.1 Feedback Loops

Feedback loops concern the speed at which developers receive information about their work, whether from compilers, tests, code review, colleagues, or other sources. Faster feedback generally supports better developer experience by reducing wait times and enabling quicker iteration [71]. Empirical research suggests that feedback loops are "the characterizing criterion of contemporary software development", with developers actively seeking information from peers, static analysis tools, and CI environments [7].

AI coding assistants provide a distinctive form of feedback: real-time suggestions as developers write code. This immediacy could shorten feedback loops considerably compared to waiting for code review or discovering issues later in the development cycle. Early evidence supports this: a year-long study of 300 engineers found that AI-assisted code review reduced pull request cycle time by 32% [52]. Houck et al. found that AI adoption led to fewer interruptions within teams, as developers relied less on colleagues for quick answers [42]. However, the quality and relevance of feedback matters as much as its speed, and whether AI-generated suggestions constitute feedback that developers find genuinely useful across different contexts remains an open question.

3.2.2 Cognitive Load

Cognitive load has received considerable attention in AI coding assistant research, though findings are mixed. Pinto et al. found that developers using a contextualised AI assistant reported reduced cognitive load from having information centralised within their IDE, eliminating context-switching to external sources [76].

In educational settings, AI assistance can reduce intrinsic and extraneous cognitive load for programming students while enhancing germane load, potentially supporting deeper learning [43]. However, for experienced developers working on complex tasks, the picture is less clear. Alami and Ernst found that AI-assisted code reviews sometimes created *higher* cognitive load than human peer reviews, due to excessive detail in AI-generated feedback and the effort required to evaluate suggestions lacking project context [1].

Trust calibration represents another, often overlooked, source of cognitive demand. Deciding whether to accept, modify, or reject AI suggestions requires ongoing evaluation, and this effort is not trivial. Brown et al. identified several factors shaping developer trust in AI code completion: the perceived quality of suggestions, the developer's proficiency in the relevant language, and the context of the work being undertaken [14]. Trust is not a one-time decision but an ongoing calibration process that consumes attention and mental resources. The phenomenon of suggestions that are "almost right, but not quite" creates particular friction: the 2025 Stack Overflow survey found this to be a leading source of developer frustration with AI tools [89]. Suggestions that are close but incorrect may require more cognitive effort to evaluate and fix than writing code from scratch. Physiological evidence supports this, with a study measuring heart rate variability finding that the opacity and unpredictability of AI-generated code led to elevated stress markers and frequent interpretive confusion among novice users [103].

These findings suggest that cognitive demands may not simply decrease with AI assistance; they shift and redistribute. The mental effort required for trust calibration, for instance, is largely invisible in productivity metrics but shapes how developers experience their work.

3.2.3 Flow State

Flow is a state of complete absorption in an activity, where concentration is intense and the work feels intrinsically rewarding [24]. In software engineering, this typically means sustained focus free from disruptive interruptions. The cost of interruptions is well-documented: in a laboratory study simulating office work, interrupted workers completed tasks faster by compensating with increased speed, but experienced significantly higher stress, frustration, time pressure, and mental workload after only twenty minutes [60]. For software developers specifically, Meyer et al. found that half cite uninterrupted focus as central to feeling productive [62]. A recent systematic review identified several factors that affect flow,

including context switching between tasks, the frequency and timing of interruptions, and the degree of concentration a task permits [81]. Development tools also play a role: a survey of 401 developers found that tools providing positive developer experience facilitate flow [83].

How AI coding assistants affect flow state is not yet well understood. These tools might reduce interruptions by answering questions within the IDE, or the cycle of prompting and evaluating suggestions might itself constitute a form of interruption. An interview study with 12 IT professionals found that AI tools did not significantly enhance or disrupt flow states; participants attributed their flow experiences primarily to the nature of the task [54].

3.3 Skills, Learning, and Professional Development

Whether AI coding assistants affect skill development remains an open question. In a survey of over 700 Microsoft developers, only 10% believed AI tools might cause skill atrophy, though this figure may be conservative given how recently these tools had been introduced [49]. A related concern is dependency: if developers come to rely on AI assistance, will they be able to work effectively without it? This worry surfaces frequently in practitioner discourse, yet empirical research on how such dependency actually develops over time remains limited.

The relationship between AI tool use and skill development may differ substantially by experience level. Research suggests that AI coding assistants function as an asset for experienced developers, who can effectively filter incorrect or suboptimal suggestions, but may be a liability for less experienced developers, who lack the knowledge to recognise poor recommendations [66]. A survey of over 3,000 developers found that while coding experience does not predict AI adoption, it significantly shapes how developers conceptualise the tool's role: less experienced developers tend to view AI as a teacher, while experienced developers are more likely to see it as a junior colleague or content generator [102]. In educational settings, programming students who struggled often finished tasks with an "illusion of competence", believing they had performed better than they actually had [78]. Research with students in software engineering projects found that AI tools are most useful in early development stages for foundational code structures [80]. Whether similar dynamics apply to professional developers remains an open question.

Beyond objective skill impacts, AI tools may affect developers' beliefs about their own capabilities. Self-efficacy, an individual's belief in their capacity to ac-

complish tasks, shapes how developers approach challenges, persist through difficulties, and adopt new tools [4]. It is distinct from actual skill: a developer might possess strong technical abilities yet doubt their competence, or conversely feel confident while lacking foundational knowledge. In educational settings, early findings are mixed: some studies report increased self-efficacy among students using AI tools [101], while others find no significant short-term impact [38]. How these dynamics play out for professional developers remains largely unexplored. These tools might boost confidence by providing an ever-present collaborator, or undermine self-belief by creating dependency or making developers question whether achievements reflect their own abilities or the AI's. Understanding how self-efficacy both shapes and is shaped by sustained AI tool use requires longitudinal research.

3.4 Emerging Competencies: Prompt Engineering

The term *prompt engineering* refers to the practice of crafting targeted instructions that steer LLM output toward desired results without modifying the model's underlying parameters [85]. The concept emerged rapidly following the release of LLMs, and researchers have responded accordingly: systematic surveys now catalogue prompting techniques [85], taxonomies of prompt patterns have been developed specifically for software engineering tasks [86], educational researchers have proposed frameworks for assessing prompt engineering as a distinct 21st century skill [36], and validated instruments have been developed to measure prompt engineering competence [40]. In industry, the term has entered professional vocabulary through job titles and training courses; analysis of over 20,000 job postings found prompt engineering emerging as a distinct profession with identifiable skill requirements [95]. Some researchers have gone further, arguing that prompts function as programs and proposing formal methodologies that adapt software engineering principles to prompt development [58, 21]. This growing academic and professional attention suggests that prompt engineering is being treated as a genuine competency, though how practitioners themselves perceive its importance remains less clear.

What is clear is that how developers formulate their inputs affects outcomes. Mastropaolo et al. found that small changes in prompt phrasing and structural clarity could substantially alter the quality of code generated by GitHub Copilot [61]. Similarly, a large-scale usability survey identified prompt formulation as

a key determinant of whether developers found AI tools effective [57], while case study research found that vague or ambiguous inputs often yielded suboptimal results [72]. Research with students has found that higher-quality prompt engineering skills predict LLM output quality, suggesting prompt engineering is a measurable competency rather than mere trial and error [50].

Whether prompt engineering will complement or replace traditional coding skills remains contested. One developer survey found that roughly a third of respondents believed software engineering may evolve into a form of prompt engineering [46], with the core skill shifting from writing code to specifying requirements and guiding AI. Others argue that prompt engineering simply reframes existing competencies: communication, specification, and problem decomposition have always been central to software engineering, and effective prompting draws on the same abilities. Research with programming students suggests that prompt engineering may promote computational thinking and reshape how coding skills develop [27]. Research with professional developers reinforces the distinctiveness of prompt development: while prompts function as programs, the process of developing them differs fundamentally from traditional software development, and even experienced prompt programmers report struggling to form reliable mental models of model behaviour [58]. The concept itself continues to evolve; recent discourse has shifted toward "context engineering", reflecting recognition that providing appropriate context extends beyond crafting individual prompts.

Despite growing interest, including calls from educational researchers for prompt engineering to be embedded in curricula [96], research on how practitioners actually perceive it remains limited. We know little about whether engineers view it as a core professional competency, a supplementary technique, or a passing trend. Understanding how these perceptions evolve as developers gain experience with AI tools is essential for informing both professional development and educational curricula.

3.5 Gaps in Current Understanding

This review reveals several gaps in our understanding of how software engineers experience AI coding assistants. These gaps cluster around three themes: temporal limitations, population focus, and methodological scope.

The most significant gap is temporal. The productivity evidence in Section 3.1 derives largely from benchmark evaluations and short-term usage studies. Benchmarks assess model capability at fixed moments [100, 48]; controlled experiments

measure task performance in single sessions [74, 94]; enterprise trials track outcomes over evaluation periods lasting weeks [18, 72]; cross-sectional surveys capture perceptions at isolated points [90, 46]. In fact, the 2025 Stack Overflow survey reported sentiment shifting considerably from the previous year [89], which suggests that perceptions evolve as experience deepens. But annual surveys sample different populations each year, so understanding how individual developers' views change over time is difficult. Recent research has begun to address this limitation: longitudinal studies tracking professional developers over months or years have emerged, examining productivity metrics such as commit activity, pull requests, and code volume [92, 99]. A recent systematic literature review of 37 studies on AI coding assistants and developer productivity similarly observed that most research lacks longitudinal designs [64]. Importantly, these emerging longitudinal studies primarily measure objective outcomes. Research examining how professional developers perceive their productivity, experience, skill development, and emerging competencies as they gain sustained experience with AI tools remains limited.

A second gap concerns population. Throughout the review, we noted a pattern: empirical findings often derive from student populations. Research on cognitive load benefits comes from programming students [43]. Studies of prompt engineering and computational thinking focus on CS1 courses [27, 50]. The "illusion of competence" phenomenon was observed in educational settings [78], as was research on human-AI collaboration in software engineering projects [80]. While such studies provide valuable insights, professional software engineering involves different pressures: legacy codebases, ambiguous requirements, team coordination, and real consequences for failures. Whether findings from educational contexts transfer to professional practice remains unclear. Research with professional developers exists, as in the Microsoft survey on AI preferences [49] and studies of mental models in prompt development [58], but remains less common than student-focused work.

A third gap concerns methodological scope. Many studies measure what AI tools achieve: task completion times, code correctness, bug detection rates. Fewer examine how engineers perceive these tools' impacts on their work. Yet perceptions matter: they drive adoption decisions, shape satisfaction, and influence how engineers invest in their professional development. Furthermore, within experiential research, certain dimensions remain underexplored. We noted that cognitive load has received attention, but feedback loops and flow state remain largely

unexamined in the context of AI coding assistants. Self-efficacy has received attention in educational contexts, with mixed findings on whether AI tools boost or undermine student confidence, but research examining how professional developers' self-efficacy evolves with sustained AI tool use remains limited. Moreover, existing research tends to examine these dimensions in isolation, yet they likely interact: an engineer who perceives productivity gains but experiences cognitive strain may develop different attitudes than one who experiences both benefits.

These gaps motivate the research questions that guide this study. By tracking a cohort of professional software engineers over six months, we address the temporal gap directly. By focusing on practitioners rather than students, we address the population gap. By examining perceptions across task allocation (RQ1), developer experience and productivity (RQ2), skill development and self-efficacy (RQ3), and emerging competencies like prompt engineering (RQ4), we address both the methodological scope gap and the need for a more holistic view.

Chapter 4

Methodology

This chapter describes the design, data collection, and analytical approach used to investigate how professional software engineers perceive the impact of AI coding assistants on their work. It covers a longitudinal mixed-methods design combining quantitative analysis with qualitative accounts of practitioners' experiences.

4.1 Research Paradigm

We adopted a pragmatic interpretivist stance for this research due to the socio-technical nature of the topic and the researcher's position as a practising software engineer. An interpretivist approach allows us to treat subjective perceptions as meaningful data, while a pragmatic stance recognises that our aim is not only to interpret these experiences but to generate practical insights that are useful for software engineers, their teams, organisations and future research. This combination guided our mixed-methods approach, collecting both quantitative and qualitative data to address our research questions. It also guided our choice of reflexive thematic analysis for the qualitative data, where we treated participants' responses as personal interpretations of their experiences rather than objective accounts.

4.2 Study Design

This section describes the longitudinal design, data collection, questionnaire structure, recruitment, and ethics approval.

4.2.1 Longitudinal Design

We invited the same participants to complete two questionnaires six months apart. AI coding assistants are evolving rapidly and adoption continues to grow; a single time point would only have captured a snapshot, whereas two time points allowed us to examine how perceptions evolved over time. The first questionnaire (Q1) was administered in October 2024 and the second (Q2) in April 2025, each open for four weeks.

4.2.2 Data Collection Approach

Both questionnaires were administered online through Qualtrics, enabling us to reach professional software engineers globally. Each included quantitative and open-ended questions aligned to our research questions, was designed to take approximately 15 minutes to complete, and followed a convergent parallel mixed-methods design [23], with qualitative accounts contextualising quantitative patterns. We also collected demographic and organisational context information to support our analysis.

4.2.3 Questionnaire Structure

Each questionnaire had seven sections, of which six aligned to our research questions: qualifying questions; experience and perceptions of AI tools; perceived productivity and developer experience; perceived impacts on skill development and self-efficacy; prompt engineering; and participant demographics. The seventh invited broader reflections on the profession. These open-ended questions captured wider context rather than addressing any of the four research questions, so although the responses were collected and coded, they are not reported here to keep the thesis focused on those questions.

The questionnaires included Likert-scale, multiple choice, ranking, and open-ended questions to support a mixed-methods analysis. The structure was developed to align with the study's four research questions (see Chapter 1). The full study instrument is provided in Appendix A.

The *Experience and Perceptions* section captured tool usage patterns and attitudes. Participants reported which AI coding assistants they used (Q3.1), how frequently (Q3.2), and their initial impressions (Q3.3 in Q1) or anticipated disappointment if tools were removed (Q3.3 in Q2). Participants also ranked seven

concerns about using AI coding assistants in their workflow (Q3.5): quality, readability, maintainability, security, dependency, learning curve, and other; we use *primary concern* to refer to whichever category a participant ranked first. These served as predictors or grouping variables across all research questions.

RQ1 (How does the introduction of AI coding assistants shift software engineers' perceived focus across various development tasks?) was also explored within the *Experience and Perceptions* section, which asked how participants perceived changes in focus across six core development tasks (Q3.4): designing, writing code, refactoring code, reviewing code, testing, and debugging. These tasks reflect core activities identified in empirical studies of software engineering practice [63]. Participants were also asked to describe how AI coding assistants had changed their workflow (Q3.6).

RQ2 (How does the use of AI coding assistants impact software engineers' perceptions of their developer experience and productivity?) was explored through the *Developer Experience and Productivity* section, which included questions derived from the DevEx framework [71], with participants rating the impact of AI coding assistants on each of its three dimensions (Q4.2): cognitive load, feedback loops, and flow state. Participants were also asked about perceived productivity change (Q4.1). They were invited to describe specific instances where AI coding assistants helped or hindered their work in Q1, and any unexpected benefits or challenges they had experienced in Q2 (Q4.3).

For *RQ3* (How do software engineers perceive the impact of AI coding assistants on their skill development and self-efficacy when building software?), the *Impact on Skill Development and Self-Efficacy in Software Development* section examined two constructs: perceived impact on skill development (Q5.1) and self-efficacy when building software with AI coding assistants (Q5.2). The latter was measured using an adapted version of the New General Self-Efficacy Scale (NGSE), an 8-item instrument [19]. We also invited participants to describe how AI coding assistants had influenced their coding skills and practices (Q5.3).

To explore *RQ4* (How do software engineers perceive the role of prompt engineering as a new core competency in the AI-assisted era of software development?), the *Prompt Engineering* section examined two complementary dimensions: anticipated future importance (Q6.1; unipolar scale) and anticipated skills replacement (Q6.2; bipolar scale). Participants were also asked how engineers should develop their prompt engineering skills in Q1, and whether their own approach had evolved over the six-month study period in Q2 (Q6.3).

Together, these research questions capture three distinct construct types: RQ1 examines *observational* perceptions (what engineers notice happening in their workflows), RQ2 and RQ3 examine *experiential* perceptions (how engineers feel about their productivity, developer experience, skill development, and capabilities), and RQ4 examines *predictive beliefs* (what engineers anticipate about prompt engineering as a future competency). This distinction becomes relevant when interpreting which factors predict which outcomes.

A number of demographic and organisational variables were captured in the *Demographics and Experience* section. This included gender, age, years of professional experience, self-reported expertise (using the Dreyfus model [33]), role, primary programming language, industry, company size, country of residence and country of company headquarters. In Q2, demographic questions were optional; participants could choose to update selected fields (e.g., job role or primary language) if their circumstances had changed since Q1. Where no updates were made, original responses from Q1 were used in the analysis.

4.2.4 Participant Recruitment

We recruited participants using a combination of convenience and referral-chain sampling [3]. The primary recruitment channels were our professional networks, including LinkedIn, X (formerly Twitter), Facebook, Discord, and several Slack communities, where invitations were shared directly or via community administrators. Additionally, 33 organisations were contacted to request internal distribution of which nine agreed to share the invitation with their engineering teams. Recruitment therefore drew from both individual networks and organisational channels, enabling participation from software engineers across a range of roles, experience levels, industries and countries.

The *Qualifying Questions* section at the beginning of both questionnaires included eligibility criteria. Participants were required to be professional software engineers and to use at least one AI coding assistant. Those who answered "No" to either qualifying question were routed to the end of the questionnaire and did not complete the sections aligned with the research questions.

4.2.5 Ethics

This study received approval from the University of Auckland Human Participants Ethics Committee (Reference number UAHPEC27902). All participants were provided with a Participant Information Sheet and completed an electronic consent

form. They could withdraw their questionnaire data up to two weeks after submission. Organisations were first invited to participate via email, which included the Employer Information Sheet and a formal Employer Consent Form. Those who agreed provided written consent, after which they were sent a recruitment message to forward to their software engineering staff. All individual participants completed their own electronic Participant Consent Form prior to beginning the questionnaire.

The study was confidential rather than anonymous. Email addresses were collected for longitudinal follow-up and the prize draw, then deleted per the ethics protocol.

A $200 NZD gift card was offered as a prize draw incentive for those who completed both questionnaires. Participants were assigned unique identifiers for data linkage between questionnaires and for the prize draw.

4.2.6 Data Preparation

To support consistent analysis across both questionnaires and statistical models, we applied a two-stage data preparation process. First, raw responses were cleaned and harmonised. For questions that included an *"Other (please specify)"* option, we manually reviewed and reclassified the free-text entries into a canonical set of labels—for example, mapping *"Rails"* to *"Ruby"* and collapsing AI tool variants like "Claude 3.5 Sonnet" and "claude.ai" into a single standard name.

Second, these cleaned values were transformed into standardised analysis variables that reduced sparsity and reflected meaningful distinctions in professional practice. Demographic and organisational context variables were collapsed into ordered or unordered groups that correspond to typical career stages, technical ecosystems, and organisational scales. Low-frequency responses (for example, infrequent roles or rare programming languages) were combined into an "Other" category. These standardised variables were used consistently across all descriptive analyses, regression, and mixed-effects models. Groupings are shown in Appendix B.

4.2.6.1 Data Not Used in Analyses

The following variables were used for descriptive summaries (Table 4.1) but not in inferential models:

- **Age** – strong collinearity with experience (Spearman $\rho = 0.78$, $p < 0.001$).

- **Self-rated expertise** – professional experience offered a more reliable indicator of career progression.
- **Industry and country of residence** – many categories with low cell counts.

Company size and **usage location** were included in cross-sectional regressions but not in longitudinal mixed-effects models to maintain parsimony given sample-size constraints. **Low-frequency AI tools** (< 10 users) were also omitted from tool-level statistical tests due to insufficient sample sizes.

4.3 Participants

A total of 224 responses were recorded for Q1. Of these, 35 were incomplete (Progress $< 100\%$), 16 respondents indicated they were not professional software engineers, and 15 reported not using AI coding assistants, yielding a final Q1 sample of 158 eligible participants (71% eligibility rate). For Q2, 111 responses were recorded, of which 6 were incomplete and 4 reported not using AI coding assistants, yielding 101 eligible participants (91% eligibility rate). Responses were also checked for unusually short completion times and for straightlining, but no further exclusions were needed. Within the qualitative analysis, all open-ended answers were read in full during familiarisation (Section 4.4.2.2), and in the rare cases where a reply did not address the question, it was left uncoded.

Longitudinal Attrition Analysis. Of the Q1 and Q2 eligible participants, 95 (60%) participated in both and formed the matched longitudinal cohort. Fisher's exact tests compared those retained versus lost to follow-up across six variables: gender, experience, role, programming language, company size, and country. No significant differences were found (all $p > 0.05$), and effect sizes were small (Cramér's $V \leq 0.23$), suggesting that attrition was not systematic.

4.3.1 Participant Demographics

Table 4.1 summarises sample characteristics using the standardised variable groupings employed throughout the analysis (see Section 4.2.6).

Participants represented 28 countries, with the highest participation from New Zealand (41%). The sample is not intended to be statistically representative of the global software engineering population. As with any convenience sampling,

Table 4.1: Participant Demographics

Variable	Category	Includes	Q1 n (%)	Q2 n (%)
Gender				
	Man		135 (85%)	86 (85%)
	Woman		19 (12%)	12 (12%)
	Non-binary		1 (1%)	1 (1%)
	Self-described	Prefer to self-describe	2 (1%)	1 (1%)
	Undisclosed	Prefer not to disclose	1 (1%)	1 (1%)
Age				
	18–24		5 (3%)	2 (2%)
	25–34		55 (35%)	33 (33%)
	35–44		63 (40%)	45 (45%)
	45–54		28 (18%)	15 (15%)
	55+	55–64; 65+ years old	5 (3%)	4 (4%)
	Other	Prefer not to say; Missing	2 (1%)	2 (2%)
Experience Level				
	Junior	<1, 1–2, 3–5 years	21 (13%)	13 (13%)
	Mid-Career	6–10, 11–15 years	77 (49%)	51 (50%)
	Senior	16–20, 20+ years	59 (37%)	37 (37%)
	Other	Missing	1 (1%)	0 (0%)
Self-Rated Expertise				
	Novice		2 (1%)	1 (1%)
	Advanced Beginner		9 (6%)	6 (6%)
	Competent		19 (12%)	15 (15%)
	Proficient		65 (41%)	38 (38%)
	Expert		63 (40%)	41 (41%)
Role				
	Backend	Backend Engineer	37 (23%)	28 (28%)
	Frontend	Frontend Engineer	10 (6%)	7 (7%)
	Fullstack	Fullstack Engineer	57 (36%)	32 (32%)
	Manager	Engineering Manager; Software Architect	24 (15%)	16 (16%)
	Mobile	Mobile Engineer	12 (8%)	7 (7%)
	Specialised	Data/ML; Platform; Quality Engineer	15 (9%)	9 (9%)
	Other	Desktop; Generalist	3 (2%)	2 (2%)
Primary Programming Language				
	Web	JavaScript; TypeScript	40 (25%)	32 (32%)
	Compiled	C#; Go; Java; Kotlin; Rust; Swift	76 (48%)	49 (49%)
	Interpreted	PHP; Python; Ruby	38 (24%)	19 (19%)
	Other	C++; Scala; Terraform	4 (3%)	1 (1%)
Company Size				
	Small	Self-employed; 1–50 employees	44 (28%)	25 (25%)
	Medium	51–500 employees	44 (28%)	25 (25%)
	Large	501–5,000 employees	34 (22%)	27 (27%)
	Very Large	5,001+ employees	33 (21%)	22 (22%)
	Other	Missing	3 (2%)	2 (2%)
Industry				
	Software/IT	Software Development, IT, or Technology	52 (33%)	26 (26%)
	Banking/Financial	Banking/Financial Services	32 (20%)	26 (26%)
	Consulting	Consulting/Professional Services	18 (11%)	11 (11%)
	Fintech	Fintech	11 (7%)	9 (9%)
	Internet/Telecom	Internet, Telecommunications, or Information Services	9 (6%)	4 (4%)
	Other	All other industries	36 (23%)	25 (25%)
Country of Residence				
	New Zealand		65 (41%)	48 (48%)
	Netherlands		16 (10%)	10 (10%)
	United Kingdom		14 (9%)	11 (11%)
	Australia		14 (9%)	6 (6%)
	United States		9 (6%)	3 (3%)
	Other	All other countries	40 (25%)	23 (23%)

potential biases include self-selection, platform effects, geographic clustering, and English-only administration.

4.4 Analytical Structure

Before addressing the research questions, we first characterised the AI tool landscape at the time of the study, including which tools participants used, their usage frequency, attitudes, and primary concerns. Given the pace of change in AI coding assistants, this contextual snapshot helps situate the research question findings within the specific tool ecosystem that participants were experiencing. This involved descriptive statistics for both cross-sectional samples and, where relevant, non-parametric paired tests for the matched cohort to identify significant shifts over the study period.

Across all four research questions, we followed the same three-step analytical structure. Given our two-time-point mixed-methods design, our analyses examined (i) perceptions at each time point using the full samples, (ii) within-participant change across the six-month period using the matched cohort, and (iii) participants' qualitative explanations of their experiences. Steps 1 and 2 below describe the quantitative analyses, while Step 3 describes the qualitative analysis.

We used Claude Code (Anthropic) to assist with developing R scripts for statistical analysis and reviewing written text for clarity and readability. All outputs were critically reviewed and validated, and responsibility for the correctness of the analysis and interpretations rests entirely with the researcher.

Step 1: Quantitative Cross-sectional Analysis. For each research question, we first analysed quantitative data from Q1 and Q2 separately. This allowed us to describe baseline perceptions and compare the two samples as cross-sectional snapshots, examining demographic, organisational and usage patterns. Since the Q1 and Q2 samples differ in size ($n = 158$ vs $n = 101$) and are analysed without participant matching in this step, any Q1–Q2 differences reported here reflect between-sample comparisons rather than within-participant change. Depending on the outcome, this stage included descriptive statistics, one-sample tests against midpoints, subgroup comparisons, correlations, cross-tabulations, and regression modelling (see Section 4.4.1 for test selection rationale and specific methods).

Step 2: Quantitative Longitudinal Analysis. We then restricted analyses to participants who completed both Q1 and Q2. This enabled us to examine within-participant change over the study period. Depending on the construct, this included paired Wilcoxon signed-rank tests, transition matrices and movement classifications, and mixed-effects models (see Sections 4.4.1.2 and 4.4.1.4 for specific methods). As RQ2 and RQ3 each involved multiple constructs, this two-step structure was applied separately to each component (see Section 4.4.1.5 for details). This ensured methodological consistency while respecting conceptual differences between constructs.

Step 3: Qualitative Analysis. We conducted a reflexive thematic analysis [12, 11] of the open-ended responses to understand how participants described and contextualised their experiences with AI coding assistants, complementing the quantitative results. The full qualitative methodology is described in Section 4.4.2.

4.4.1 Quantitative Analysis

All quantitative analyses were conducted in R. The packages used are listed in Appendix C.

Measurement and Scoring. Most constructs were measured with single Likert-scale items. The constructs were sufficiently straightforward, and single items helped minimise respondent burden across two questionnaires. Where a construct comprised multiple dimensions (task focus shifts across six development activities; developer experience across feedback loops, cognitive load, and flow state), each dimension was analysed independently. The exception was self-efficacy, which used the validated eight-item NGSE scale. The composite score was computed by averaging items, following standard practice for this instrument [19]. We report internal consistency (Cronbach's α) calculated on the sample on which the analysis was applied, with $\alpha \geq .70$ considered acceptable for exploratory research [53].

4.4.1.1 Predictor Hierarchy

To ensure consistency across all research questions, we organised measured variables into a *predictor hierarchy* of five levels, ordered from most stable to most

malleable in terms of individual agency, progressing from fixed demographic characteristics to the impact perceptions we aimed to explain. This guided variable selection and ordering in all analyses:

1. **Demographic Factors**: Relatively stable individual characteristics including gender and professional experience. Experience level is particularly relevant given industry questions about differential AI impacts on junior versus senior developers.

2. **Contextual/Organisational Factors**: Work environment characteristics including role, primary programming language, and company size. These are changeable but require significant effort.

3. **Tool Usage Patterns**: Behavioural choices about tool engagement, including usage frequency, usage location (work only, personal only, or both), and AI tool selection. These reflect daily choices about tool use.

4. **Tool Attitudes**: How individuals feel about AI tools, including initial impressions (Q1), anticipated disappointment if tools were removed (Q2), and primary concern about using AI tools.

5. **Impact Perceptions**: The primary outcomes for each research question: perceived productivity, developer experience, self-efficacy, skill development impact, and prompt engineering beliefs.

4.4.1.2 Statistical Methods

Descriptive Statistics. For each outcome, we computed means, standard deviations, and response category distributions for each time point. Likert-scale responses were summarised as numeric scores (1–5) and grouped into meaningful categories (e.g., less/same/more time, positive/negative/neutral) used in both descriptive reporting and the inferential tests described below. For longitudinal data, we constructed Q1-to-Q2 transition matrices, summarised as movement patterns and visualised using alluvial diagrams.

Test Selection. Single Likert items are ordinal, so we used non-parametric tests for all comparisons involving these measures. Shapiro-Wilk tests confirmed that most composite scores, including self-efficacy, also violated normality ($p < 0.05$), so we used non-parametric tests throughout.

Inferential Tests. For bipolar scales, we tested whether responses differed from the neutral midpoint (3.0) using one-sample Wilcoxon signed-rank tests; this was not applied to the unipolar importance rating (Q6.1), where the midpoint carries no theoretical meaning. For longitudinal within-participant change, we used paired Wilcoxon signed-rank tests, and for paired binary outcomes, McNemar's test. For independent group comparisons, we used Kruskal-Wallis tests for ordinal and continuous outcomes, chi-square tests for larger-sample categorical comparisons, and Fisher's exact test for smaller subgroups.

Multiple Comparison Corrections. All multiple comparison corrections used the Holm-Bonferroni method. For cross-sectional analyses, correction was applied within each time point separately, as Q1 and Q2 represent independent cross-sections. For constructs measured through multiple related items or dimensions, corrections were applied across all items within that construct: the six task focus items (RQ1), three developer experience dimensions (RQ2), and two prompt engineering dimensions (RQ4).

Effect Sizes. Effect sizes quantify the magnitude of observed differences, independent of statistical significance. For Wilcoxon tests, we report rank-biserial correlation (r_{rb}); for Kruskal-Wallis tests, epsilon-squared (ε^2); and for chi-square and Fisher's exact tests, Cramér's V. Effect sizes were interpreted using conventional thresholds via the `effectsize` R package [8].

Correlations. We used Spearman rank correlations rather than Pearson correlations throughout, reflecting the ordinal nature of our Likert-scale measures. For analyses correlating binary variables (e.g., specific AI tool usage) with continuous composite scores, we used point-biserial correlations.

4.4.1.3 Regression Modelling

We used ordinal logistic regression for Likert-scale outcomes and linear regression for continuous composites (self-efficacy), fitted separately at each time point for cross-sectional analyses. Predictor selection followed the hierarchy described in Section 4.4.1.1. Throughout, we interpret odds ratios as associations rather than causal effects, given the observational design.

Reference Levels. We selected reference levels based on theoretical relevance, sufficient group size, and interpretive clarity, keeping them consistent across all analyses. We used "Senior" (16+ years experience) as the reference for *experience*, facilitating interpretation of junior-senior differences given industry concerns about AI coding assistants' differential impact on less experienced developers. Other reference levels were: "Man" for *gender*, "Backend" for *role*, "Web" for *programming language group*, and "Medium" for *company size*.

Category Exclusions. We omitted categories with very small cell sizes from regression analyses: for *gender*, participants who identified as non-binary, self-described their gender, or chose not to disclose; for *experience* and *company size*, participants with missing values; and for *role* and *programming language*, participants in the "Other" category (rare but valid responses grouped together due to low frequency). These exclusions prioritise statistical reliability over comprehensiveness, acknowledging that findings may not generalise to underrepresented groups. Missingness was minimal across Likert-scale items (Q1: <1%, Q2: <0.1%), supporting complete-case analysis without risk of systematic bias.

4.4.1.4 Mixed-Effects Modelling

For longitudinal analyses of the matched cohort, we used Cumulative Link Mixed Models (CLMMs) with random intercepts for participants to account for repeated measures on ordinal outcomes. For self-efficacy, analysed as a continuous composite, we used linear mixed-effects models. We adopted a progressive model-building approach across three nested models to evaluate the contribution of different predictor categories given our sample sizes ($n = 85$–91 matched participants, varying due to missing data on specific outcomes or predictor exclusions). Organisational factors (role, programming language, company size) were excluded to maintain parsimony given sample size constraints.

Model Structure. Following the predictor hierarchy (Section 4.4.1.1), Model 1 (M1) included time (coded 0 = Q1, 1 = Q2) and demographic factors (gender, experience). Model 2 (M2) added tool usage patterns (usage frequency). Model 3 (M3) added impact perceptions from other research questions to examine cross-construct relationships (e.g., whether self-efficacy predicts productivity, or whether developer experience predicts skill development perceptions), with the specific factors depending on the construct being modelled.

Standardisation. Ordinal predictors (usage frequency, productivity, skill development, prompt engineering perceptions) were converted to numeric scales. All numeric predictors, including the self-efficacy composite, were z-scored (mean = 0, SD = 1) using the grand mean and pooled standard deviation across both time points, placing predictors on comparable scales. For CLMMs, the resulting odds ratios represent the multiplicative change in odds of a higher response category per one standard deviation increase in the predictor. For linear models (self-efficacy only), coefficients represent the expected change in the outcome per one standard deviation change in the predictor. We report 95% confidence intervals where space permits; where omitted for table clarity, they are available on request.

Model Comparison. We compared models using AIC (Akaike Information Criterion), where lower values indicate better fit relative to model complexity. We also report ICC (Intraclass Correlation Coefficient) to quantify the proportion of variance attributable to between-person differences.

4.4.1.5 RQ-Specific Analytical Extensions

Each research question involved additional analyses tailored to its constructs, summarised below.

RQ1 (Task Focus Shifts). The six tasks were analysed independently. We also constructed two theoretically-motivated task groupings based on the Vee model [65]: *creation tasks* (designing, writing, refactoring) and *verification tasks* (reviewing, testing, debugging). These are theoretical distinctions rather than psychometric subscales, so we retain them regardless of internal consistency. Detailed predictor analyses focused on the tasks showing the most divergent patterns (see Section 5.2). We also tested whether time saved from writing code was reinvested in higher-level design work.

RQ2 (Developer Experience and Productivity). For developer experience mixed-effects models, M3 psychological predictors were self-efficacy and perceived importance of prompt engineering (not productivity, since the DevEx framework posits that developer experience drives productivity rather than the reverse). We identified a *Negative* cohort (participants reporting worsened experience on at least one dimension) and tracked transitions over time. We examined the developer experience–productivity relationship through Spearman correlations and

ordinal logistic regression, and used cross-lagged regression to test temporal precedence between self-efficacy and productivity. Given the strong association between these constructs, we assessed construct distinctness by comparing their correlation against the 0.85 threshold commonly used to indicate problematic overlap [22].

RQ3 (Skill Development and Self-Efficacy). Self-efficacy was analysed at both the item level and as a composite score. The Reliable Change Index [45] was used to distinguish reliable individual change in self-efficacy from measurement error. We examined how skill development and self-efficacy related through correlations and by including skill-development impact as a predictor in self-efficacy models.

RQ4 (Prompt Engineering Perceptions). We categorised each dimension into Low (1–2), Moderate (3), or High (4–5) and crossed them to form five perception profiles: *Additive* (High Importance + Low Replacement), *Substitutive* (High Importance + High Replacement), *Sceptic* (Low Importance + Low Replacement), *Displaced* (Low Importance + High Replacement), and *Moderate* (Moderate on either dimension). Two profiles were of particular theoretical interest: Additive, reflecting a view that prompt engineering augments traditional skills, and Substitutive, reflecting a view that it will partially replace them. For the matched cohort, we examined profile stability and classified transition patterns based on joint changes across both dimensions.

4.4.2 Qualitative Analysis

We used reflexive thematic analysis following Braun and Clarke [12, 11] to analyse the open-ended responses. The researcher's position as a practising software engineer shaped interpretation; this was examined through reflexive memoing, actively seeking counter-examples, periodic reflection on how domain expertise might influence interpretation, and iterative discussions with the research supervisor throughout the analysis process (see Section 4.4.2.5). The phases of Braun and Clarke's approach are detailed in the subsections that follow.

4.4.2.1 Open-Ended Questions

The open-ended questions associated with each research question were analysed across both time points. For RQ1 (Q3.6) and RQ3 (Q5.3), the same question was

asked in both questionnaires; for RQ2 (Q4.3) and RQ4 (Q6.3), the wording was modified in Q2 to elicit richer longitudinal reflections.

4.4.2.2 Phase 1: Familiarisation with the Data

We read through all open-ended responses across both questionnaires multiple times, actively searching for meanings and patterns rather than passively absorbing content. Responses varied considerably in depth and style, from detailed reflective accounts to brief lists and single-sentence answers, but most were substantive. These initial readings gave us a sense of the diversity of perspectives among professional software engineers and informed preliminary ideas that we carried into the coding phase.

4.4.2.3 Phase 2: Generating Initial Codes

Using NVivo, we followed an inductive approach, generating codes from participant responses rather than applying a pre-existing framework. Codes were applied across all open-ended questions, as participants frequently discussed similar themes regardless of which question prompted them, making a shared codebook more practical than separate per-question codebooks.

To illustrate the coding process, consider the following response to Q3.6 ("In what ways has using an AI coding assistant changed your workflow?"):

> *"I find them helpful when working with languages and technologies that I am less familiar with, or use less frequently. They are often wrong but can help you move more quickly; A wrong-but-close answer can give you a jump to the next thing to search for and solve the problem."* (P36, Q1)

Rather than coding the response as a single unit, we identified distinct segments conveying different ideas. The opening clause about working with unfamiliar technologies was coded as *Capability Expansion*; the observation that AI helps "move more quickly" was coded as *Speed & Automation*; and the framing of imperfect output as enabling forward momentum was coded as *Confidence Boost*.

To manage the growing number of codes, we grouped them into three top-level categories (*Technical Capability & Practice*, *Professional & Collaborative Practices*, and *Mindset, Confidence & Behaviour*); these were used solely for organisation during coding and did not carry forward into theme development. The

codebook was iteratively refined throughout the coding process: new codes were added when existing codes did not adequately capture a segment, similar codes were merged to reduce redundancy, and code definitions were clarified to improve consistency.

For some open-ended questions, we applied an additional layer of cross-cutting valence codes to capture the evaluative dimension of participants' accounts. For Q4.3, each segment was coded with a valence reflecting the question wording: *helpful* or *hindrance* in Q1, and *unexpected benefit* or *unexpected challenge* in Q2. For Q6.3 in Q2, segments were additionally coded for degree of evolution: *no evolution*, *some evolution*, or *significant evolution*.

4.4.2.4 Phases 3–5: Generating, Reviewing, and Defining Themes

Following the coding phase, we developed themes at the research-question level rather than for each open-ended question, examining coded responses from both questionnaires to identify themes spanning the longitudinal period while revealing where accounts differed between time points. Code frequency summaries helped orient this process, but themes were generated based on conceptual significance rather than prevalence. We reviewed candidate themes against the original responses and refined them through multiple iterative passes until they provided a coherent interpretation of participants' accounts. Some codes contributed to multiple themes depending on contextual framing, and where themes were multi-faceted, sub-themes captured distinct dimensions. Where similar experiences surfaced across multiple research questions, themes were shaped to reflect the specific lens of each question rather than treated as identical.

To illustrate theme generation, for RQ3, codes including *Capability Expansion*, *Conceptual Learning*, and *Speed & Automation* frequently co-occurred in responses about engaging with unfamiliar technologies; these were combined into a theme about how AI tools expand developers' practical reach.

4.4.2.5 Trustworthiness

All coding was conducted by the researcher. The codebook and its application in NVivo were reviewed with the research supervisor through several iterations of feedback and discussion, with attention to whether codes adequately captured the range of participant experiences. Ambiguous segments were flagged during initial coding and revisited after the full dataset was coded to ensure consistent application. Candidate themes were also reviewed as they developed. This approach

is consistent with reflexive thematic analysis, where trustworthiness derives from transparency and reflexive engagement rather than procedural checks such as independent coder agreement.

Chapter 5

Results

This chapter presents the findings from both questionnaires. To provide context for the analyses that follow, we begin by describing which AI coding assistants participants reported using, their usage patterns, and their attitudes and concerns about using these tools. We then address each of the study's four research questions in turn. Participant demographics and sample characteristics are described in Chapter 4.

5.1 Study Context

5.1.1 AI Coding Assistants in Use

Participants reported using a wide range of AI coding assistants, with 37 unique tools mentioned across both time points (Appendix D), including 29 at Q1 ($n = 158$) and 24 at Q2 ($n = 101$). GitHub Copilot and ChatGPT were the clear front-runners, both at 70% adoption at Q1 and 58% at Q2 (Figure 5.1).

Among matched participants ($n = 95$), the tools in use evolved considerably over the study period. The number of distinct tools increased from 18 to 24, 82% changed their tool combinations, and the mean number of tools per participant increased from 1.9 ($SD = 0.8$) to 2.9 ($SD = 1.6$).

Table 5.1 shows how tool usage changed among matched participants. Three tools showed statistically significant growth: Gemini, Claude, and Microsoft Copilot. Two tools not mentioned at Q1 emerged at Q2: Claude Code and Windsurf. The two dominant tools, GitHub Copilot and ChatGPT, showed net losses among matched participants, though these changes were not statistically significant.

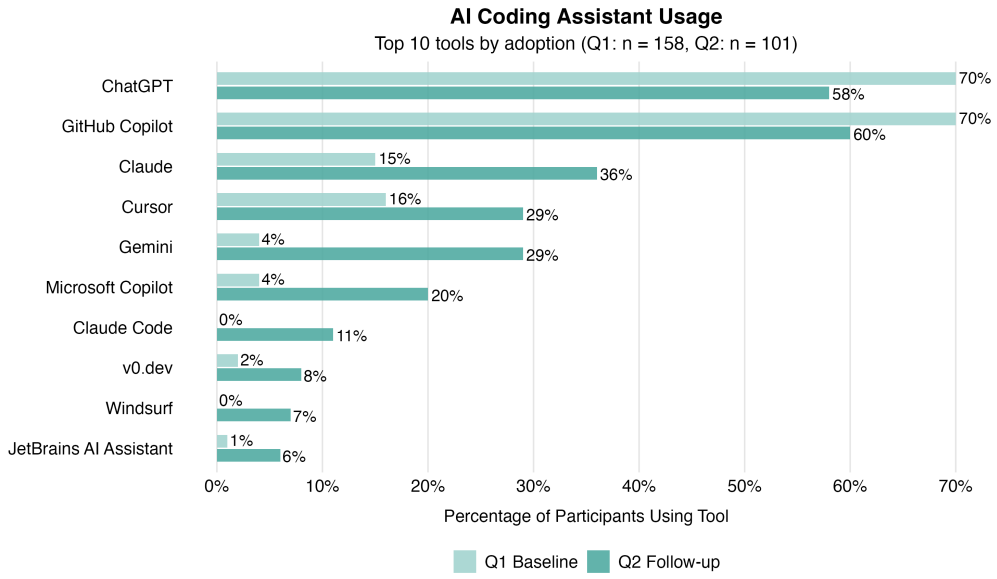


Figure 5.1: AI coding assistant adoption rates at Q1 and Q2. Top 10 tools by maximum usage at either time point shown.

Table 5.1: AI Tool Adoption Changes Among Matched Participants ($n = 95$)

Tool	**Q1**	**Q2**	**Adopted**	**Dropped**	**Net**	p_{adj}
Gemini	2	26	25	1	+24	**<0.001*****
Claude	15	35	25	5	+20	**0.013***
Microsoft Copilot	6	20	15	1	+14	**0.027***
Claude Code†	0	11	11	0	+11	0.057
Cursor	18	28	14	4	+10	0.644
v0.dev	1	8	7	0	+7	0.490
Windsurf†	0	7	7	0	+7	0.490
GitHub Copilot	63	56	6	13	−7	1.000
ChatGPT	67	57	4	14	−10	0.644

Note. McNemar tests with Holm-Bonferroni correction. Tools shown had net change ≥ 5 or were new tools with ≥ 5 users at Q2.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
†New tool not mentioned at Q1.

5.1.2 Usage Frequency and Location

Over half of the participants in our study used AI coding assistants daily at both Q1 and Q2, and this pattern remained stable across time points (paired Wilcoxon signed-rank, $p = 0.17$, $n = 95$ matched; Figure 5.2). The majority used them in both work and personal contexts (65% at Q1, 70% at Q2), with work-only usage less common (27% at Q1, 24% at Q2) and personal-only usage rare (8% at Q1, 6% at Q2).

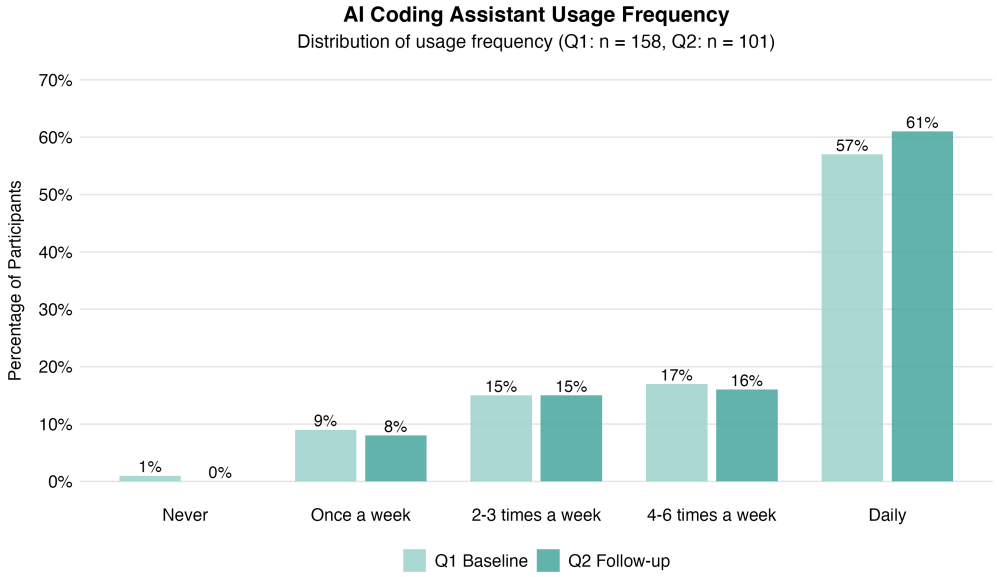


Figure 5.2: Distribution of AI coding assistant usage frequency at Q1 and Q2.

5.1.3 Attitudes Towards AI Coding Assistants

Participants' attitudes towards AI coding assistants were captured through two measures: initial impressions at Q1 (Figure 5.3) and anticipated disappointment if tools were removed at Q2 (Figure 5.4). Initial impressions were predominantly positive, while anticipated disappointment was more evenly distributed. Among matched participants, the two measures showed a moderate positive correlation ($\rho = 0.30$, $p = 0.003$).

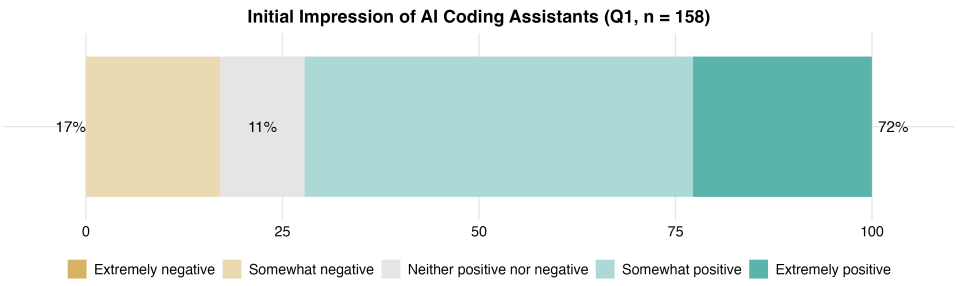


Figure 5.3: Distribution of initial impressions of AI coding assistants at Q1.

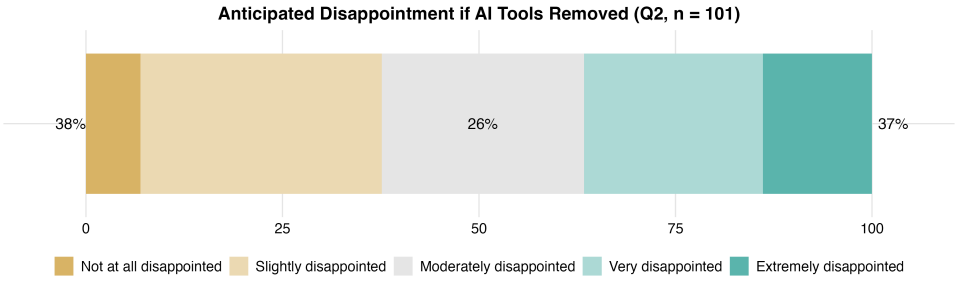


Figure 5.4: Distribution of anticipated disappointment if AI coding assistants were removed at Q2.

5.1.4 Concerns About Using AI Coding Assistants

Participants ranked seven concerns about AI coding assistants from most to least concerning. Quality was the dominant primary concern at both time points, though it declined from 44% to 36%, followed by security. The most notable shift was in maintainability, which rose from 3% to 19% as a primary concern between Q1 and Q2 (Figure 5.5). Paired Wilcoxon signed-rank tests confirmed maintainability was the only statistically significant change in ranking ($n = 89$ matched, $p = 0.003$, moderate effect).

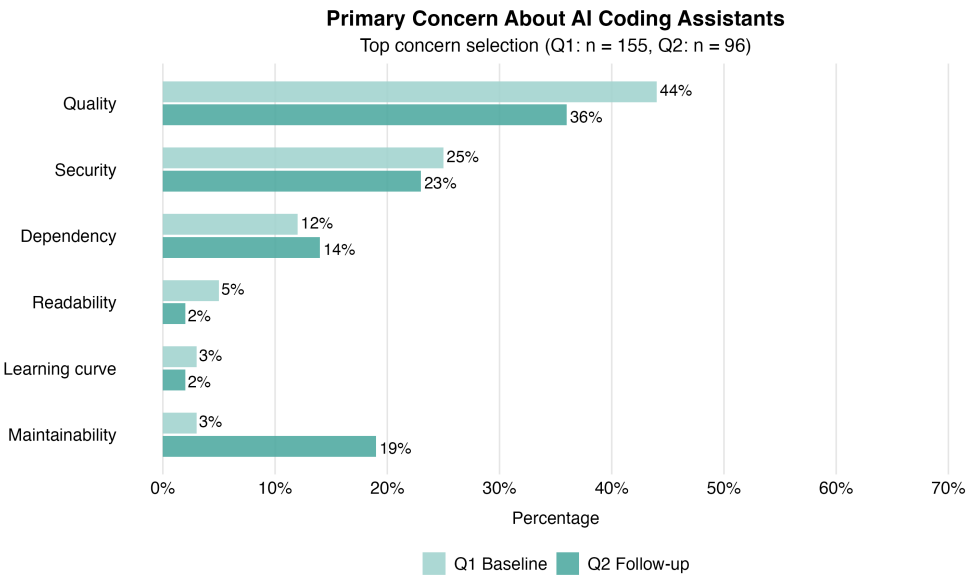


Figure 5.5: Primary concern distribution at Q1 and Q2. "Other" excluded from analysis.

5.2 RQ1: Task Focus Shifts

This section addresses RQ1: *How does the introduction of AI coding assistants shift software engineers' perceived focus across various development tasks?* We analyse perceived time reallocation across six core development activities: designing, writing, refactoring, testing, debugging, and reviewing code.

5.2.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete data across all six tasks (Q1: $n = 147$; Q2: $n = 101$).

Composite Overview

Internal consistency was modest (Cronbach's $\alpha = 0.70$ at Q1, $\alpha = 0.55$ at Q2), reflecting conceptually distinct activities rather than a unidimensional construct. Nevertheless, the composite showed perceived time savings at both time points (Q1: $M = 2.65$, $SD = 0.57$; Q2: $M = 2.65$, $SD = 0.52$). One-sample Wilcoxon signed-rank tests confirmed that these values were below the neutral midpoint (both $p < 0.001$; $r_{rb} = -0.65$ at Q1, $r_{rb} = -0.70$ at Q2; both large effects). A majority of respondents reported spending less time overall across the six tasks at both time points (composite < 3.0): 69% at Q1 ($n = 102$) and 69% at Q2 ($n = 70$), with 12–13% neutral and 18–19% reporting more time overall. Group proportions were stable across time ($\chi^2(2) = 0.09$, $p = 0.954$, Cramér's $V = 0.02$).

Task-level Patterns

Table 5.2: Task-level Time Allocation Means and Standard Deviations by Time Point

Task	**Q1** ($n = 147$)		**Q2** ($n = 101$)		
	M	***SD***	***M***	***SD***	**ΔM**
Writing code	2.10	0.82	1.93	0.75	−0.17
Refactoring code	2.49	0.98	2.42	1.03	−0.07
Testing	2.54	0.97	2.71	0.97	+0.17
Designing	2.88	0.79	2.84	0.94	−0.04
Debugging	2.87	0.93	2.87	0.84	0.00
Reviewing code	3.03	0.88	3.15	1.02	+0.12

Note. Scale: 1 = much less time, 3 = about the same, 5 = much more time. ΔM = Q2 mean − Q1 mean. Tasks ordered by Q1 mean.

Table 5.2 presents means and standard deviations for each task at both time points. Writing code showed the strongest perceived time reduction at both time

points ($M = 2.10$ at Q1, $M = 1.93$ at Q2), with 82% of participants reporting less time by Q2 and only 2% reporting more. Refactoring code and testing also showed means below neutral, though testing moved upward between time points (from 2.54 to 2.71). In contrast, designing and debugging remained stable near neutral. Reviewing code was the only task with means above the neutral midpoint at both time points (Q1: 3.03, Q2: 3.15).

Figure 5.6 provides an overview of the response distributions across all six tasks at both time points.

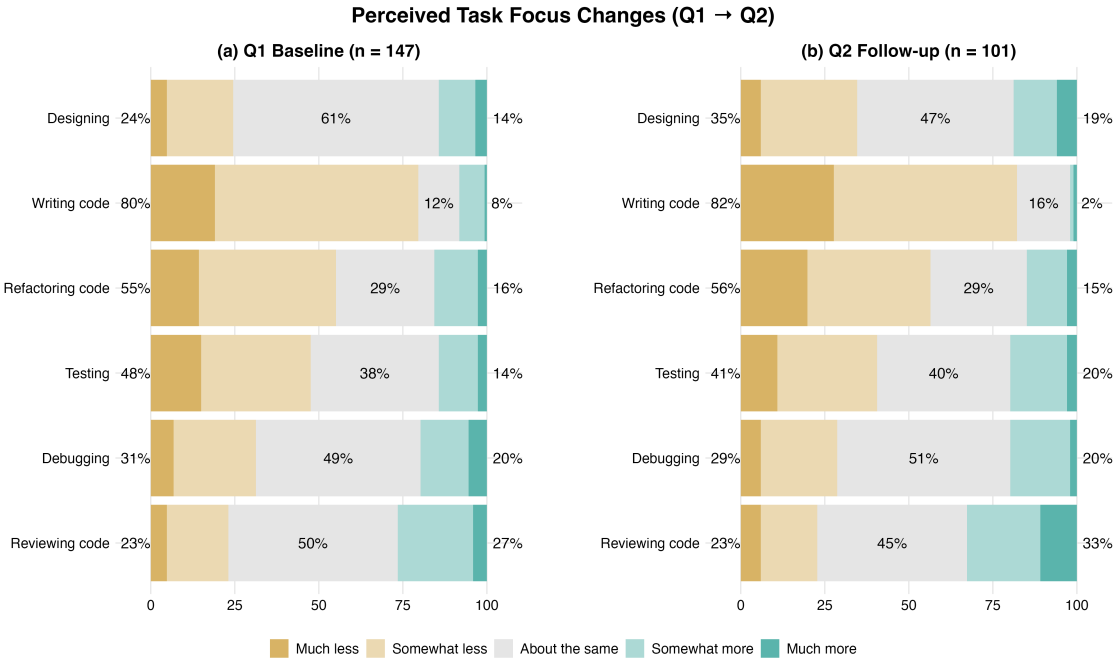


Figure 5.6: Task focus shifts across time points.

Inter-Task Correlations

Table 5.3 presents Spearman correlations between all task pairs at each time point. A positive correlation indicates that perceived time changes on two tasks shifted together; a negative correlation indicates they shifted in opposing directions.

At Q1, most task pairs showed significant positive correlations (10 of 15 pairs), with the strongest between writing and refactoring code ($\rho = 0.40$) and between refactoring and reviewing code ($\rho = 0.40$). By Q2, correlations weakened substantially and only one pair remained significant: reviewing and testing ($\rho = 0.46$).

Table 5.3: Inter-Task Correlations in Perceived Task Focus Shifts

Task A	**Task B**	**Q1** ρ	**Q2** ρ
Writing code	Refactoring code	**0.40*****	0.20
Refactoring code	Reviewing code	**0.40*****	0.26
Reviewing code	Testing	**0.38*****	**0.46*****
Refactoring code	Debugging	**0.37*****	0.14
Refactoring code	Testing	**0.33*****	0.15
Reviewing code	Debugging	**0.32*****	0.21
Writing code	Testing	**0.28*****	0.08
Writing code	Debugging	**0.27****	0.11
Designing	Refactoring code	**0.23***	0.17
Testing	Debugging	**0.23***	0.23
Designing	Debugging	0.18	−0.02
Writing code	Reviewing code	0.16	−0.05
Designing	Reviewing code	0.13	0.16
Designing	Writing code	0.07	0.11
Designing	Testing	0.04	0.15

Note. Spearman ρ values for Q1 ($n = 147$) and Q2 ($n = 101$). Pairs ordered by Q1 correlation strength.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$ (Holm-Bonferroni corrected).

The writing-reviewing correlation, which showed a small positive association at Q1 ($\rho = 0.16$, $p = 0.060$), became essentially zero by Q2 ($\rho = -0.05$).

5.2.1.1 Writing and Reviewing Code

Given that writing code showed the strongest reductions while reviewing code was the only task with means above neutral at both time points, we examined this pair in detail as the most divergent patterns within the six tasks.

Predictors of Writing and Reviewing Code Perceptions

We fitted ordinal logistic regression models to identify factors associated with perceived shifts in time spent on writing and reviewing code when using AI (predictors per Section 4.4.1.1). Table 5.4 shows the results.

At Q1, writing code perceptions showed significant associations with gender, role, and company size: women reported spending more time than men, fullstack developers more than backend developers, and small-company developers less than medium-company developers. For reviewing code, frontend developers reported spending more time than backend developers.

By Q2, the Q1 patterns had largely dissipated and new associations emerged. Writing code perceptions showed significant associations with experience, role, and usage frequency: mid-career developers reported spending more time than

Table 5.4: Ordinal Logistic Regression of Perceived Shifts in Time Spent on Writing and Reviewing Code (Q1 vs Q2)

Predictor	**Writing Code**		**Reviewing Code**	
	OR Q1	**OR Q2**	**OR Q1**	**OR Q2**
Gender (Reference: Man)				
Woman vs Man	**6.01****	2.28	0.39	**0.18***
Experience Level (Reference: Senior)				
Junior vs Senior	2.00	2.28	0.70	1.86
Mid-Career vs Senior	1.39	**2.86***	1.11	0.98
Role (Reference: Backend)				
Frontend vs Backend	3.39	1.68	**6.51***	0.75
Fullstack vs Backend	**3.72***	1.23	1.60	1.46
Manager vs Backend	1.89	0.48	0.72	2.07
Mobile vs Backend	2.10	0.80	1.43	4.12
Specialised vs Backend	0.68	**0.08***	1.28	1.20
Programming Language (Reference: Web)				
Compiled vs Web	1.32	1.85	0.94	0.89
Interpreted vs Web	1.35	1.10	2.25	0.42
Company Size (Reference: Medium)				
Small vs Medium	**0.20*****	0.41	0.76	0.52
Large vs Medium	0.86	0.68	0.50	**0.28***
Very Large vs Medium	1.69	0.39	0.60	0.34
Usage Location (Reference: Both)				
Work Only vs Both	0.93	0.83	0.94	2.03
Personal Only vs Both	0.27	0.36	0.90	2.04
AI Usage Frequency	0.73	**0.44****	0.98	1.03

OR = Odds Ratio for moving to a higher response category.
Significance: $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.
Q1: Writing $n = 145$, Reviewing $n = 140$; Q2: $n = 94$ for both. McFadden R^2 range: 0.041–0.133.

senior developers, specialised roles substantially less than backend developers, and more frequent AI users reported greater time savings. For reviewing code, women reported spending less time than men and large-company developers less than medium-company developers. No predictors showed consistent effects across both time points.

No Significant Associations Found for AI Tool Selection

We examined whether specific AI tool choices associated with perceived task focus shifts using Fisher's exact tests with Holm-Bonferroni correction. No tools reached significance at either time point for writing or reviewing code.

Attitudes Towards AI Tools Show Selective Associations

Table 5.5 shows Spearman correlations between attitudes towards AI tools and perceived time shifts for writing and reviewing code.

Table 5.5: Spearman Correlations Between Attitudes and Perceived Time Shifts (Writing and Reviewing Code)

Task	**Q1 Initial Impressions**		**Q2 Anticipated Disappointment**	
	ρ	p_{adj}	ρ	p_{adj}
Writing code	-0.27^{**}	0.005	-0.42^{***}	<0.001
Reviewing code	-0.23^{*}	0.018	-0.05	1.000

Negative correlations indicate more positive attitudes associated with reporting less time on the task.
Significance: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$. Holm-Bonferroni corrected within each time point.
Q1 $n = 147$; Q2 $n = 101$.

At Q1, initial impressions showed significant negative associations with both tasks, with more positive impressions associated with greater perceived time savings. At Q2, only writing code remained significantly associated with anticipated disappointment ($\rho = -0.42$, moderate effect), while reviewing code showed no significant association.

Primary concerns showed no significant associations with writing or reviewing time at either time point (Kruskal-Wallis tests with Holm-Bonferroni correction).

Focused Analysis: Participants Reporting Less Time Writing Code

Having examined predictors across all participants, we narrowed our focus to those who reported spending less time writing code, as this represents the largest group and most common response pattern. Within this subset of 117 participants at Q1, reviewing time was distributed across all categories: 46% reported

no change in reviewing time, 26% reported more time reviewing, and 28% reported less time reviewing. By Q2, patterns shifted among the 83 participants reporting less time writing code: the proportion reporting more time reviewing increased to 36%, while those reporting less time reviewing declined to 20% (Figure 5.7).

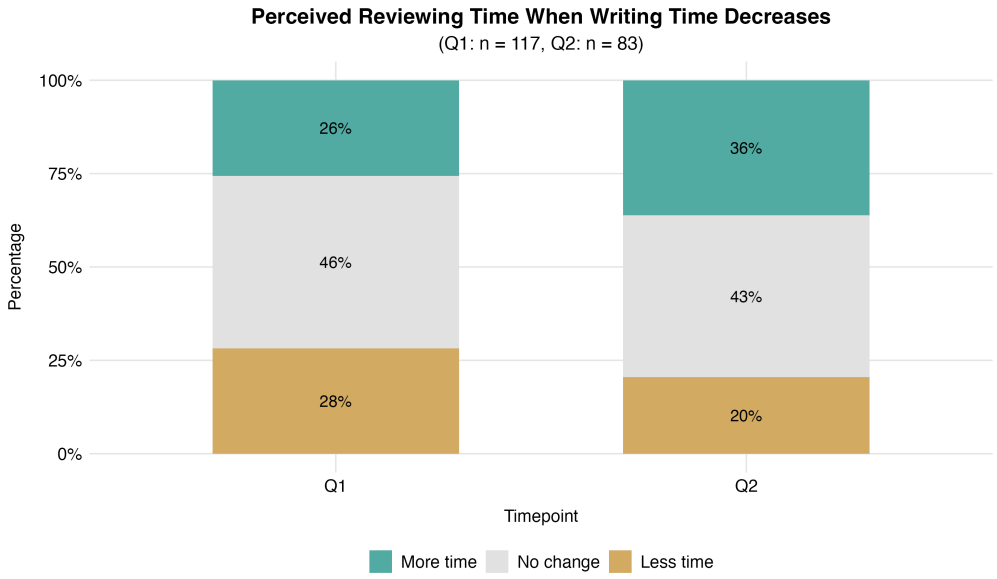


Figure 5.7: Distribution of reviewing time allocation among participants who reported spending less time writing code.

5.2.2 Longitudinal Change in Task Focus

Next, we examined how task focus perceptions changed within participants ($n = 88$ matched participants).

Aggregate Stability Across Tasks

Paired Wilcoxon signed-rank tests with Holm-Bonferroni correction showed no statistically significant changes, though effect sizes varied from very small to moderate (Table 5.6).

Writing code showed the largest effect in the expected direction ($r_{rb} = -0.35$, moderate effect), while testing and reviewing showed small increases ($r_{rb} = 0.29$ and $r_{rb} = 0.26$ respectively). These directional patterns, though not statistically significant, inform the subsequent analyses of task reallocation.

Table 5.6: Longitudinal Wilcoxon Signed-Rank Tests: Changes in Task Focus Perceptions Within Matched Participants ($n = 88$)

Task	**Q1**		**Q2**			**Test Statistics**		
	M	***SD***	***M***	***SD***	**ΔM**	***V***	p_{adj}	r_{rb}
Designing	2.94	0.72	2.89	0.94	-0.06	595.5	1.000	-0.10
Writing code	2.10	0.80	1.92	0.76	-0.18	255.0	0.263	-0.35
Refactoring code	2.48	0.93	2.39	1.06	-0.09	688.0	1.000	-0.11
Reviewing code	2.93	0.83	3.16	1.04	+0.23	967.0	0.333	0.26
Testing	2.52	0.92	2.77	0.98	+0.25	1030.5	0.263	0.29
Debugging	2.84	0.93	2.84	0.79	0.00	556.0	1.000	-0.01

ΔM = Mean change (Q2 - Q1). V = Wilcoxon signed-rank test statistic. r_{rb} = rank-biserial correlation.

Transition Patterns

While aggregate statistics showed minimal change, individual transition patterns revealed substantial variation beneath these stable averages. Here we examine the three tasks showing the clearest directional effects: writing code (downward) and testing and reviewing code (upward).

Table 5.7: Transition Patterns for Key Tasks ($n = 88$)

Task	**Moved Toward Less**	**Stayed Stable**	**Moved Toward More**
Writing code	30% ($n = 26$)	56% ($n = 49$)	15% ($n = 13$)
Testing	22% ($n = 19$)	36% ($n = 32$)	42% ($n = 37$)
Reviewing code	23% ($n = 20$)	38% ($n = 33$)	40% ($n = 35$)

Percentages may not sum to 100% due to rounding.

Table 5.7 confirms divergent patterns: writing code showed high stability (56%) with continued downward drift (30% moved toward less time), while testing and reviewing showed upward movement (42% and 40% toward more time). Joint analysis revealed that only 8% simultaneously reduced writing time while increasing reviewing time, likely reflecting a floor effect: 81% had already reported spending less time writing code at baseline.

Creation-to-Verification Shift Analysis

Using the creation and verification task groupings defined in Section 4.4.1.5, we examined whether each task group changed over time and whether the balance between them shifted. Internal consistency was low (Creation: $\alpha = 0.40$–0.49; Verification: $\alpha = 0.56$–0.57). Table 5.8 presents the results.

Neither creation nor verification tasks changed significantly in isolation ($p_{adj} = 0.092$ and 0.060 respectively). However, the balance between task groups showed

Table 5.8: Task Group-Level Wilcoxon Signed-Rank Tests: Creation vs Verification Task Group Averages ($n = 88$)

Task Group	**Q1**		**Q2**			**Test Statistics**		
	M	SD	M	SD	ΔM	V	p_{adj}	r_{rb}
Creation	2.51	0.58	2.40	0.64	-0.11	814.5	0.092	-0.24
Verification	2.77	0.66	2.92	0.69	+0.16	1527.5	0.060	0.30
Balance (V-C)	0.26	0.58	0.53	0.83	+0.27	2365.5	0.006	**0.39****

Each group score is the mean of its three constituent task scores.
ΔM = Mean change (Q2 - Q1). Balance = Verification - Creation. V = Wilcoxon signed-rank test statistic. r_{rb} = rank-biserial correlation.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$. Holm-Bonferroni corrected.

a significant shift toward verification ($p_{adj} = 0.006$, $r_{rb} = 0.39$, moderate effect), with the mean balance score increasing from 0.26 to 0.53.

Figure 5.8 shows individual variation beneath the aggregate shift. The largest group (42%) showed stability on at least one dimension, with the remaining 58% distributed across the four quadrants. Of these, the predicted pattern (decreased creation, increased verification) was the most common at 20%, followed by both dimensions increasing (17%) and both decreasing (15%).

Shift Toward Higher-Level Design?

A common suggestion is that time saved from code generation could be reinvested in higher-level design work. However, both designing and writing code trended toward decreased time allocation ($r_{rb} = -0.10$ and -0.35 respectively), opposite to this prediction. We examined whether individual-level changes in these tasks were related.

Figure 5.9 shows that only 9% of participants exhibited the predicted pattern (upper-left quadrant). Most clustered near the origin, with both tasks trending toward decreased time. Fisher's exact test confirmed no association between design and writing transitions ($p = 0.598$, Cramér's $V = 0.07$), and the correlation between changes was very small ($\rho = 0.07$, $p = 0.509$).

Mixed-Effects Models of Task Focus Shifts

To examine factors explaining individual differences in task allocation perceptions, we applied mixed-effects modelling to three tasks—Testing, Writing code, and Reviewing code—selected for their directional effects observed in the aggregate analysis above (see Section 4.4.1.4 for model structure).

Table 5.9 shows the progressive model results. Time point and experience level were not significant for any task. Productivity perceptions were the most

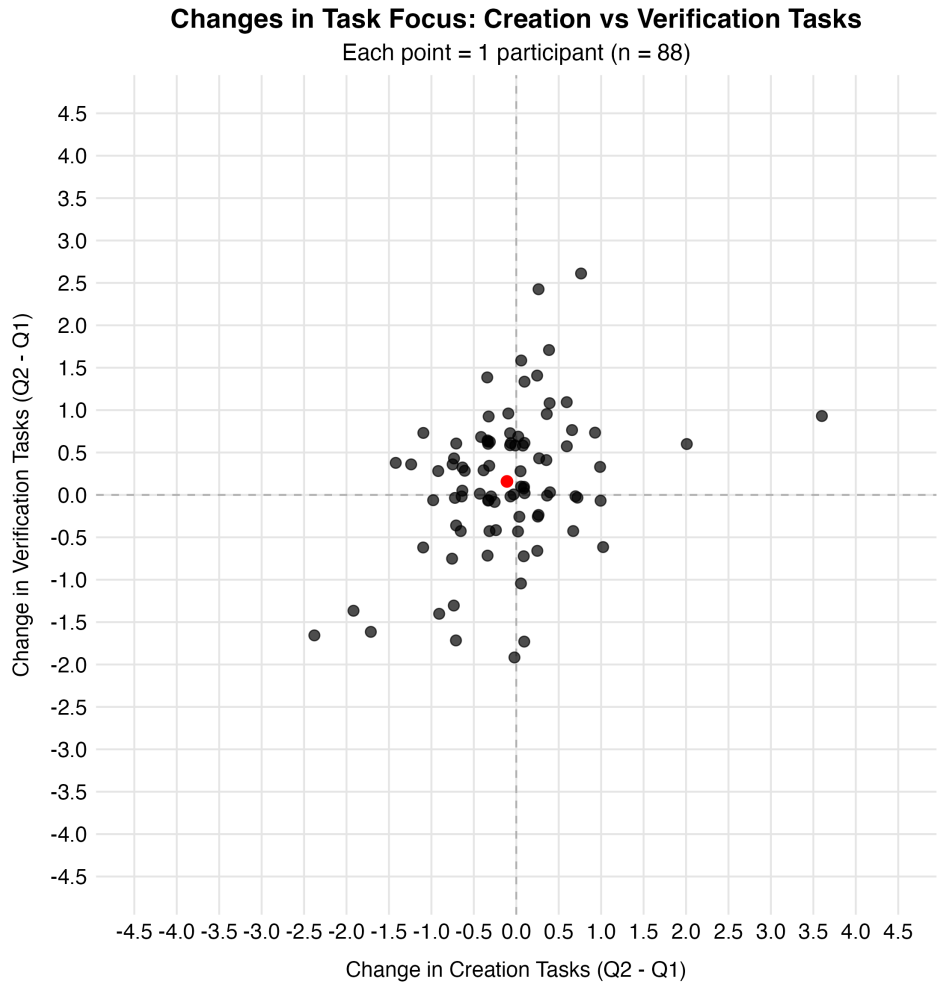


Figure 5.8: Individual changes in creation versus verification task group averages. Red point indicates mean change. Upper-left quadrant represents the predicted pattern of decreased creation and increased verification time.

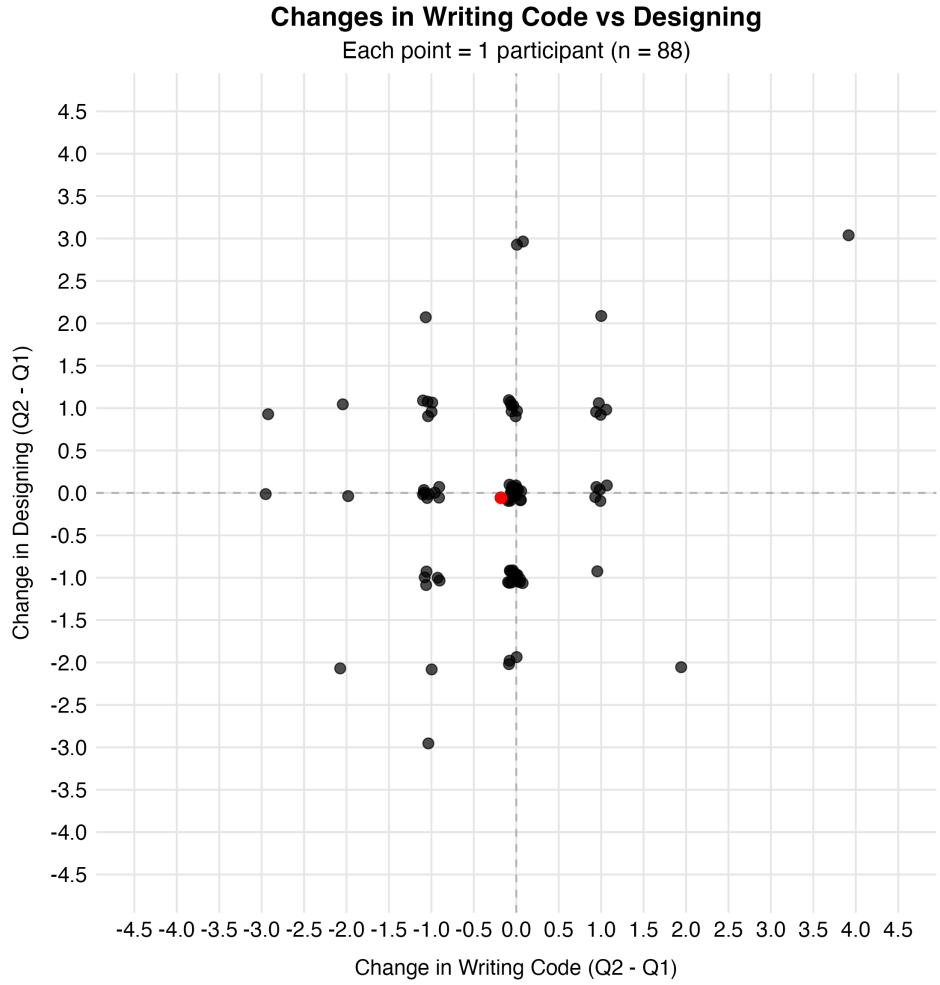


Figure 5.9: Individual changes in writing code versus designing. Red point indicates mean change. Upper-left quadrant represents the predicted pattern of decreased writing and increased designing time.

Table 5.9: Cumulative Link Mixed Model Analysis of Task Focus Predictors ($n = 85$)

	Testing			**Writing Code**			**Reviewing Code**		
Predictor	**M1**	**M2**	**M3**	**M1**	**M2**	**M3**	**M1**	**M2**	**M3**
Time (6-month change)	1.70	1.72	1.55	0.62	0.65	0.62	1.56	1.52	1.64
Gender: Woman vs Man	0.93	0.93	0.88	**4.38***	**4.31***	3.09	**0.31***	**0.32***	**0.25****
Exp: Junior vs Senior	1.58	1.65	1.64	1.32	1.78	3.26	0.82	0.71	0.96
Usage Frequency		0.94	1.13		**0.63***	0.75		1.28	1.35
Self-Efficacy			1.37			0.63			**0.62***
Productivity			**0.44*****			**0.57***			1.08
Model Fit									
AIC	461.8	463.7	452.5	373.8	370.2	354.0	470.7	470.7	468.4
ICC (%)	20.7	20.6	16.6	30.0	27.8	18.5	17.0	16.7	14.7

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Productivity.
Values are odds ratios (OR). OR > 1 (< 1) indicates higher (lower) odds of reporting more time spent.
Continuous predictors z-scored. Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
2 time points (170 observations).

consistent predictor: participants who perceived greater productivity gains reported spending less time on both testing and writing code. For reviewing code, a different pattern emerged; men were more likely than women to report spending additional time on review, and lower self-efficacy was also associated with reporting more review time. Women were also more likely than men to report spending additional time writing code (significant in both M1 and M2), but this effect weakened once psychological factors were added in M3.

Adding psychological factors improved model fit for all three tasks, with the largest improvements for writing code and testing. Individual differences (ICC) accounted for 15–30% of response variance, decreasing as predictors were added.

5.2.3 Qualitative Perspectives on Workflow Changes

Reflexive thematic analysis of responses to Q3.6, describing how AI coding assistants had changed participants' workflows, identified four themes capturing changes in their interactions with code, tools, and information sources.

Theme 1: Compressing routine creation work. Participants across both time points consistently described AI coding assistants as reducing the effort required for routine implementation tasks. This theme aligns closely with the substantial quantitative reductions observed in writing code and, to a lesser extent, refactoring and test scaffolding. At both time points, engineers reported typing less, delegating boilerplate and scaffolding to the assistant, and accelerating the creation of "first-draft" implementations.

Sub-theme 1A: Reducing manual implementation effort. Participants at both time points highlighted how AI tools helped them generate boilerplate, small functions, or repetitive structures. As one participant explained, *"I spend less time writing repetitive and boilerplate code"* (P25, Q1) and *"Less time spent on the daily grind of coding"* (P37, Q1). Others noted that they now typed less: *"Reduced my keystrokes. I mostly use AI only for line completion and most of the time I only have to type less than half a line before AI figures out what I want"* (P44, Q1). These descriptions remained stable across time, with participants in Q2 reporting similar experiences, for example: *"Mostly, the routine tasks like coding and testing now require less manual effort."* (P98, Q2) and *"Saves a ton of pure coding time with essentially context-aware autocomplete."* (P147, Q2).

Sub-theme 1B: Supporting small-scale debugging and error handling. Participants described targeted improvements in resolving smaller issues, such as obtain-

ing explanations of error messages or addressing minor test failures. This pattern appeared consistently across Q1 and Q2. As one participant noted, *"I give the error message directly to ChatGPT and get a primer on what could be the possible cause of the issue"* (P135, Q1). Another wrote, *"Troubleshooting and debugging become faster."* (P91, Q1). Longitudinally, the types of examples remained similar, for example: *"Mostly still using it for debugging purposes."* (P154, Q2).

Theme 2: Rerouting information seeking and comprehension into AI interactions. Participants described noticeable changes in how they acquired information, understood unfamiliar code and learned new concepts. Rather than searching documentation or external websites, many relied on asking AI tools for explanations, summaries, and examples. This theme appeared strongly at Q1 and remained clear at Q2, although Q2 participants often described these behaviours as normalised rather than novel.

Sub-theme 2A: From search engines to real-time assistance. Developers increasingly used AI as the first step when encountering errors, clarifying APIs, or checking implementation details, for example: *"Instead of going to Stack Overflow for a coding problem, I now go to ChatGPT first, even before any search engine."* (P135, Q1) and *"Instead of googling the syntax for one language, I immediately ask ChatGPT to provide an example"* (P149, Q2). For many, AI effectively displaced older search habits: *"It has basically replaced google for me"* (P143, Q1). Longitudinally, this behaviour remained stable, with Q2 accounts often framing AI as a routine replacement for traditional search tools, for example: *"AI coding Assistants are my first source of information."* (P174, Q2) and *"AI tools have replaced most of my Google and Stack Overflow search."* (P48, Q2).

Sub-theme 2B: Accelerated learning, onboarding, and code understanding. Participants in both time points used AI to accelerate understanding across unfamiliar languages, frameworks, or legacy codebases. For example, one participant wrote: *"Much easier to pick up new tools, libraries, languages and get productive."* (P132, Q1). Another wrote: *"It has allowed me to learn how to use a poorly documented code base or library very quickly."* (P127, Q1). This sentiment persisted longitudinally, with Q2 participants reporting similar experiences, for example: *"It has been particularly helpful in explaining legacy code and detecting bugs more efficiently."* (P2, Q2). Others reported gaining conceptual clarity: *"Definitely felt like it's given me a level up on understanding new concepts"* (P5, Q2).

Theme 3: From doing to supervising AI-generated code. Across both time points, engineers described shifting effort from directly producing code to supervising AI-generated output. Rather than focusing on hands-on implementation, participants increasingly emphasised planning, evaluating, and correcting the assistant's suggestions. This shift toward oversight of AI-generated code aligns with the slight (non-significant) upward trend observed for reviewing activities. At Q2, this supervisory framing became slightly more prominent. More participants described routinely assessing the assistant's output, iterating on prompts, or refining generated code, consistent with longer-term integration of AI into daily practice.

While some participants framed this shift in terms of "design" or "higher-level thinking", their descriptions more consistently reflected supervision-oriented activities, such as directing the assistant, evaluating its suggestions, and deciding what to accept, modify, or discard. This supervisory work replaced portions of hands-on implementation effort but does not map cleanly onto traditional software development task categories. As one participant explained, *"Sometimes, I'll not be happy with the original reference, and will 'coach' the AI to produce something that works better for my use-case"* (P75, Q1). Another described a similar evaluative stance: *"I ask AI to make changes first and review them before accepting or rejecting them"* (P82, Q1). At Q2, participants increasingly articulated the effort involved in directing and controlling the assistant itself. One noted, *"I spend more time considering how to craft the prompt"* (P46, Q2), while another described their role as *"Mostly reading the code and directing AI on the right way"* (P54, Q2).

Overall, although participants occasionally labelled these changes as increased design thinking, the underlying accounts point to a distinct supervisory layer of work focused on coordination, evaluation, and intent-specification rather than architectural or conceptual design. This interpretation is consistent with quantitative results, which showed no corresponding increase in reported time spent on design tasks across either time point.

Theme 4: Verification and trust calibration moderating perceived benefits. Participants varied in how much they trusted or relied upon AI-generated code. Some reported substantial workflow improvements, while others emphasised the need to verify every suggestion or restrict use to low-risk contexts. This theme was evident at both time points, with participants consistently describing verification and judgement as central to their use of AI coding assistants. One participant

described this careful approach succinctly: *"I almost never let it write production code, or at least never leave in code that I don't understand and or wouldn't have written myself"* (P119, Q1). Others stressed the limits of trust, particularly for complex or critical work: *"I do not trust it for strategic or critical code."* (P8, Q1).

At Q2, similar concerns remained evident, though participants reflected on how their use of AI had evolved over time. One participant noted, *"I think 6 months ago, I was using the tools a lot more cautiously with a lot more fact checking"* (P109, Q2), while another highlighted the increasing verification effort required: *"has increased the 'is this code correct', 'is it maintainable', 'is it secure' and 'is this just probabilistic hallucination' time up."* (P88, Q2). A smaller subset of participants reported minimal workflow change in Q2, using AI only sporadically or for limited tasks. As one explained, *"Overall workflow is much the same. Using the AI is still an after thought for me."* (P76, Q2). These accounts illustrate that trust calibration varied substantially across participants, shaping both how AI was used and the extent to which it was perceived as beneficial.

5.2.4 Summary

A majority of participants (69%) perceived spending less time overall on the six development tasks at both time points. Writing code showed the largest reduction, with 82% reporting less time by Q2 and means well below neutral at both time points ($M = 2.10$ at Q1; $M = 1.93$ at Q2). All tasks except reviewing showed means below neutral, with writing code furthest from the midpoint and designing and debugging closest. Testing and reviewing were the only tasks to trend upward between time points. Among matched participants, a significant shift toward verification activities emerged ($r_{rb} = 0.39$, $p_{adj} = 0.006$, moderate effect). However, only 9% showed the predicted pattern of reduced writing alongside increased design time, and the two tasks were not correlated ($\rho = 0.07$). Productivity perceptions predicted less time writing code (OR = 0.57, $p < 0.05$) while lower self-efficacy predicted more time reviewing (OR = 0.62, $p < 0.05$). Qualitatively, participants described compressed routine implementation, rerouted information seeking, a new supervisory layer of directing and evaluating AI output, and ongoing verification and trust calibration that moderated perceived benefits.

5.3 RQ2: Developer Experience and Productivity

This section addresses RQ2: *How does the use of AI coding assistants impact software engineers' perceptions of their developer experience and productivity?* We analyse two related constructs: developer experience (cognitive load, feedback loops, and flow state) and perceived productivity.

5.3.1 Developer Experience

5.3.1.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete data across all three dimensions (Q1: $n = 157$; Q2: $n = 101$).

Figure 5.10 presents the distribution of responses at both time points. At Q1, participants reported similar improvements in cognitive load and feedback loops, while flow state lagged behind and attracted more negative ratings. By Q2, feedback loops showed the greatest improvement while cognitive load and flow state showed smaller improvements.

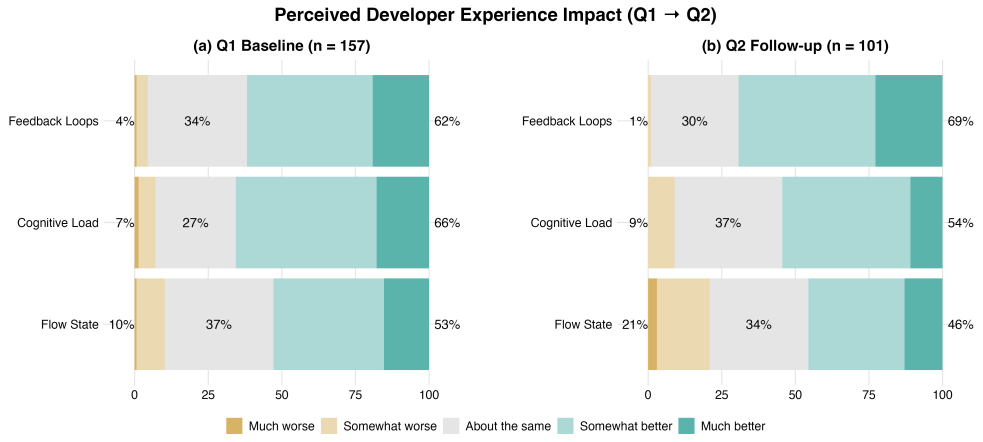


Figure 5.10: Perceived developer experience impact across time points.

Table 5.10 presents means and standard deviations for each dimension at both time points. One-sample Wilcoxon signed-rank tests showed that all three dimensions were rated above the neutral midpoint at both time points ($p_{adj} < 0.01$), with effect sizes ranging from moderate (flow state at Q2: $r_{rb} = 0.43$) to large (feedback loops at Q2: $r_{rb} = 0.98$).

Table 5.10: Developer Experience Dimension Means and Standard Deviations by Time Point

Dimension	**Q1** ($n = 157$)		**Q2** ($n = 101$)		
	M	***SD***	***M***	***SD***	**Δ*M***
Feedback Loops	3.76	0.83	3.91	0.75	+0.15
Cognitive Load	3.75	0.86	3.56	0.81	−0.19
Flow State	3.57	0.89	3.35	1.01	−0.23

Note. Scale: 1 = much worse, 3 = about the same, 5 = much better.
ΔM = Q2 mean − Q1 mean.

Inter-Dimension Correlations

At Q1, all three dimension pairs showed significant correlations (Holm-Bonferroni corrected), with the largest between cognitive load and flow state ($\rho = 0.42$, $p_{adj} < 0.001$, moderate effect). By Q2, all correlations weakened and none reached significance.

Predictors of Developer Experience

We fitted ordinal logistic regression models to identify factors associated with each developer experience dimension (predictors per Section 4.4.1.1). Table 5.11 shows the results.

At Q1, AI usage frequency and very large company size were the most consistent predictors, each significantly associated with two developer experience dimensions. Programming language effects were limited to cognitive load only. None of these Q1 effects persisted to Q2, where only two role-based effects emerged with extreme odds ratios that should be interpreted cautiously due to small subgroup sizes. Overall, cognitive load attracted the majority of significant effects, while feedback loops and flow state showed few significant associations with the predictors examined. Demographic factors showed no significant effects at either time point.

No Significant Associations Found for AI Tool Selection

We examined whether specific AI tool choices associated with developer experience outcomes using Fisher's exact tests with Holm-Bonferroni correction. No tools reached significance at either time point.

Attitudes Towards AI Tools Show Consistent Associations

Table 5.12 shows Spearman correlations between attitudes towards AI tools and developer experience dimensions. Both initial impressions at Q1 and anticipated disappointment at Q2 showed significant positive associations with all three

Table 5.11: Ordinal Logistic Regression of Developer Experience Dimensions (Q1 vs Q2)

Predictor	**Cognitive Load**		**Feedback Loops**		**Flow State**	
	OR Q1	**OR Q2**	**OR Q1**	**OR Q2**	**OR Q1**	**OR Q2**
Gender (Reference: Man)						
Woman vs Man	0.82	1.48	0.73	0.70	0.92	0.59
Experience Level (Reference: Senior)						
Junior vs Senior	1.01	2.84	1.36	1.90	0.96	1.41
Mid-Career vs Senior	0.78	0.95	0.99	1.22	0.75	0.47
Role (Reference: Backend)						
Frontend vs Backend	0.62	0.44	0.45	0.54	0.49	0.81
Fullstack vs Backend	0.75	1.09	1.79	1.36	0.49	0.62
Manager vs Backend	1.45	3.06	2.22	2.53	0.64	1.55
Mobile vs Backend	1.51	2.38	3.45	**0.12***	1.01	0.39
Specialised vs Backend	3.48	**15.82****	1.53	2.81	0.80	4.04
Programming Language (Reference: Web)						
Compiled vs Web	**0.26****	0.65	0.73	0.35	0.58	1.91
Interpreted vs Web	**0.35***	0.52	0.94	0.41	0.92	0.74
Company Size (Reference: Medium)						
Small vs Medium	2.00	0.97	1.57	2.20	1.14	1.91
Large vs Medium	**3.06***	0.82	1.57	1.25	0.94	0.96
Very Large vs Medium	**3.59***	2.69	**2.97***	3.03	1.40	1.33
Usage Location (Reference: Both)						
Work Only vs Both	1.14	0.45	0.54	0.62	0.98	0.77
Personal Only vs Both	1.10	0.31	1.78	3.85	3.50	2.53
AI Usage Frequency	**1.81*****	1.42	1.12	1.54	**1.52***	1.22

OR = Odds Ratio for moving to a higher response category.
Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
Q1: $n = 145$ (Flow State: $n = 144$); Q2: $n = 94$. McFadden R^2 range: 0.034–0.133.

dimensions, with feedback loops consistently showing the strongest association (moderate effect at both time points).

Primary concerns showed no significant associations with any dimension at either time point (Kruskal-Wallis tests with Holm-Bonferroni correction).

5.3.1.2 Longitudinal Change in Developer Experience

Next, we examined how developer experience perceptions changed within participants ($n = 94$ matched participants).

Aggregate Change Across Dimensions

Paired Wilcoxon signed-rank tests with Holm-Bonferroni correction revealed

Table 5.12: Spearman Correlations Between Attitudes and Developer Experience Dimensions

Dimension	**Q1 Initial Impressions**		**Q2 Anticipated Disappointment**	
	ρ	p_{adj}	ρ	p_{adj}
Feedback Loops	**0.32*****	<0.001	**0.33****	0.002
Flow State	**0.27****	0.001	**0.26***	0.015
Cognitive Load	**0.22****	0.007	**0.23***	0.021

Positive correlations indicate more positive attitudes associated with better developer experience.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$. Holm-Bonferroni corrected within each time point.
Q1 $n = 157$; Q2 $n = 101$.

divergent trajectories across dimensions (Table 5.13). Only feedback loops improved significantly; cognitive load and flow state showed non-significant declines.

Individual variation was substantial: for feedback loops, 33% improved and 18% declined; for cognitive load, 19% improved and 29% declined; for flow state, 27% improved and 35% declined.

Table 5.13: Longitudinal Wilcoxon Signed-Rank Tests: Changes in Developer Experience Dimensions Within Matched Participants ($n = 94$)

Dimension	**Q1**		**Q2**			**Test Statistics**		
	M	SD	M	SD	ΔM	V	p_{adj}	r_{rb}
Feedback Loops	3.73	0.78	3.95	0.74	+0.21	810.5	0.038	**0.38***
Cognitive Load	3.74	0.80	3.60	0.79	−0.15	382.5	0.195	−0.26
Flow State	3.54	0.84	3.36	1.01	−0.18	651.5	0.195	−0.24

ΔM = Mean change (Q2 - Q1). V = Wilcoxon signed-rank test statistic. r_{rb} = rank-biserial correlation.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.

Transition Patterns

Among matched participants, we classified perceptions into cohorts based on meaningful thresholds: *Positive* (all three dimensions rated 4–5), *Negative* (any dimension rated 1–2), and *Mixed/Neutral* (neither fully positive nor containing any negative ratings). Figure 5.11 visualises how participants transitioned between these cohorts over the study period.

The cohort analysis reveals a drift toward negative developer experience. The Negative cohort nearly doubled from 14% to 27%, while the Positive cohort showed poor retention at 37%. No participant who started in the Negative cohort

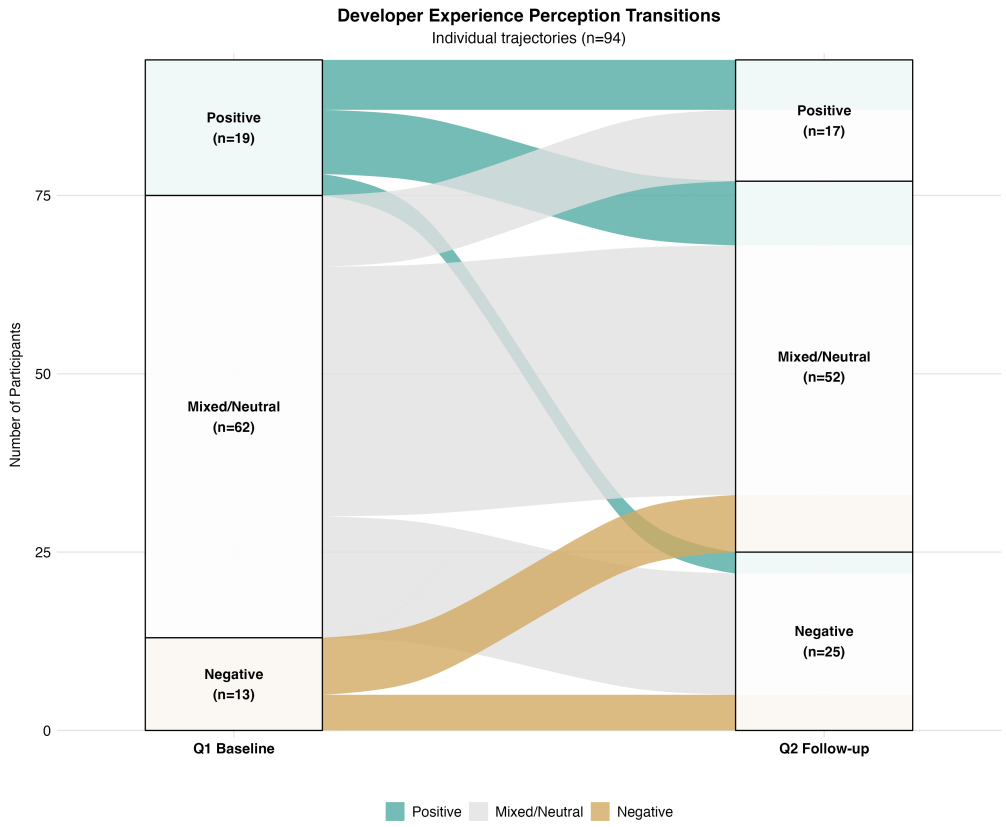


Figure 5.11: Developer experience perception transitions between Q1 and Q2.

recovered to fully Positive by Q2. Complete stability across all three dimensions was rare (9%).

Within the Negative cohort, flow state was the predominant issue, affecting 54% at Q1 and rising to 76% at Q2.

Demographic Differences in Change Patterns

To explore whether demographic factors were associated with perception changes, we examined mean changes across gender, experience, role, programming language, company size, and usage location (Figure 5.12).

These patterns are descriptive; we did not test between-group differences inferentially given sample sizes. For feedback loops, mobile developers were the only group reporting a substantial decline ($\Delta M = -0.83$), while most other groups improved; frontend ($\Delta M = +0.71$) and web developers showed the greatest gains. For cognitive load, most groups reported worsening perceptions; only women,

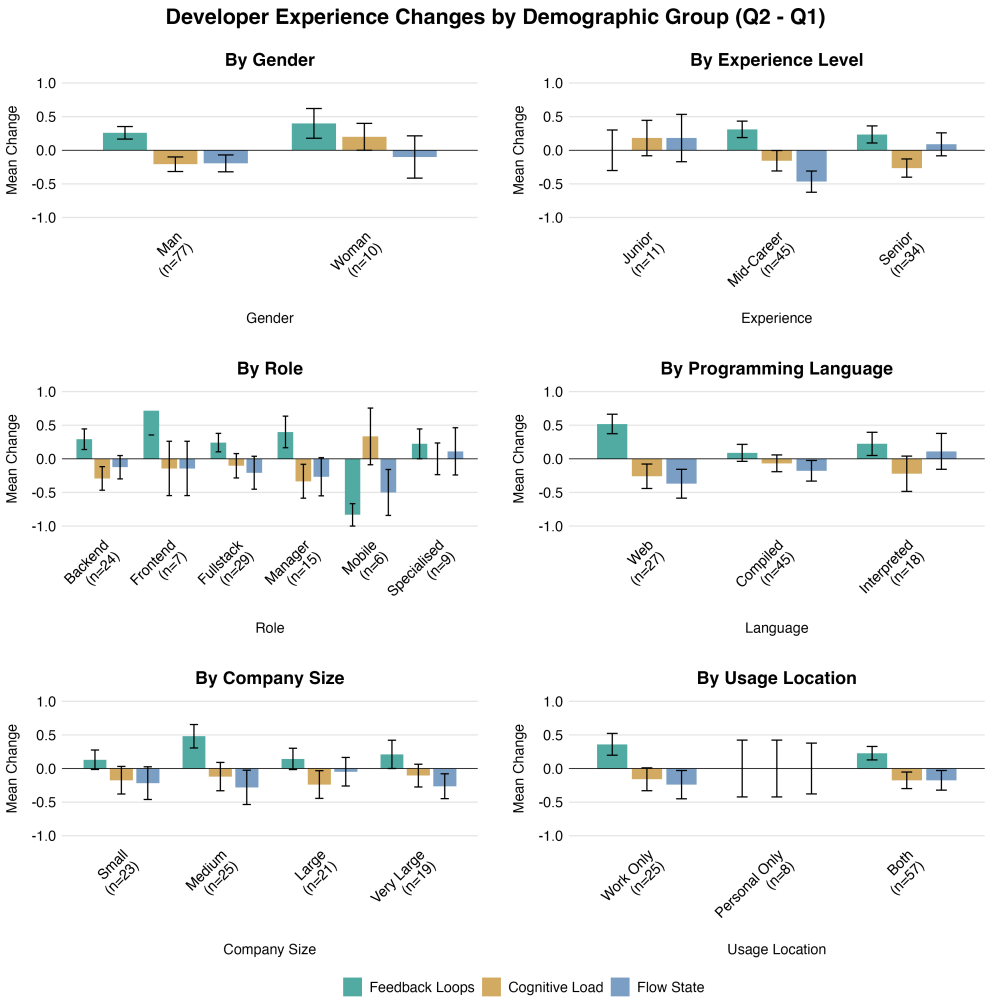


Figure 5.12: Mean changes in developer experience dimensions by demographic group from Q1 to Q2. Error bars represent standard error.

juniors, and mobile developers reported improvement. For flow state, juniors, seniors, specialised roles, and interpreted language users reported improvement while all other groups reported decline. Mobile and mid-career developers reported the greatest declines ($\Delta M = -0.50$ and -0.49 respectively). Overall, most groups followed a pattern of improved feedback loops but worsening cognitive load and flow state. Mobile developers were a notable exception: they showed an inverted pattern on feedback loops and cognitive load (worsening where most improved, improving where most worsened), alongside the largest flow state decline.

Mixed-Effects Models of Developer Experience

To examine factors explaining individual differences in developer experience perceptions, we applied mixed-effects modelling ($n = 90$; see Section 4.4.1.4 for model structure).

Table 5.14 shows the progressive model results. Demographic factors showed no significant effects on any dimension. The time effect revealed divergent trajectories: feedback loops improved significantly between time points, while cognitive load worsened and flow state showed a non-significant decline. Self-efficacy was the strongest and most consistent predictor, significant across all three dimensions, with a particularly large effect on feedback loops. More frequent AI usage was associated with better cognitive load outcomes but showed no significant relationship with feedback loops or flow state.

Adding psychological factors improved model fit across all three dimensions, with the largest improvement for feedback loops. Individual differences (ICC) accounted for 23–45% of response variance, decreasing as predictors were added.

5.3.2 Productivity

5.3.2.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete responses (Q1: $n = 158$; Q2: $n = 101$).

At Q1, productivity perceptions were predominantly positive ($M = 4.08$, $SD = 0.65$), with 84% reporting improvement. At Q2, perceptions remained stable ($M = 4.03$, $SD = 0.59$), with identical 84% reporting improvement (Figure 5.13). One-sample Wilcoxon signed-rank tests indicated that productivity ratings were above the neutral midpoint at both time points ($p < 0.001$, $r_{rb} = 0.99$ at Q1, $r_{rb} = 1.00$ at Q2; both large effects).

Table 5.14: Cumulative Link Mixed Model Analysis of Developer Experience Dimensions ($n = 90$)

	Feedback Loops			**Cognitive Load**			**Flow State**		
Predictor	**M1**	**M2**	**M3**	**M1**	**M2**	**M3**	**M1**	**M2**	**M3**
Time (6-month change)	**2.47****	**2.40****	**2.41****	0.60	0.55	**0.53***	0.68	0.67	0.66
Woman vs Man	0.36	0.36	0.56	1.15	1.24	1.61	0.42	0.43	0.52
Junior vs Senior	3.87	3.38	1.47	3.10	2.34	1.45	0.78	0.76	0.51
Mid-Career vs Senior	1.14	1.05	1.05	1.62	1.36	1.32	0.53	0.52	0.53
Usage Frequency (z)		1.26	1.16		**1.59***	**1.55***		1.06	1.00
Self-Efficacy (z)			**3.27*****			**1.73****			**1.69****
Prompt Engineering Importance (z)			1.45			1.25			1.01
Model Fit									
AIC	391.6	392.3	360.6	422.5	418.6	411.9	481.4	483.3	478.8
ICC (%)	44.5	44.9	33.9	36.1	34.3	26.0	27.0	26.6	23.2

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Prompt Engineering Importance.
Values are odds ratios (OR). OR > 1 (< 1) indicates higher (lower) odds of reporting better experience.
Continuous predictors z-scored. Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
2 time points (180 observations).

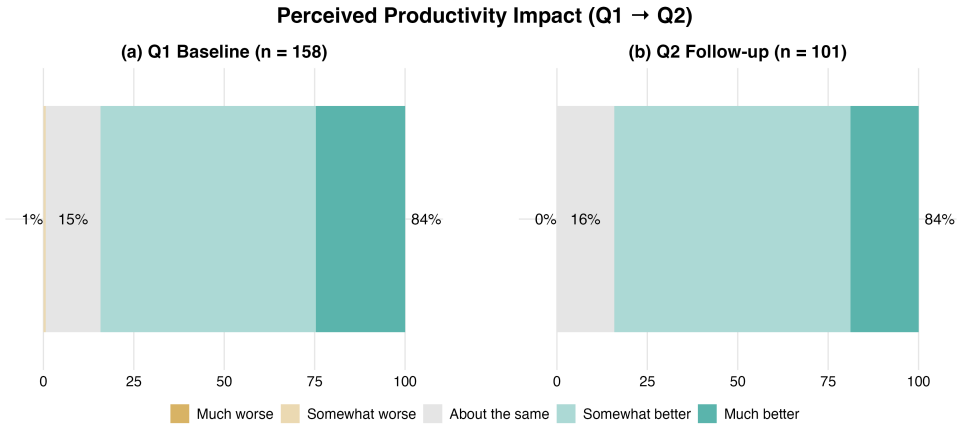


Figure 5.13: Perceived productivity impact across time points.

Predictors of Productivity

We fitted ordinal logistic regression models to identify factors associated with productivity perceptions (predictors per Section 4.4.1.1).

Table 5.15 presents the regression results. Usage frequency was the only consistent predictor across both time points, with more frequent users reporting greater productivity gains. Programming language and company size each showed one significant Q1 effect that did not persist to Q2. Demographic factors showed no significant effects at either time point.

AI Tool Selection Shows One Significant Association

We examined whether specific AI tool choices associated with productivity perceptions using Fisher's exact tests with Holm-Bonferroni correction. ChatGPT use showed a significant positive association at Q1 ($p_{adj} = 0.030$). No other tools reached significance at either time point.

Attitudes Towards AI Tools Show Selective Associations

Spearman correlations showed a significant moderate positive association between initial impressions and productivity at Q1 ($\rho = 0.31$, $p < 0.001$, moderate effect). At Q2, anticipated disappointment showed a stronger positive correlation ($\rho = 0.48$, $p < 0.001$, moderate effect). Primary concerns showed no significant association with productivity at Q1, but at Q2 productivity perceptions differed by primary concern ($\chi^2 = 18.34$, $p = 0.005$, $\varepsilon^2 = 0.19$, large effect), with those citing dependency reporting the highest productivity gains.

Table 5.15: Ordinal Logistic Regression of Productivity Perceptions (Q1 vs Q2)

Predictor	**Q1**		**Q2**	
	OR	**95% CI**	**OR**	**95% CI**
Gender (Reference: Man)				
Woman vs Man	0.98	[0.33, 2.86]	0.84	[0.19, 3.68]
Experience Level (Reference: Senior)				
Junior vs Senior	1.75	[0.55, 5.54]	4.29	[0.81, 22.74]
Mid-Career vs Senior	0.79	[0.37, 1.70]	0.81	[0.29, 2.32]
Role (Reference: Backend)				
Frontend vs Backend	0.18	[0.03, 1.12]	0.63	[0.05, 7.28]
Fullstack vs Backend	0.34	[0.12, 1.01]	0.64	[0.15, 2.82]
Manager vs Backend	1.38	[0.42, 4.57]	1.65	[0.34, 8.00]
Mobile vs Backend	1.63	[0.39, 6.77]	2.31	[0.35, 15.27]
Specialised vs Backend	0.68	[0.14, 3.34]	2.49	[0.28, 22.22]
Programming Language (Reference: Web)				
Compiled vs Web	**0.25****	[0.09, 0.71]	0.58	[0.13, 2.55]
Interpreted vs Web	0.41	[0.13, 1.25]	1.02	[0.23, 4.57]
Company Size (Reference: Medium)				
Small vs Medium	**2.59***	[1.01, 6.60]	1.28	[0.34, 4.77]
Large vs Medium	0.97	[0.34, 2.72]	0.44	[0.11, 1.76]
Very Large vs Medium	1.86	[0.61, 5.67]	0.95	[0.23, 3.87]
Usage Location (Reference: Both)				
Work Only vs Both	0.82	[0.36, 1.88]	0.33	[0.10, 1.12]
Personal Only vs Both	3.55	[0.90, 14.11]	3.45	[0.38, 30.96]
AI Usage Frequency	**1.99*****	[1.36, 2.91]	**2.05***	[1.15, 3.66]

OR = Odds Ratio for moving to a higher response category; CI = 95% Confidence Interval.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
Q1: $n = 145$, McFadden $R^2 = 0.122$.
Q2: $n = 94$, McFadden $R^2 = 0.160$.

Cross-Sectional Association with Developer Experience

All three developer experience dimensions showed significant positive correlations with productivity at both time points (Holm-Bonferroni corrected). At Q1, flow state was strongest ($\rho = 0.49$, $p_{adj} < 0.001$, moderate effect). By Q2, all three correlations weakened, and feedback loops replaced flow state as the strongest ($\rho = 0.37$, $p_{adj} < 0.001$, moderate effect), while flow state dropped to a small effect ($\rho = 0.20$, $p_{adj} < 0.05$).

Ordinal regression controlling for all dimensions confirmed this shift (Table 5.16). At Q1, flow state was the strongest predictor. By Q2, feedback loops became the strongest predictor while flow state's effect weakened to non-significance.

Table 5.16: Ordinal Regression of Productivity on Developer Experience Dimensions

Dimension	**Q1**		**Q2**	
	OR	**95% CI**	**OR**	**95% CI**
Flow State	**2.94*****	[1.86, 4.65]	1.51	[0.98, 2.31]
Cognitive Load	**2.25****	[1.38, 3.68]	**2.05***	[1.17, 3.59]
Feedback Loops	**1.87****	[1.19, 2.94]	**3.08*****	[1.66, 5.74]

OR = Odds Ratio for moving to a higher productivity category; CI = 95% Confidence Interval.
Model includes all three dimensions as simultaneous predictors. Q1: $n = 157$; Q2: $n = 101$.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.

5.3.2.2 Longitudinal Change in Productivity

Next, we examined how productivity perceptions changed within participants ($n = 95$ matched participants).

Aggregate and Individual Stability

Paired Wilcoxon signed-rank tests showed no significant change ($\Delta M = -0.05$, $V = 99$, $p = 0.326$, $r_{rb} = -0.22$, small effect). Stability was high: 77% maintained identical ratings, while 14% decreased and 9% increased intensity. No participant transitioned to negative perceptions.

Transition Patterns

Figure 5.14 visualises individual-level transitions between rating levels. Stability varied by initial position: 85% of "Somewhat better" respondents maintained their rating compared to 65% of "Much better" and 62% of neutral. Movement from neutral was entirely positive, with 38% shifting to "Somewhat better".

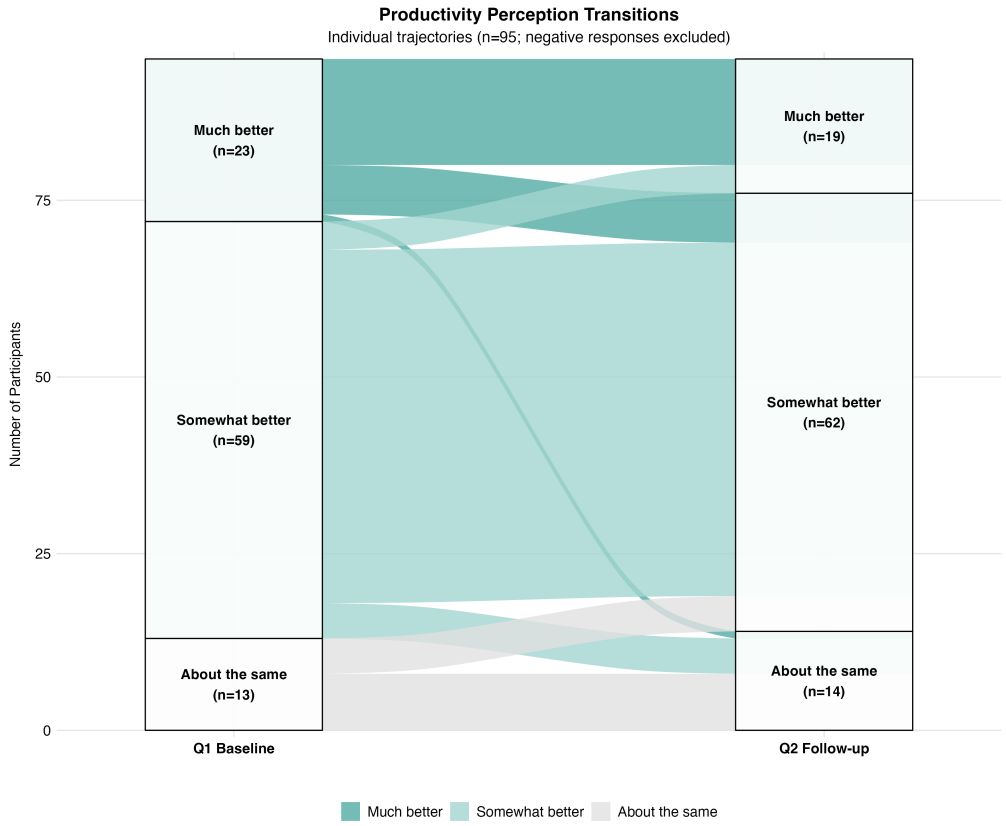


Figure 5.14: Productivity perception transitions between Q1 and Q2.

Demographic Differences in Change Patterns

To explore whether demographic factors were associated with changes in productivity perceptions, we examined mean changes across gender, experience, role, programming language, company size, and usage location (Figure 5.15).

These patterns are descriptive; we did not test between-group differences inferentially given sample sizes. Women reported no change ($\Delta M = 0.00$), while men showed a slight decline ($\Delta M = -0.05$). By experience, junior and mid-career engineers remained flat while seniors declined ($\Delta M = -0.14$). By role, mobile developers showed the largest positive change ($\Delta M = +0.33$) while back-end ($\Delta M = -0.12$) and managers ($\Delta M = -0.13$) showed the largest declines. Interpreted language users showed a small increase ($\Delta M = +0.11$) while Web and compiled users both declined. By company size, medium-sized companies showed

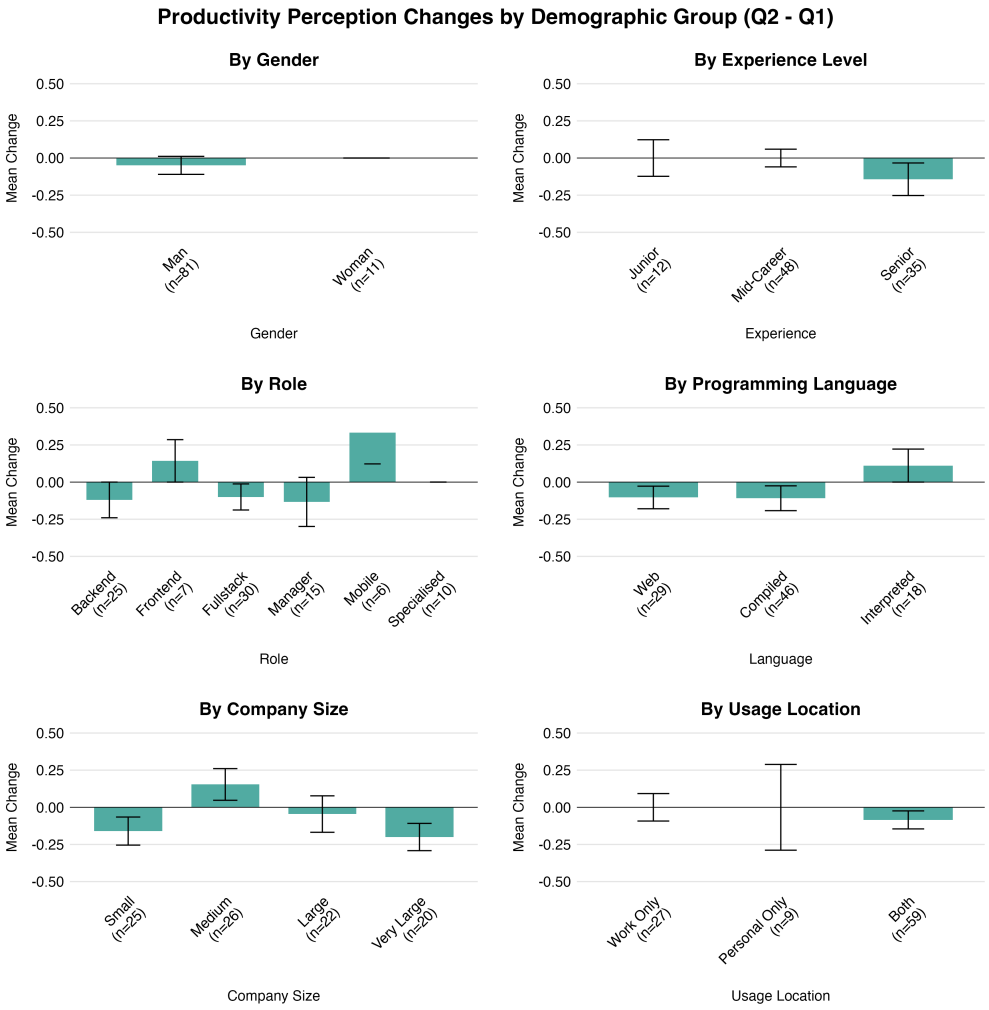


Figure 5.15: Mean changes in productivity perceptions by demographic group from Q1 to Q2. Error bars represent standard error.

the only positive change ($\Delta M = +0.15$) while very large companies showed the largest decline ($\Delta M = -0.20$).

Longitudinal Alignment with Developer Experience Change

We correlated change scores (Q2 minus Q1) for each developer experience dimension with productivity change ($n = 94$, Holm-Bonferroni corrected). None reached significance (smallest $p_{adj} = 0.313$), contrasting with the moderate-to-strong cross-sectional correlations reported above.

Mixed-Effects Models of Productivity

To examine factors explaining individual differences in productivity perceptions, we applied mixed-effects modelling ($n = 91$; see Section 4.4.1.4 for model structure).

Table 5.17: Cumulative Link Mixed Model Analysis of Productivity Perceptions ($n = 91$)

Predictor	**M1**	**M2**	**M3**
Time (6-month change)	0.66	0.49	0.45
Gender: Woman vs Man	0.41	0.49	0.94
Exp: Junior vs Senior	8.87	11.50	1.00
Exp: Mid-Career vs Senior	0.96	0.59	0.62
Usage Frequency		**2.53***	**2.86****
Self-Efficacy			**10.68*****
Prompt Engineering Importance			**1.92***
Model Fit			
AIC	277.6	272.2	231.7
ICC (%)	90.4	93.7	57.2

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Prompt Engineering Importance.
Values are odds ratios (OR). OR > 1 (< 1) indicates higher (lower) odds of reporting better productivity.
Continuous predictors z-scored. Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
2 time points (182 observations).

Table 5.17 shows the progressive model results. Demographics and time point showed no significant effects. Self-efficacy was the dominant predictor, with each standard deviation increase associated with over ten times higher odds of reporting improved productivity. More frequent AI usage and higher prompt engineering importance ratings were also significant positive predictors. Adding psychological factors substantially improved model fit (AIC decreased from 277.6 to 231.7),

and the ICC decreased from 90% to 57%, indicating that these factors explained a large share of between-participant variance.

Construct Validity

Given the magnitude of the self-efficacy effect, we examined whether self-efficacy and productivity might capture the same underlying phenomenon. Correlations were moderate ($\rho = 0.62$ at Q1, $\rho = 0.61$ at Q2), well below the 0.85 threshold for problematic construct overlap (Section 4.4.1.5), representing 37–39% shared variance. Cross-lagged analysis ($n = 94$) found bidirectional effects: Q1 self-efficacy predicted Q2 productivity ($\beta = 0.23$, $p = 0.025$) controlling for Q1 productivity, and Q1 productivity predicted Q2 self-efficacy ($\beta = 0.27$, $p = 0.007$) controlling for Q1 self-efficacy.

5.3.3 Qualitative Perspectives on Experience and Perceived Productivity

Reflexive thematic analysis of responses to Q4.3, which asked participants to describe instances where AI had significantly helped or hindered their work (Q1) or unexpected benefits or challenges (Q2), identified three themes capturing accelerated throughput with uneven experience, expanded capability in unfamiliar work, and verification as ongoing engineering work.

Theme 1: Accelerated throughput with uneven day-to-day experience. Across both time points, participants described AI coding assistants as accelerating progress, particularly for early-stage or repetitive work such as boilerplate generation, test scaffolding, and quick prototyping. Many framed this as a tangible reduction in effort, for example: *"Now with the AI coding assistants, I can do it all within 15 minutes"* (P4, Q1) and *"Boring mechanical tasks like refactoring files, updating tests, adding translations, creating test fixtures are 10x less effort with AI assistants"* (P132, Q1). Others described compressed delivery timelines: *"Without the tool, the changes would have needed 3 days. But with the tool, completed them in 3 hours"* (P83, Q1). At Q2, participants described similar gains: *"It helped me complete the initial version of the project within hours instead of days . . . It made the entire process much faster and more efficient"* (P91, Q2). AI support was most valued when it enabled rapid starts, reduced mechanical effort, and sustained momentum.

However, participants emphasised that these throughput gains were not uniformly experienced as smoother or easier work. Many described hidden costs in

the form of repeated prompting, adaptation effort, or time lost to incorrect output. At Q1, some described prompt loops and rework, for example: *"wasted some time getting into prompt loop"* (P18, Q1). Others reverted to traditional sources when AI was unreliable: *"... it would reference methods which were no longer relevant, even after specifying a specific version I was using, I ended up going to docs"* (P26, Q1). By Q2, participants continued to describe speed alongside friction. Some described the variability directly: *"Sometimes you gain productivity boost, the other time your wasting time on hallucinations or fixing the generated code"* (P89, Q2). Others noted that flow could be disrupted even as output accelerated, for example: *"This does break the flow, but still speeds up the overall process"* (P28, Q2) and *"less 'in the zone' time, because while code is being produced, I now find myself switching context more often"* (P159, Q2). Altogether, AI often accelerated throughput while redistributing effort into supervision, correction, and verification. Perceived productivity and lived experience did not always move together.

Theme 2: Expanded perceived capability and willingness to engage with unfamiliar work. Participants described AI coding assistants as lowering the threshold for engaging with unfamiliar technologies, languages, and problem domains at both time points. Rather than claiming mastery, accounts emphasised practical reach, being able to start, explore, or contribute in areas that previously felt inaccessible or too time-consuming to approach. At Q1, this was often described as a faster path from uncertainty to a workable first step, for example: *"I don't have a lot of experience in authentication flows and got good directions from Gemini on designing a solution"* (P84, Q1) and *"Without knowing Python or Shell scripting, I made a functional web scraper in a day - all by ChatGPT itself."* (P135, Q1). Participants continued to describe this enabling effect in Q2, with AI supporting learning-through-doing and improving their ability to work across less familiar stacks, for example: *"Learning new languages/frameworks by "just doing" together with AI assistant"* (P15, Q2) and *"More confident picking up new tools that I'm otherwise unfamiliar with even if they don't have beginner friendly docs"* (P175, Q2).

This expanded capability was framed as reducing hesitation and encouraging experimentation through readily available examples, explanations, and first-pass implementations. However, participants drew boundaries around what this confidence represented. Some cautioned that AI-enabled progress could short-circuit

learning even while increasing output: *"Destroys process of learning of something new - you get a working solution, but not understanding why and how it may work"* (P41, Q1). This broader reach still depended on critical judgement and active oversight rather than passive trust; at Q2, some described maintaining a critical stance even when AI sounded plausible: *"I do like the fact that I can ask it about completely unfamiliar domains and it has a reasonable.. sounding at least.. response. I generally feel that I'm critical of all output, but am usually pleasantly surprised"* (P75, Q2). Overall, these accounts position AI coding assistants as expanding perceived capability by making unfamiliar work more approachable and actionable, though sometimes at the expense of deeper understanding.

Theme 3: Verification and trust management as ongoing engineering work. AI coding assistants introduced new forms of verification and trust management that remained firmly within the engineer's remit. Participants framed correctness, quality and consequence as fundamentally human responsibilities rather than treating AI output as authoritative. This required active review, judgement, and intervention. Trust calibration through practices such as sceptical oversight, deliberate steering, and collaborative verification within teams recurred across both time points. These accounts position trust management as an ongoing form of engineering work that shapes how AI-generated code is evaluated, integrated, and governed in practice.

Sub-theme 3A: Individual oversight: scepticism, accountability, and steering practices. Participants framed AI-generated output as provisional, emphasising that responsibility for correctness and consequences remained with the engineer. This accountability manifested through both scepticism and concrete steering practices. At Q1, participants articulated the need for active control, for example: *"You need to always be fully in control, and guide the AI into the direction you want, and not let the AI guide you"* (P101, Q1), with some describing early over-trust as a corrective experience: *"It's gotten code templates completely wrong (yaml) and I was overconfident in the output"* (P129, Q1). This scepticism translated into iterative refinement: *"Often, it takes multiple tries and prompts to achieve the desired results"* (P3, Q1). Steering sometimes required more precise articulation of intent, for example *"E.g. "Track the width of this react component" didn't work, but "Use a react reference to track the width of this react component when it's rendered" did work"* (P82, Q1). By Q2, participants described both recognising model failures and intervening accordingly, for example: *"Identifying*

when the LLM has lost the thread of the problem and halting it is essential" (P8, Q2). Constraint-setting became more explicit, such as deliberately breaking work into smaller units to reduce hallucination and drift: *"it's been beneficial to break it down into smaller more specific "modules" and then tie them together at the end"* (P79, Q2). Across both time points, effective AI use was framed as requiring ongoing human judgement—through deliberate scepticism and active steering—rather than increasing reliance or unquestioned trust in AI output.

Sub-theme 3B: Collective trust, reputation, and collaborative implications. Participants described trust management as extending beyond individual practice into shared team contexts, where AI coding assistants affected code review effort, shared understanding, and reputational risk. At Q2, verification concerns extended to reputational and team-level stakes: *"I might use AI chat to help with a question but I do not trust the answer enough for me to hang my reputation on it. I'll always look for a corroborating answer elsewhere"* (P76, Q2). These concerns extended to how participants judged others' AI-generated work, with some expressing reduced confidence in the author's understanding of the code: *"If I know that a dev is using AI heavily I trust them much less to know what exactly their code is doing when reviewing their code"* (P152, Q2), alongside worries about unchecked exploration by less experienced colleagues, such as *"When left unsupervised with AI, junior colleagues sometimes have a tendency to pursue inappropriate solutions to the wrong problems much further"* (P158, Q2). While these concerns were most explicit at Q2, Q1 accounts also signalled the need to retain human ownership over shared technical direction, for example: *"Hence I keep control of the overall architecture"* (P55, Q1). Altogether, verification was positioned as both an individual and a shared supervisory responsibility, involving additional review discipline, corroboration practices, and clear accountability.

5.3.4 Summary

Productivity perceptions were positive and stable: 84% reported improvement at both time points, and 77% of matched participants gave identical ratings over six months. Developer experience was more variable, with the proportion reporting worsened experience in at least one dimension growing from 14% to 27%. Among matched participants, feedback loops improved significantly, while cognitive load and flow state showed non-significant declines. Flow state showed the strongest correlation with productivity at Q1 ($\rho = 0.49$) but weakened by Q2, with feedback loops becoming the strongest significant predictor. Critically, changes in

developer experience did not correlate with changes in productivity. Self-efficacy was the strongest predictor of productivity perceptions (OR = 10.68, $p < 0.001$), with cross-lagged analysis confirming a bidirectional relationship between the two constructs. Qualitatively, participants described accelerated throughput and expanded capability alongside hidden costs, with trust calibration emerging as ongoing engineering work.

5.4 RQ3: Skill Development and Self-Efficacy

This section addresses RQ3: *How do software engineers perceive the impact of AI coding assistants on their skill development and self-efficacy when building software?* We analyse two related constructs: perceived impact on skill development and self-efficacy when building software with AI coding assistants.

5.4.1 Skill Development

5.4.1.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete responses (Q1: $n = 157$; Q2: $n = 101$).

At Q1, participants reported moderate perceptions of skill development impact ($M = 3.36$, $SD = 1.27$), with 53% perceiving impact. At Q2, perceptions were similar ($M = 3.41$, $SD = 1.26$), with 57% perceiving impact (Figure 5.16). One-sample Wilcoxon signed-rank tests showed that ratings were above the neutral midpoint at both time points (Q1: $p = 0.001$, $r_{rb} = 0.32$; Q2: $p = 0.003$, $r_{rb} = 0.35$; both moderate effects).

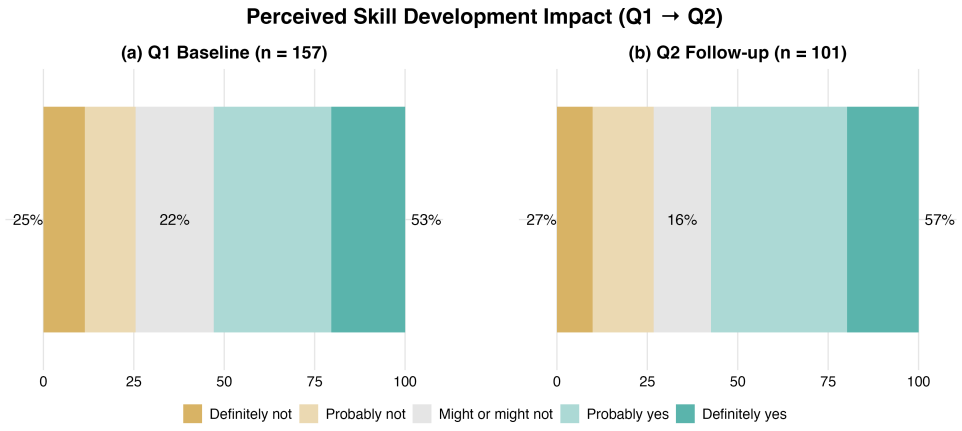


Figure 5.16: Perceived impact on skill development across time points.

Predictors of Skill Development Perceptions

We fitted ordinal logistic regression models to identify factors associated with perceived skill development impact (predictors per Section 4.4.1.1).

Table 5.18: Ordinal Logistic Regression of Skill Development Impact (Q1 vs Q2)

Predictor	**Q1**		**Q2**	
	OR	**95% CI**	**OR**	**95% CI**
Gender (Reference: Man)				
Woman vs Man	2.10	[0.77, 5.73]	2.32	[0.61, 8.77]
Experience Level (Reference: Senior)				
Junior vs Senior	2.13	[0.74, 6.11]	2.17	[0.51, 9.11]
Mid-Career vs Senior	1.12	[0.57, 2.19]	0.82	[0.34, 1.99]
Role (Reference: Backend)				
Frontend vs Backend	0.42	[0.09, 2.08]	1.54	[0.20, 11.77]
Fullstack vs Backend	0.55	[0.23, 1.34]	1.10	[0.33, 3.75]
Manager vs Backend	0.69	[0.23, 2.02]	1.04	[0.25, 4.24]
Mobile vs Backend	2.22	[0.63, 7.81]	1.03	[0.18, 5.98]
Specialised vs Backend	0.79	[0.18, 3.44]	2.55	[0.36, 17.94]
Programming Language Group (Reference: Web)				
Compiled vs Web	0.51	[0.22, 1.21]	1.22	[0.37, 4.00]
Interpreted vs Web	0.82	[0.31, 2.15]	2.40	[0.65, 8.90]
Company Size (Reference: Medium)				
Small vs Medium	1.99	[0.88, 4.53]	2.08	[0.71, 6.08]
Large vs Medium	0.77	[0.32, 1.88]	2.73	[0.85, 8.71]
Very Large vs Medium	0.98	[0.38, 2.55]	1.59	[0.47, 5.39]
Usage Location (Reference: Both)				
Work Only vs Both	0.60	[0.29, 1.23]	1.20	[0.42, 3.39]
Personal Only vs Both	0.52	[0.15, 1.79]	2.02	[0.30, 13.68]
AI Usage Frequency	0.94	[0.68, 1.30]	**1.74***	[1.06, 2.85]

OR = Odds Ratio for moving to a higher response category; CI = 95% Confidence Interval.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
Q1: $n = 144$, McFadden $R^2 = 0.043$.
Q2: $n = 94$, McFadden $R^2 = 0.063$.

Table 5.18 presents the regression results. At Q1, no predictors reached significance. At Q2, usage frequency emerged as the only significant predictor, with more frequent users more likely to perceive skill development impact. Demographic factors showed no significant effects at either time point.

AI Tool Selection Shows One Significant Association

We examined whether specific AI tool choices associated with skill development perceptions using Fisher's exact tests with Holm-Bonferroni correction. ChatGPT use showed a significant positive association at Q1 ($p_{adj} = 0.033$). No other tools reached significance at either time point.

Attitudes Towards AI Tools Show Selective Associations

Spearman correlations showed no significant association between initial impressions and skill development perceptions at Q1. At Q2, anticipated disappointment if AI tools were removed showed a significant positive correlation ($\rho = 0.21$, $p = 0.038$, small effect). Primary concerns showed no significant association at either time point (Kruskal-Wallis tests).

5.4.1.2 Longitudinal Change in Skill Development Impact Perceptions

Next, we examined how skill development impact perceptions changed within participants ($n = 95$ matched participants).

Aggregate Stability Masks Individual Variation

Paired Wilcoxon signed-rank tests showed no significant aggregate change ($\Delta M = +0.03$, $V = 770.5$, $p = 1.000$, $r_{rb} < 0.01$). However, this stability masked individual variation: 42% showed no change, while 29% shifted toward less impact and 28% toward greater impact, with these opposing movements cancelling out.

Transition Patterns

Figure 5.17 presents the transition matrix. Stability varied by initial position, with 58% of "Definitely yes" respondents maintaining their view compared to only 33% of "Definitely not". Movement was also asymmetric: 50% of those initially perceiving no impact shifted toward impact, while only 20% moved in the opposite direction.

Demographic Differences in Change Patterns

To explore whether demographic factors were associated with changes in perceptions of impact to skill development, we examined mean changes across gender, experience, role, programming language, company size, and usage location (Figure 5.18).

These patterns are descriptive; we did not test between-group differences inferentially given sample sizes. Women showed larger increases ($\Delta M = +0.36$) than men ($\Delta M = +0.05$). Experience showed a modest gradient: junior ($\Delta M =$

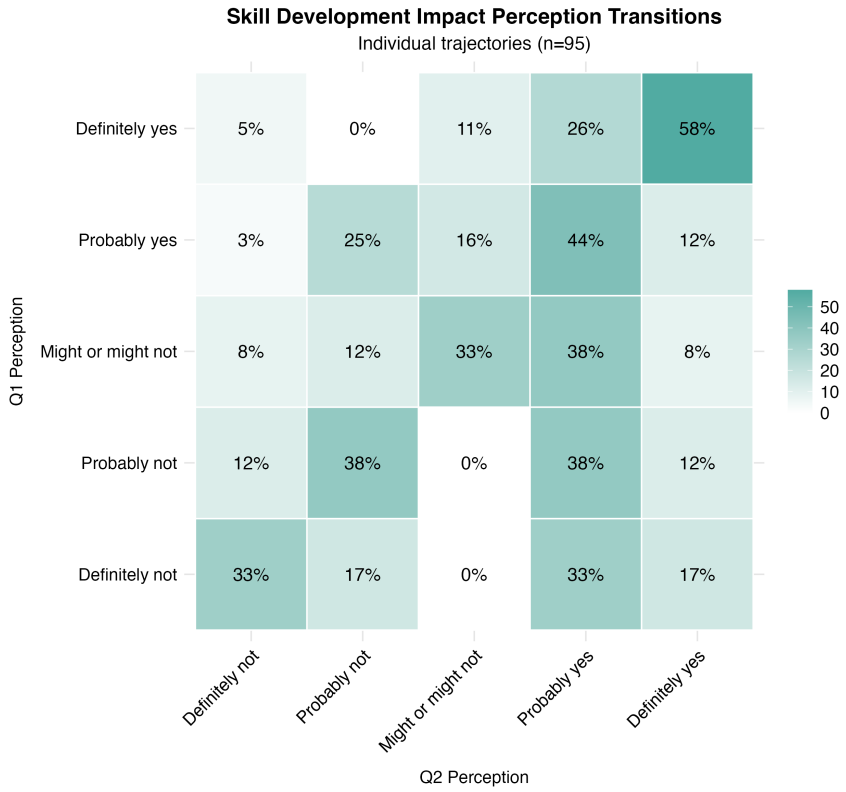


Figure 5.17: Transition matrix showing changes in skill development perceptions from Q1 to Q2.

+0.08), mid-career ($\Delta M = +0.06$), senior ($\Delta M = -0.03$). By role, fullstack was the only group with a decrease ($\Delta M = -0.10$); backend and frontend showed the largest increases. Interpreted language users showed the largest increase ($\Delta M = +0.33$) while Web developers showed the largest decrease ($\Delta M = -0.21$). By company size, large companies showed the largest increase ($\Delta M = +0.45$), whilst medium-sized companies showed the largest decrease ($\Delta M = -0.23$). Personal-only users showed notably high increases ($\Delta M = +0.44$).

Mixed-Effects Models of Skill Development Perceptions

To examine factors explaining individual differences in skill development perceptions, we applied mixed-effects modelling ($n = 91$; see Section 4.4.1.4 for model structure).

Table 5.19 shows the progressive model results. Junior engineers were significantly more likely than senior engineers to perceive positive skill development im-

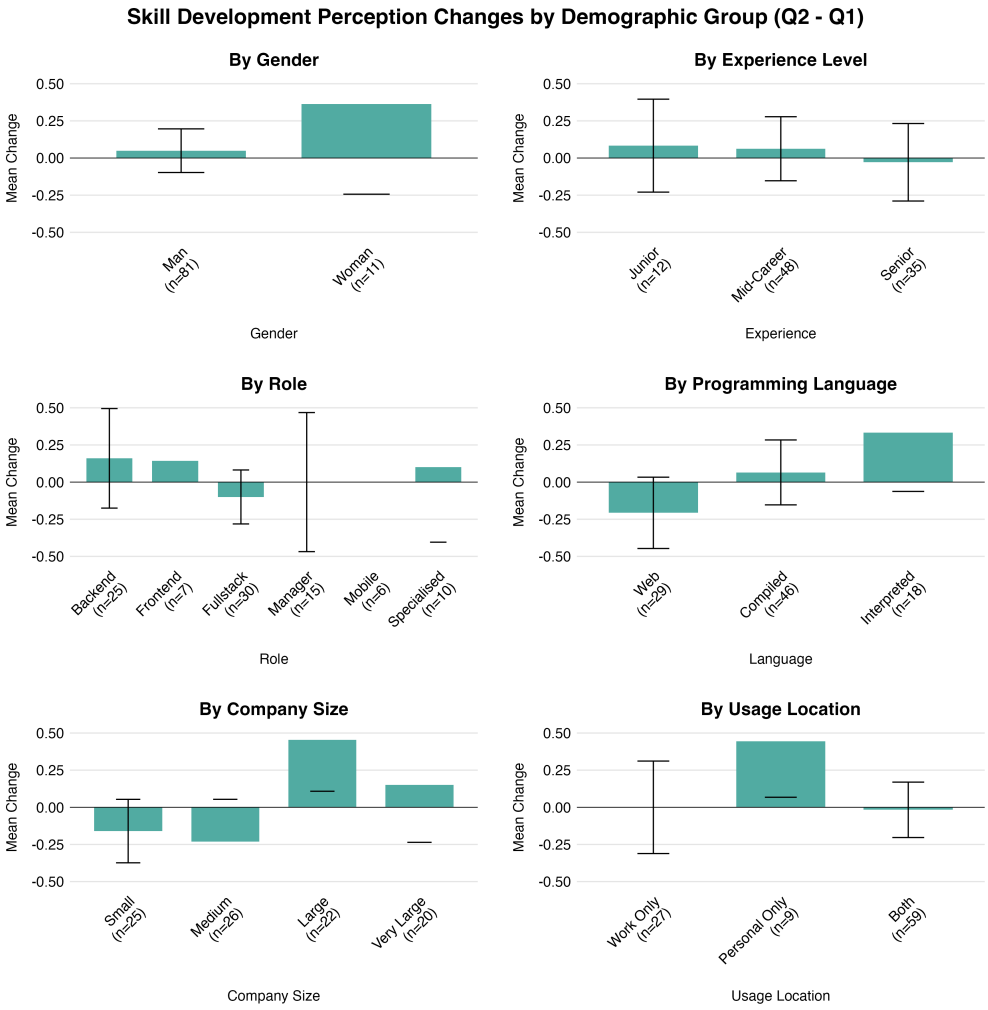


Figure 5.18: Mean changes in skill development perceptions by demographic group from Q1 to Q2. Error bars represent standard error.

Table 5.19: Cumulative Link Mixed Model Analysis of Skill Development Perceptions ($n = 91$)

Predictor	**M1**	**M2**	**M3**
Time (6-month change)	1.13	1.09	1.03
Gender: Woman vs Man	2.23	2.35	3.25
Exp: Junior vs Senior	**5.53***	**4.94***	2.95
Usage Frequency		1.26	1.17
Self-Efficacy			**2.06****
Productivity			0.92
Model Fit			
AIC	551.4	551.9	544.5
ICC (%)	38.0	39.5	36.1

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Productivity.
Values are odds ratios (OR). OR $>$ 1 ($<$ 1) indicates higher (lower) odds of reporting perceived skill development impact.
Continuous predictors z-scored. Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
2 time points (182 observations).

pact in M1, but this effect weakened and became non-significant once self-efficacy was controlled for in M3. Self-efficacy emerged as the only significant predictor in the full model. Time point, gender, usage frequency, and productivity showed no significant effects. Adding psychological factors improved model fit (AIC decreased from 551.4 to 544.5). Individual differences (ICC) remained moderate at 36–39% across models.

5.4.2 Self-Efficacy

5.4.2.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete NGSE responses (Q1: $n = 156$; Q2: $n = 100$).

Figure 5.19 presents item-level responses. Most participants reported no change or increased self-efficacy, with negative ratings rare (0–4% per item). Table 5.20 shows item-level means and standard deviations; most items showed stability or modest changes, with the largest movements in “overcome challenges” (+0.13) and “comparative ability” (−0.14).

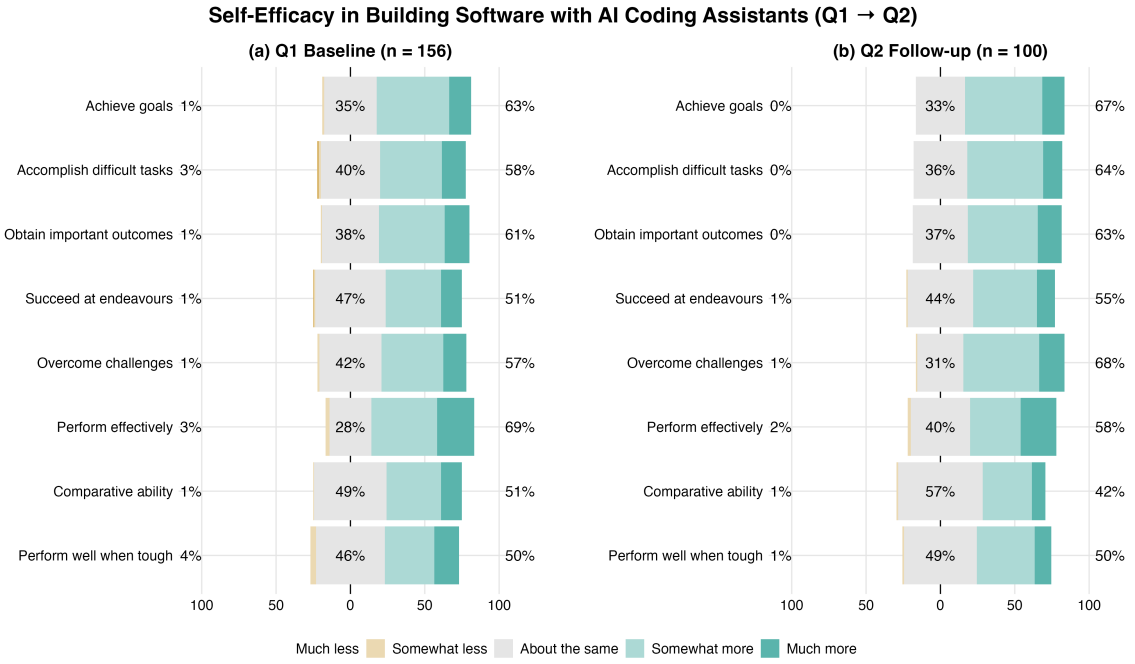


Figure 5.19: Self-efficacy item responses across time points.

Table 5.20: Self-Efficacy Item Means and Standard Deviations by Time Point

Item	**Q1** ($n = 156$)		**Q2** ($n = 100$)		
	M	***SD***	***M***	***SD***	**Δ*M***
Achieve goals	3.77	0.71	3.82	0.67	+0.05
Accomplish difficult tasks	3.70	0.80	3.77	0.66	+0.07
Obtain important outcomes	3.77	0.73	3.79	0.70	+0.02
Succeed at endeavours	3.63	0.75	3.66	0.70	+0.03
Overcome challenges	3.71	0.74	3.84	0.71	+0.13
Perform effectively	3.92	0.79	3.80	0.83	−0.12
Comparative ability	3.64	0.73	3.50	0.67	−0.14
Perform well when tough	3.63	0.80	3.60	0.70	−0.03

Note. Scale: 1 = much less, 3 = about the same, 5 = much more.
ΔM = Q2 mean − Q1 mean. Items presented in standard NGSE order.

Scale Reliability and Composite Scores

Internal consistency was excellent at both time points (Cronbach's $\alpha = 0.91$). Participants reported moderately high self-efficacy at Q1 ($M = 3.72$, $SD = 0.60$) that remained virtually unchanged at Q2 ($M = 3.72$, $SD = 0.56$). One-sample Wilcoxon signed-rank tests showed that composite scores were above the neutral

midpoint at both time points ($p < 0.001$, $r_{rb} = 0.98$ at Q1, $r_{rb} = 1.00$ at Q2; both large effects).

Predictors of Self-Efficacy

We fitted linear regression models to identify factors associated with self-efficacy (predictors per Section 4.4.1.1).

Table 5.21: Linear Regression of Self-Efficacy Composite Score (Q1 vs Q2)

Predictor	$\boldsymbol{\beta}$ ($\boldsymbol{SE}$)	
	Q1	**Q2**
Gender (Reference: Man)		
Woman vs Man	−0.02 (0.16)	−0.20 (0.18)
Experience Level (Reference: Senior)		
Junior vs Senior	0.30 (0.17)	**0.53*** (0.21)
Mid-Career vs Senior	−0.03 (0.11)	−0.02 (0.13)
Role (Reference: Backend)		
Frontend vs Backend	−0.26 (0.26)	−0.33 (0.30)
Fullstack vs Backend	−0.13 (0.15)	−0.17 (0.18)
Manager vs Backend	0.18 (0.17)	0.31 (0.19)
Mobile vs Backend	0.33 (0.20)	−0.07 (0.25)
Specialised vs Backend	−0.05 (0.23)	0.27 (0.27)
Programming Language Group (Reference: Web)		
Compiled vs Web	−0.24 (0.14)	−0.11 (0.17)
Interpreted vs Web	−0.07 (0.16)	−0.08 (0.18)
Company Size (Reference: Medium)		
Small vs Medium	0.23 (0.13)	0.05 (0.15)
Large vs Medium	0.00 (0.15)	0.02 (0.17)
Very Large vs Medium	0.25 (0.15)	0.21 (0.17)
Usage Location (Reference: Both)		
Work Only vs Both	−0.13 (0.12)	**−0.35*** (0.15)
Personal Only vs Both	0.28 (0.21)	0.49 (0.26)
AI Usage Frequency	**0.11*** (0.05)	0.13 (0.07)

β = unstandardised coefficient; SE = standard error.
Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
Q1: $n = 143$, $R^2 = 0.190$, Adj. $R^2 = 0.087$, $F(16, 126) = 1.85$, $p = 0.031$.
Q2: $n = 93$, $R^2 = 0.331$, Adj. $R^2 = 0.190$, $F(16, 76) = 2.35$, $p = 0.007$.

Table 5.21 presents the regression results. At Q1, usage frequency was the only significant predictor. At Q2, model fit substantially improved with two significant

predictors: junior engineers reported higher self-efficacy than senior engineers, and work-only users reported lower self-efficacy than those using AI in both contexts.

AI Tool Selection Shows One Significant Association

We examined whether specific AI tool choices associated with self-efficacy using point-biserial correlations with Holm-Bonferroni correction. GitHub Copilot use showed a significant negative association at both time points (Q1: $r = -0.25$, $p_{adj} = 0.005$; Q2: $r = -0.28$, $p_{adj} = 0.039$). No other tools reached significance.

Attitudes Towards AI Tools Show Selective Associations

Spearman correlations showed a significant positive association between initial impressions and self-efficacy at Q1 ($\rho = 0.32$, $p < 0.001$, moderate effect). At Q2, anticipated disappointment if AI tools were removed showed an even stronger correlation ($\rho = 0.48$, $p < 0.001$, moderate effect). Primary concerns showed no significant association at Q1 (Kruskal-Wallis test, $p = 0.413$) but did at Q2 ($\chi^2 = 18.97$, $p = 0.004$, $\varepsilon^2 = 0.20$, large effect), though small cell sizes limit interpretation.

Cross-Sectional Association with Skill Development Perceptions

Perceived skill development impact and self-efficacy showed a moderate positive correlation at Q1 ($\rho = 0.37$, $p < 0.001$). This association weakened but remained significant at Q2 ($\rho = 0.23$, $p = 0.022$, small effect).

5.4.2.2 Longitudinal Change in Self-Efficacy

Next, we examined how self-efficacy changed within participants ($n = 94$ matched participants).

Aggregate and Individual Stability

Using the Reliable Change Index (RCI; see Chapter 4), most participants showed no reliable change in self-efficacy (70%), while reliable improvement and reliable decline were equally represented (15% each). Those who declined began with higher initial self-efficacy ($M = 4.36$) than those who improved ($M = 3.58$), reflecting regression to the mean. A paired Wilcoxon signed-rank test confirmed no significant aggregate change ($\Delta M = +0.03$, $V = 1713.5$, $p = 0.194$, $r_{rb} = 0.17$, small effect).

Predictors of Change

We fitted an exploratory linear regression with self-efficacy change (Q2 − Q1) as the outcome (Table 5.22).

Table 5.22: Linear Regression: Predictors of Change in Self-Efficacy ($n = 90$)

Predictor	$\boldsymbol{\beta}$ (***SE***)	$\boldsymbol{p}$
Experience Level (Reference: Senior)		
Junior vs Senior	−0.01 (0.20)	0.948
Mid-Career vs Senior	−0.02 (0.12)	0.887
Role (Reference: Backend)		
Frontend vs Backend	−0.33 (0.23)	0.151
Fullstack vs Backend	0.06 (0.15)	0.689
Manager vs Backend	−0.05 (0.17)	0.780
Mobile vs Backend	−0.20 (0.27)	0.458
Specialised vs Backend	0.06 (0.20)	0.775
Company Size (Reference: Medium)		
Small vs Medium	**−0.36*** (0.15)	0.017
Large vs Medium	−0.07 (0.17)	0.694
Very Large vs Medium	−0.18 (0.18)	0.306
AI Usage Frequency	−0.01 (0.06)	0.851

Model fit: $R^2 = 0.111$, Adj. $R^2 = -0.014$, $F(11, 78) = 0.89$, $p = 0.556$.
Positive β indicates shift toward higher self-efficacy relative to reference group.
Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

The overall model was not significant ($F(11, 78) = 0.89$, $p = 0.556$, Adj. $R^2 = -0.014$). One predictor reached individual significance (small vs medium company, $\beta = -0.36$, $p = 0.017$), but this is not reliable given the non-significant overall model.

Longitudinal Alignment with Skill Development Perceptions

We correlated change scores (Q2 minus Q1) for skill development perceptions and self-efficacy within matched participants ($n = 94$). The correlation was not significant ($\rho = 0.10$, $p = 0.349$).

Mixed-Effects Models of Self-Efficacy

To examine factors explaining individual differences in self-efficacy, we applied mixed-effects modelling ($n = 91$; see Section 4.4.1.4 for model structure).

Table 5.23 shows the progressive model results. Junior engineers reported significantly higher self-efficacy than senior engineers, an effect that weakened but persisted after controlling for psychological factors. Perceived productivity was the strongest predictor in the full model, followed by a smaller but significant positive association with perceived skill development impact. Time point, gender, and usage frequency showed no significant effects. Adding psychological factors

Table 5.23: Linear Mixed-Effects Analysis of Self-Efficacy ($n = 91$)

Predictor	**M1**	**M2**	**M3**
Time (6-month change)	0.04	0.03	0.06
	(0.05)	(0.05)	(0.05)
Gender: Woman vs Man	−0.24	−0.23	−0.19
	(0.16)	(0.15)	(0.11)
Exp: Junior vs Senior	**0.48****	**0.44****	**0.27***
	(0.17)	(0.17)	(0.12)
Exp: Mid-Career vs Senior	0.00	−0.02	0.01
	(0.11)	(0.11)	(0.08)
Usage Frequency		0.06	−0.01
		(0.04)	(0.03)
Skill Development			**0.09****
			(0.03)
Productivity			**0.29*****
			(0.04)
Model Fit			
AIC	264.5	264.3	211.3
ICC (%)	60.1	59.4	32.6

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Skill Development, Productivity.
Continuous predictors z-scored. Standard errors in parentheses. Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
2 time points (182 observations).

substantially improved model fit (AIC decreased from 264.5 to 211.3), and the ICC decreased from 60% to 33%, indicating that these factors explained much of the between-participant variance.

5.4.3 Qualitative Perspectives on Skill Development and Self-Efficacy

Reflexive thematic analysis of responses to Q5.3, describing how AI coding assistants had influenced participants' coding skills and practices, identified five themes capturing expanded practical reach, shifted learning processes, confidence gains without mastery, trust calibration, and a recurring speed-depth trade-off.

Theme 1: Expanded practical reach into unfamiliar domains. Across both time points, participants described AI coding assistants as expanding what

they felt practically able to take on, particularly in unfamiliar languages, frameworks, or parts of a codebase. Engineers emphasised practical reach over claims of mastery: the ability to start, explore, and contribute meaningfully in areas that would previously have required substantial ramp-up time. AI-provided examples, scaffolding, and contextual explanations lowered the barrier to entry, enabling engineers to operate across a wider technical surface area.

Participants framed this expansion as a shift in what felt accessible, even without prior experience. One engineer explained that with AI support, *"Syntax becomes just a dialect, with no prior experience of golang i was able to contribute new functionalities to an existing code base"* (P55, Q1). Others described moving into unfamiliar stacks more quickly, for example, *"It helped me pick up Svelte and work in a Svelte project almost instantly"* (P63, Q1), or *"AI has been most helpful for me building a new (side) project from scratch, especially the parts of the stack implemented in languages I'm not very familiar with (Swift) and where I'm rusty (Java)"* (P74, Q2).

In other cases, this expansion compressed the time and effort required to engage with unfamiliar technologies. One noted they could still achieve similar outcomes without AI assistance, but doing so would take substantially longer: *"I can do pretty much anything an AI assistant can do, without its help, but it will take me anywhere from an extra hour to an extra week to do so"* (P14, Q1). In this sense, AI reduced the cost of engaging with unfamiliar work without fundamentally altering skill levels.

Expanded reach was also evident beyond traditional individual contributor boundaries. Participants in engineering management roles described contributing practically despite limited recent hands-on experience. One engineer explained, *"I never wrote much code, being in eng leadership now. But I can now better contribute to the teams efforts by throwing together a quick tool or prototype to help the team or unblock myself"* (P46, Q2).

Theme 2: Learning shifts from recall to evaluation. Participants described a change in how learning occurred when working with AI coding assistants. Where learning once centred on recalling syntax, searching documentation, or assembling information from multiple sources, engineers increasingly described it in terms of evaluating AI-generated suggestions. This involved assessing, comparing, and sense-checking AI-generated suggestions to determine whether they were correct, appropriate, or aligned with established practices.

Central to this shift was exposure to multiple possible approaches, which sometimes surfaced ideas not previously considered. One engineer noted, *"When asking for a solution, there's a tendency to be presented with a number of potential options, which helps broaden your horizons a bit"* (P42, Q1) while another said *"AI will make suggestions that I've not considered and which are sometimes better than my solution."* (P33, Q2). Others described using this optionality to exercise judgement: *"I always ask AI to give me different ways of doing things to examine if there's a better way to perform the task"* (P31, Q1).

This shift was closely linked to changes in information-seeking practices. Participants reported reduced reliance on documentation, search engines, or remembering exact syntax, instead treating AI as a first point of reference that synthesised and contextualised information. One participant explained, *"Rather than having to piece together information from multiple sources, AI assistants make it easy to bring together that information in a way that is conversational and iterative."* (P77, Q1). At Q2, this pattern continued: *"I don't have to remember so many syntax, as well as any new features released by libs or framework"* (P51, Q2).

Learning was also described as occurring through review of AI output. Instead of generating solutions independently, participants described learning by inspecting, adapting, or correcting suggestions provided by the assistant. As one noted, *"I may learn new techniques when reviewing AI generated code"* (P41, Q2), reframing learning as judgement applied to generated material.

Several participants described changes in how they approached learning new languages or technologies. One reflected, *"It has definitely changed how I approach learning languages, as I don't bother with things like specific formatting and other things as chatgpt does it for me"* (P180, Q1). Learning, in these accounts, was increasingly experienced as deciding what to accept, modify, or reject, reflecting a shift in how learning occurs rather than a claim that it was deeper or more complete. AI functioned as a generator of candidate solutions to be evaluated, not a source of authoritative answers.

Theme 3: Confidence through reduced friction, not mastery. Participants linked the use of AI coding assistants to increased confidence in their day-to-day work across both time points. This confidence emerged from reduced friction in the development process rather than from gaining mastery or deep expertise. Engineers described faster feedback loops, lower cognitive load, and

easier experimentation as making it less daunting to attempt new ideas, explore unfamiliar areas, or iterate on early solutions.

Participants described feeling more willing to venture beyond familiar territory when AI support was available. One participant noted, *"I feel more confident exploring new topics, languages and frameworks outside my comfort zone"* (P131, Q1), while another explained that it *"improved my confidence, especially when working with languages and frameworks I'm less familiar with."* (P91, Q2). This willingness stemmed from reduced effort and risk of failure. As one participant described, *"Before it would feel overwhelming or like it'll take a long time to validate. Now I can use a coding assistant to validate what I think might be the bigger problem quickly, and am more open to trying different ideas"* (P27, Q2).

Confidence was also linked to the ability to prototype and iterate more freely. Participants described being able to test ideas without becoming overly invested in them, for example, *"as I spend less time and effort on the code for these, I'm not attached to continuing ideas that may not be feasible"* (P140, Q2). In this sense, AI support reduced the perceived risk of exploration, particularly wasted time, allowing participants to abandon or refine approaches more freely.

Some participants framed AI as a readily available, non-judgemental collaborator, for example, *"like having a team member with you at all times who can discuss with you much as you need"* (P150, Q2) and *"I can use it to help me as if I have a coding expert sitting beside me"* (P200, Q1). This was particularly relevant for those working alone or reluctant to interrupt colleagues, with one noting, *"I am habitually a slow starter and find it hard to 'bug' other people for help/feedback - coding assistants help with that"* (P158, Q1).

Underlying these accounts was an emphasis on reduced cognitive and emotional effort rather than increased skill. Participants described being able to focus on higher-level concerns while delegating routine or tedious aspects of work, for example, *"Let's me focus on the important stuff, and let the code assistant deal with the trivial tasks"* (P62, Q1), or *"It is just an assistant that helps me make the boring parts of my work with less effort"* (P23, Q2). This reduction in effort was closely tied to confidence, with participants describing development as feeling less intimidating and more approachable.

Overall, participants described increased confidence as a by-product of reduced friction instead of evidence of mastery. AI coding assistants lowered the perceived cost of exploration, enabling engineers to try, iterate, and discard ideas more freely, while remaining aware that their underlying skill levels had not fundamentally changed.

Theme 4: Trust calibration as ongoing engineering work. Whereas Theme 2 describes how learning occurs (evaluation of alternatives), this theme concerns risk management and quality control in production contexts. Across both time points, participants described the need to actively calibrate trust in AI-generated output. Instead of treating AI output as authoritative or autonomous, engineers framed its use as requiring ongoing oversight, review and judgement, especially for complex, critical, or production-grade code.

Participants described AI-generated code as useful but imperfect, with one noting, *"I cannot trust these examples to be correct, but I can trust them to be close enough to provide the general shape and context of using a particular technology"* (P8, Q1). Others described AI output as approximate rather than definitive, for example, *"Usually it is right, but it can be confident and wrong."* (P12, Q1). This increased the importance of thorough code reviews, with one noting, *"you need to thoroughly review every line of code and it makes several mistakes that you may not see, because the code seems to be well written"* (P127, Q1). This came at the cost of wasted time in some cases: *"Except about 50% of the time it's wrong, and I've wasted my time"* (P36, Q2).

Trust calibration was strongly context-dependent. Participants reported being more cautious when working on backend systems, production code, or architecture-level concerns. As one engineer explained, *"I'm using it mainly for front end, which I feel is ok, I would be worried if I did backend this way"* (P6, Q1), while another stated, *"I'm less confident I'm using the "best" architecture and platform."* (P205, Q2).

At Q2, participants continued to describe active trust calibration, including selectively accepting suggestions and overriding them when necessary. One participant noted, *"I can differentiate between when I just want to accept what the AI assistant suggests and when I want to do something better"* (P204, Q2), while another described rolling back generated code to implement their own approach: *"I often will rollback the code the tool has written even if it fixes the problem and then design my own solution"* (P27, Q2). Increased familiarity with AI tools did not necessarily result in greater trust, but instead supported more nuanced critical judgement about when to rely on AI and when to intervene. In this sense, trust calibration emerged as an ongoing form of engineering labour, requiring sustained attention rather than diminishing over time.

Theme 5: The speed-depth trade-off. Alongside perceived benefits, participants articulated a tension between speed and depth when using AI coding

assistants. While AI tools enabled faster progress and reduced effort, many engineers expressed concern that short-term efficiency could come at the cost of longer-term understanding, memory retention, or independent problem-solving.

Several participants linked speed gains to reduced learning or engagement with fundamentals. One noted, *"Because I am just copying and pasting, it is faster to get results, but I don't pick up the language as well as when I used to code from scratch"* (P6, Q1), while another stated more bluntly, *"but deeper understanding gets less and less with this."* (P125, Q1). Others described reasoning through problems less fully, for example: *"I am not thinking as much about the code I write but just verifying the responses from the coding assistant"* (P154, Q1).

This concern appeared alongside reduced engagement with traditional learning practices. Participants reported reading less documentation or relying less on independent research, with one explaining, *"the downside is that I read much less documentation and articles"* (P153, Q1). Some framed this as an erosion of problem-solving practice, noting that outsourcing difficulty could weaken skills over time: *"problem-solving is like any skill that you have to train to keep getting better... I'm seeing some deterioration in that aspect"* (P139, Q1). By Q2, some had begun noticing this trend: *"I notice I learn less"* (P63, Q2) and another forecasting declines in skills and practices: *"my coding skills will be worse. my practices will be worse, because the ai takes care of those details for me."* (P179, Q2).

Not all participants perceived negative effects; a substantial subset described little or no impact on their skills. As one noted, *"I don't think it influences my coding skills at this point but rather speeds them up"* (P5, Q1). Those with more experience or confidence felt less impact, for example, *"It hasn't influenced my skill level as I am already quite senior"* (P56, Q1). Similar sentiments appeared at Q2, for example, *"I was doing this before. I will continue doing this after"* (P8, Q2).

Across both time points, these accounts reflected a perceived trade-off rather than a single trajectory. While AI coding assistants reduced effort and accelerated progress, participants varied in how they weighed these benefits against concerns about learning depth, over-reliance, and long-term skill maintenance. Engineers described actively navigating this balance, rather than uniformly experiencing either improvement or decline.

5.4.4 Summary

Perceived skill development impact was stable at the group level (53–57% perceiving impact), though this masked meaningful individual variation, with opposing movements cancelling out at the aggregate level. Self-efficacy when building software was moderately high and stable ($M = 3.72$), with Reliable Change Index analysis confirming 70% showed no reliable change. The two constructs were positively associated (OR = 2.06, $p = 0.003$), and perceived productivity gains were the strongest predictor of self-efficacy ($\beta = 0.29$, $p < 0.001$). Junior engineers reported higher self-efficacy than seniors, an effect that persisted after controlling for other factors. GitHub Copilot was the only tool negatively associated with self-efficacy, at both time points. Qualitatively, participants described expanded practical reach, a shift in learning from recall to evaluation of AI-generated options, confidence emerging from reduced friction rather than mastery, trust calibration as ongoing engineering work, and a speed-depth trade-off between short-term efficiency and longer-term skill development.

5.5 RQ4: Prompt Engineering

This section addresses RQ4: *How do software engineers perceive the role of prompt engineering as a new core competency in the AI-assisted era of software development?* We analyse two dimensions: perceived future importance and perceived skills replacement.

5.5.1 Baseline Perceptions and Cross-sectional Evolution Across Time Points

Cross-sectional analyses use participants with complete responses (Q1: $n = 157$; Q2: $n = 101$).

Perceived Future Importance

At Q1, participants rated prompt engineering's future importance as moderate-to-high ($M = 3.55$, $SD = 1.21$), with 56% rating it very or extremely important. At Q2, these perceptions increased slightly ($M = 3.72$, $SD = 1.21$), with 62% rating it very or extremely important (Figure 5.20).

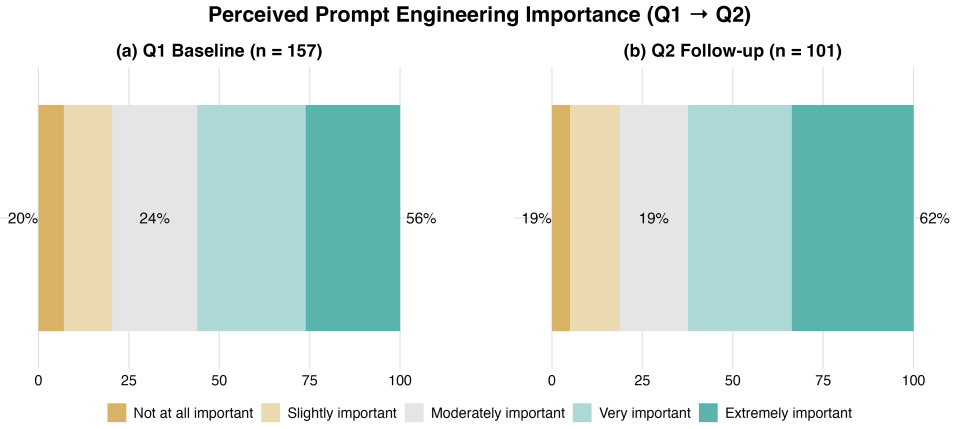


Figure 5.20: Perceived importance of prompt engineering across time points. *Note.* Scale is unipolar; midpoint (3) = "Moderately important".

Skills Replacement Beliefs

At Q1, participants were divided on whether prompt engineering would replace traditional coding skills ($M = 2.97$, $SD = 1.30$), with 48% agreeing and 41% disagreeing. At Q2, perceptions shifted slightly toward agreement ($M = 3.22$, $SD = 1.21$), with 50% agreeing (Figure 5.21). One-sample Wilcoxon signed-rank

tests showed that ratings did not differ from the neutral midpoint at Q1 ($p = 0.528$, $r_{rb} = -0.06$, very small effect), though Q2 approached significance ($p = 0.093$, $r_{rb} = 0.20$, small effect).

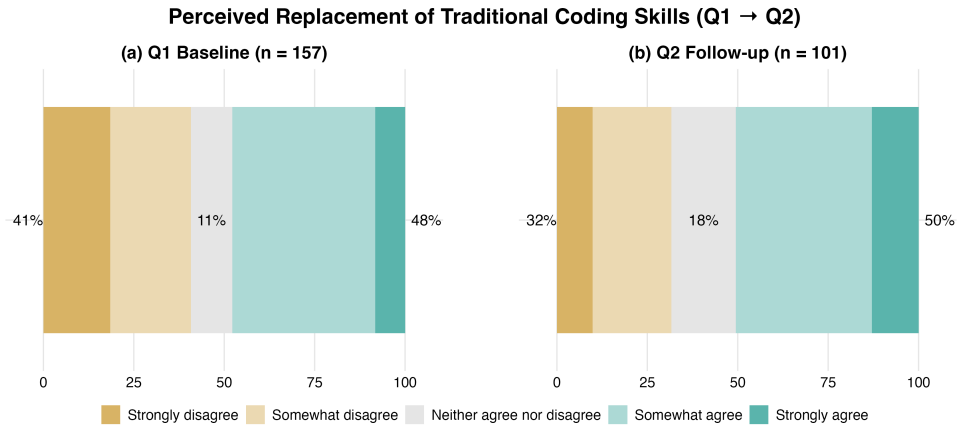


Figure 5.21: Agreement that prompt engineering will replace traditional coding skills across time points.

Relationship Between Dimensions

Importance and replacement perceptions showed a strong positive correlation at both time points ($\rho = 0.59$, $p < 0.001$, large effect).

Additive and Substitutive Perception Profiles

We examined the prevalence of perception profiles defined in Section 4.4.1.5. The *Substitutive* profile was most common at both time points (Q1: 36%, $n = 57$; Q2: 43%, $n = 43$), followed by *Sceptics* (Q1: 17%, $n = 27$; Q2: 14%, $n = 14$) and the *Additive* profile (Q1: 12%, $n = 19$; Q2: 10%, $n = 10$). The remainder held moderate positions on one or both dimensions (Q1: 33%; Q2: 33%). We focus on the Additive and Substitutive profiles because both groups consider prompt engineering important, making their disagreement about skill replacement theoretically meaningful.

Predictors of Importance Perceptions

We fitted ordinal logistic regression models to identify factors associated with perceived importance of prompt engineering (predictors per Section 4.4.1.1).

Table 5.24 presents the regression results. Programming language group was the only significant predictor at both time points: web developers (JavaScript,

Table 5.24: Ordinal Logistic Regression of Prompt Engineering Importance (Q1 vs Q2)

Predictor	**Q1**		**Q2**	
	OR	**95% CI**	**OR**	**95% CI**
Gender (Reference: Man)				
Woman vs Man	0.86	[0.33, 2.22]	1.44	[0.39, 5.41]
Experience Level (Reference: Senior)				
Junior vs Senior	1.68	[0.58, 4.94]	0.86	[0.19, 3.77]
Mid-Career vs Senior	1.03	[0.53, 2.02]	0.83	[0.34, 2.02]
Role (Reference: Backend)				
Frontend vs Backend	0.37	[0.08, 1.82]	0.16	[0.02, 1.31]
Fullstack vs Backend	0.46	[0.18, 1.20]	0.86	[0.25, 2.98]
Manager vs Backend	0.89	[0.30, 2.61]	0.95	[0.25, 3.62]
Mobile vs Backend	0.79	[0.23, 2.75]	1.03	[0.19, 5.53]
Specialised vs Backend	0.71	[0.18, 2.77]	0.45	[0.07, 2.80]
Programming Language Group (Reference: Web)				
Compiled vs Web	**0.30****	[0.12, 0.74]	**0.11****	[0.03, 0.42]
Interpreted vs Web	**0.31***	[0.11, 0.86]	**0.21***	[0.05, 0.80]
Company Size (Reference: Medium)				
Small vs Medium	1.35	[0.59, 3.09]	2.69	[0.86, 8.37]
Large vs Medium	1.91	[0.76, 4.82]	2.45	[0.75, 7.97]
Very Large vs Medium	1.77	[0.66, 4.73]	2.32	[0.68, 7.87]
Usage Location (Reference: Both)				
Work Only vs Both	0.51	[0.24, 1.07]	0.97	[0.36, 2.61]
Personal Only vs Both	0.45	[0.14, 1.49]	2.66	[0.42, 16.82]
AI Usage Frequency	0.98	[0.72, 1.34]	0.85	[0.53, 1.35]

OR = Odds Ratio for moving to a higher response category; CI = 95% Confidence Interval.
Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
Q1: $n = 144$, McFadden $R^2 = 0.042$.
Q2: $n = 94$, McFadden $R^2 = 0.071$.

TypeScript) rated importance substantially higher than users of compiled or interpreted languages, and this effect strengthened from Q1 to Q2. No other predictors, including demographic factors, reached significance at either time point.

Predictors of Replacement Beliefs

We fitted the same ordinal logistic regression models for replacement beliefs.

Table 5.25: Ordinal Logistic Regression of Skills Replacement Agreement (Q1 vs Q2)

Predictor	**Q1**		**Q2**	
	OR	**95% CI**	**OR**	**95% CI**
Gender (Reference: Man)				
Woman vs Man	2.21	[0.84, 5.82]	0.96	[0.26, 3.61]
Experience Level (Reference: Senior)				
Junior vs Senior	1.98	[0.72, 5.42]	0.39	[0.08, 1.87]
Mid-Career vs Senior	1.68	[0.84, 3.36]	0.54	[0.22, 1.35]
Role (Reference: Backend)				
Frontend vs Backend	**0.13***	[0.03, 0.63]	0.76	[0.10, 5.96]
Fullstack vs Backend	**0.25****	[0.10, 0.66]	0.55	[0.15, 1.92]
Manager vs Backend	0.36	[0.12, 1.08]	1.87	[0.48, 7.36]
Mobile vs Backend	0.58	[0.15, 2.24]	0.94	[0.18, 5.03]
Specialised vs Backend	0.34	[0.08, 1.42]	3.05	[0.50, 18.72]
Programming Language Group (Reference: Web)				
Compiled vs Web	0.54	[0.22, 1.34]	**0.20***	[0.06, 0.69]
Interpreted vs Web	0.57	[0.22, 1.48]	**0.25***	[0.06, 0.93]
Company Size (Reference: Medium)				
Small vs Medium	1.86	[0.83, 4.17]	**4.74***	[1.42, 15.75]
Large vs Medium	1.24	[0.48, 3.21]	1.20	[0.37, 3.94]
Very Large vs Medium	0.86	[0.32, 2.36]	2.01	[0.63, 6.47]
Usage Location (Reference: Both)				
Work Only vs Both	0.55	[0.27, 1.15]	0.83	[0.28, 2.45]
Personal Only vs Both	0.77	[0.21, 2.83]	2.98	[0.51, 17.35]
AI Usage Frequency	0.79	[0.57, 1.11]	0.97	[0.61, 1.56]

OR = Odds Ratio for moving to a higher response category; CI = 95% Confidence Interval.
Significance: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
Q1: $n = 144$, McFadden $R^2 = 0.044$.
Q2: $n = 94$, McFadden $R^2 = 0.091$.

Table 5.25 presents the regression results. Unlike importance, the significant predictors of replacement beliefs shifted between time points. At Q1, role was significant: frontend and fullstack developers showed lower agreement than backend developers. By Q2, role effects had disappeared; instead, programming language

group became significant (web developers higher than compiled or interpreted), and small company size emerged as a predictor of higher agreement. Demographic factors showed no significant effects at either time point.

Tool Use and Attitudes Show No Associations

We examined whether specific AI tool choices associated with importance or replacement perceptions using Fisher's exact tests with Holm-Bonferroni correction. No tools reached significance at either time point. Spearman correlations showed no significant associations between initial impressions at Q1 or anticipated disappointment at Q2 and either dimension. Primary concerns showed no significant association at either time point (Kruskal-Wallis tests with Holm-Bonferroni correction).

5.5.2 Longitudinal Change in Prompt Engineering Perceptions

Next, we examined how prompt engineering perceptions changed within participants ($n = 95$ matched participants).

Aggregate Stability Masks Individual Variation

Paired Wilcoxon signed-rank tests with Holm-Bonferroni correction showed no significant aggregate change on either dimension (Table 5.26). However, this stability masked individual variation: only 37% maintained complete stability, while 63% changed on at least one dimension.

Table 5.26: Longitudinal Wilcoxon Signed-Rank Tests: Changes in Prompt Engineering Perceptions Within Matched Participants ($n = 95$)

Dimension	**Q1**		**Q2**			**Test Statistics**		
	M	SD	M	SD	ΔM	V	p_{adj}	r_{rb}
Importance	3.65	1.15	3.73	1.20	+0.07	831.0	0.848	0.12
Replacement	3.13	1.25	3.23	1.20	+0.11	916.5	0.848	0.11

ΔM = Mean change (Q2 - Q1). V = Wilcoxon signed-rank test statistic. r_{rb} = rank-biserial correlation.
Significance: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

Transition Patterns

Table 5.27 presents the transition patterns. The most common shift was both perceptions increasing (14%), followed by single-dimension changes (33% combined). Only 8% showed decreases on both dimensions, while another 8% showed divergent movements.

Table 5.27: Longitudinal Transition Pattern Categories in Prompt Engineering Perceptions ($n = 95$)

Transition Pattern	**n**	**Percentage**
Complete Stability	35	37%
Both Perceptions Increased	13	14%
Importance Down, Replacement Stable	8	8%
Importance Stable, Replacement Down	8	8%
Importance Stable, Replacement Up	8	8%
Both Perceptions Decreased	8	8%
Importance Up, Replacement Stable	7	7%
Importance Up, Replacement Down	5	5%
Importance Down, Replacement Up	3	3%
Total Changed	**60**	**63%**
Total Stable	**35**	**37%**

Percentages may not sum to 100% due to rounding.

Examining each dimension separately revealed asymmetric movement: importance ratings increased for 31% versus decreased for 26%, while replacement beliefs showed a similar pattern (33% up vs 27% down). Stability also varied by initial position: 65% of those initially rating importance highest (5) maintained that view, compared to only 19% at the midpoint (3). Replacement beliefs showed more uniform stability across initial ratings (33–44%).

Perception Combination Stability

Figure 5.22 visualises profile transition flows. The Substitutive profile showed the highest stability (63% retention, 24/38), whilst Additive showed complete instability (0/12 retained), with 42% transitioning to Substitutive by Q2. Sceptic (46%) and Moderate (43%) showed intermediate stability.

Demographic Differences in Change Patterns

To explore whether demographic factors were associated with perception changes, we examined mean changes across gender, experience, role, programming language, company size, and usage location (Figure 5.23). These patterns are descriptive; we did not test between-group differences inferentially given sample sizes.

Most groups showed increases on both dimensions, though with notable exceptions. For importance, personal-only users showed the largest increase ($\Delta M = +0.89$), followed by women ($\Delta M = +0.55$) and managers ($\Delta M = +0.53$). Backend, mobile, and specialised roles decreased; mid-career engineers decreased

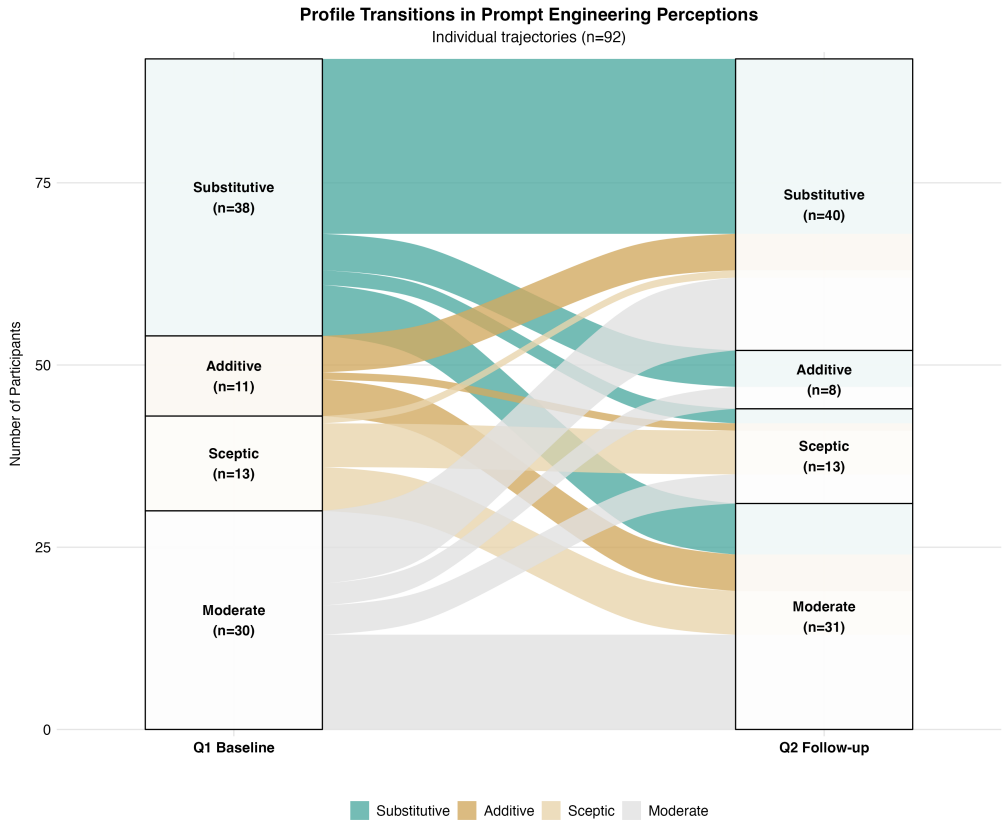


Figure 5.22: Profile transitions between Q1 and Q2. Displaced profile ($n = 2$) excluded for visual clarity.

while seniors increased. For replacement, frontend ($\Delta M = +0.86$) and managers ($\Delta M = +0.80$) showed the largest gains, while backend ($\Delta M = -0.36$) and mobile ($\Delta M = -0.33$) showed the largest decreases, with fullstack, mid-career, compiled language users, and large companies also decreasing. Overall, backend and mobile consistently decreased on both dimensions, while frontend and managers consistently increased on both.

5.5.2.1 Predictors of Perception Evolution

To examine factors explaining individual differences in prompt engineering perceptions, we applied mixed-effects modelling ($n = 91$; see Section 4.4.1.4 for model structure). Given the strong correlation between dimensions (Section 5.5.1), we included each as a covariate when modelling the other.

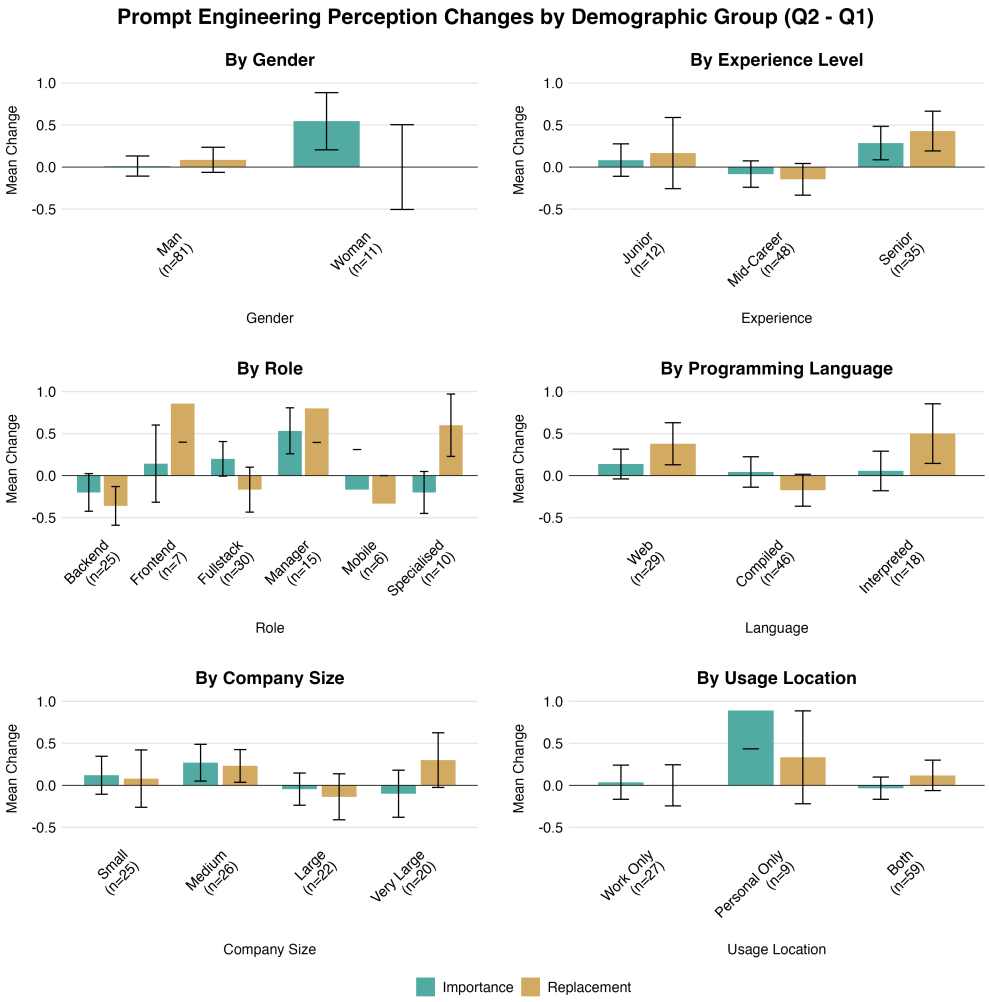


Figure 5.23: Mean changes in prompt engineering perceptions by demographic group from Q1 to Q2. Error bars represent standard error.

Predictors of Future Importance Perceptions

Table 5.28: Cumulative Link Mixed Model Analysis of Importance Perceptions ($n = 91$)

Predictor	**M1**	**M2**	**M3**
Time (6-month change)	1.29	1.36	1.16
Gender: Woman vs Man	0.64	0.59	0.57
Exp: Junior vs Senior	1.78	2.22	1.90
Usage Frequency		0.71	0.76
Self-Efficacy			1.08
Productivity			**1.62***
Replacement			**3.87****
Model Fit			
AIC	513.8	513.2	462.7
ICC (%)	57.3	57.4	19.1

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Productivity, Replacement.
Values are odds ratios (OR). OR > 1 (< 1) indicates higher (lower) odds of rating prompt engineering as more important.
Continuous predictors z-scored. Significance: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.
2 time points (182 observations).

Table 5.28 shows the progressive model results. Replacement beliefs emerged as the dominant predictor, with each standard deviation increase associated with nearly four times higher odds of rating prompt engineering as more important. Productivity perceptions also showed a significant positive association. Demographics, usage frequency, self-efficacy, and time point showed no significant effects. Adding psychological factors substantially improved model fit (AIC decreased from 513.8 to 462.7), and the ICC decreased from 57% to 19%.

Predictors of Skills Replacement Beliefs

Table 5.29 shows the progressive model results. Importance perceptions emerged as the only significant predictor, with each standard deviation increase associated with nearly four times higher odds of agreeing that prompt engineering will replace traditional skills. Self-efficacy approached but did not reach significance. Demographics, usage frequency, productivity, and time point showed no significant effects. Adding psychological factors substantially improved model fit (AIC decreased from 537.2 to 477.1), and the ICC decreased from 33% to 8%.

Table 5.29: Cumulative Link Mixed Model Analysis of Replacement Perceptions ($n = 91$)

Predictor	**M1**	**M2**	**M3**
Time (6-month change)	1.15	1.21	1.03
Gender: Woman vs Man	1.28	1.21	1.85
Exp: Junior vs Senior	0.84	1.02	0.73
Usage Frequency		0.73	0.87
Self-Efficacy			1.49
Productivity			0.86
Importance			**3.80*****
Model Fit			
AIC	537.2	536.2	477.1
ICC (%)	33.1	32.7	8.3

M1 = Gender, Experience; M2 = M1 + Usage Frequency; M3 = M2 + Self-Efficacy, Productivity, Importance.
Values are odds ratios (OR). OR > 1 (< 1) indicates higher (lower) odds of agreeing that prompt engineering will replace traditional skills.
Continuous predictors z-scored. Significance: *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.
2 time points (182 observations).

Comparing Across Both Dimensions

Both dimensions showed similar patterns: psychological factors substantially improved fit while usage frequency did not, and perceptions were stable over the six-month period. Demographics showed no significant effects for either dimension. The dimensions were bidirectionally linked, with each emerging as the strongest predictor of the other at comparable magnitudes. Secondary predictors diverged: productivity was associated with importance but not replacement, while self-efficacy showed no significant association with importance and approached but did not reach significance for replacement ($p = 0.052$).

5.5.3 Qualitative Perspectives on Prompt Engineering

Reflexive thematic analysis of responses to Q6.3, which asked how engineers should develop prompt engineering skills (Q1) or how their approach had evolved (Q2), identified four themes capturing how participants conceptualised prompting as intent specification, linked it to productivity gains, contested whether it substitutes for or depends upon traditional skills, and debated its legitimacy as an enduring competency.

Theme 1: Recasting prompting as intent specification and communication. Participants cast prompt engineering as a reframing of familiar engineering practices centred on articulating intent, specifying constraints, and communicating clearly with a collaborator. Rather than emphasising specialised prompting techniques or treating it as a distinct discipline, participants described effective prompting as a continuation of how engineers already explain problems, define requirements, and set expectations.

Sub-theme 1A: Prompting as context-setting and constraint specification. Across both time points, participants described effective prompting as providing clear context, boundaries, and expectations so that AI output aligned with the task and system at hand. At Q1, prompting was likened to explaining a problem to another engineer, for example: *"Better input descriptions will result in better outcomes"* (P16, Q1) and *"I get very good success with the best models by just explaining the problem in the language I'd use when messaging another engineer"* (P155, Q1). By Q2, participants described refining this practice through greater structure and specificity, spending more effort upfront to guide the assistant: *"I've realized that the more context and direction I give, the better the results are."* (P31, Q2). Several participants emphasised specifying technical constraints such as libraries or architectural context, tying prompt quality to understanding the problem space rather than mastering prompt syntax.

Sub-theme 1B: Prompting as iterative dialogue and negotiation. Participants across both time points described prompting as an iterative, conversational process. Rather than expecting correct output immediately, engineers reported refining prompts in response to AI output, adjusting phrasing, adding examples, or narrowing scope to steer results toward what they needed. At Q1, this was framed as trial-and-error refinement, for example: *"You try, you dont get quite what you want, you alter your query"* (P18, Q1), and *"Developers know the outcome they want so it's a matter of asking the right questions and fine tuning the prompt until the desired effects are achieved"* (P113, Q1). By Q2, participants more explicitly described this interaction as an ongoing back-and-forth, with fewer expecting the first response to be sufficient. Several noted deliberate prompt refinement over multiple turns, such as *"If I don't get the answer I'm looking for, then I follow up by improving my prompt and providing more info"* (P103, Q2), and *"I'm rarely happy with the first result, and refine over time"* (P187, Q2). A smaller subset framed this process as conversational collaboration, describing the interaction as dialogue more than instruction, for example: *"Now it feels more like a conversation"* (P153, Q2).

Sub-theme 1C: Prompting as self-directed skill-building through informal and formal learning resources. Participants also reflected on how prompting practices are learned and supported. Across both time points, participants described developing prompt engineering practices through a wide and largely informal set of learning resources rather than through a single, agreed pathway. At Q1, responses referenced an eclectic mix of experimentation, online courses, provider documentation, blogs, videos, forums, tutorials, templates, and peer advice, with several participants noting *"There's an absolute abundance of resources out there about prompt engineering now"* (P42, Q1). By Q2, participants who reflected on skill development tended to describe a more intentional engagement with learning, including selective use of formal courses or structured materials, for example: *"I did a Coursera certification in generative AI for software engineers"* (P127, Q2). Overall, these accounts suggest that participants perceive self-learning resources for developing prompt-related practices as plentiful and readily available.

Theme 2: Prompting as a gateway to productivity and expanded reach. Participants framed prompt engineering as valuable across both time points because it enabled practical benefits: faster iteration, reduced rework, and smoother progress toward a usable result. Rather than treating prompting as an end in itself, participants positioned it as a means of unlocking efficiency: *"Understanding the nuances between a regular prompt and a well crafted one will bring speed and efficiency to the user"* (P56, Q1). This productivity framing was more explicit at Q2, where participants described prompt refinement as reducing back-and-forth and iterations, such as *"I've become more directive about the format and structure of output I'm expecting. This respectively gives me a better quality answer and decreases the back-and-forth"* (P74, Q2), and *"This has strongly diminished the number of iterations to get to the end result I want"* (P159, Q2). Others described being more deliberate upfront to avoid later correction costs: *"Where I used to rush through writing prompts and spend a lot of time correcting code later. I now spend more time writing a longer, more specific prompt to achieve better results"* (P15, Q2). These accounts frame prompt engineering as pragmatic enablement, providing a way of accessing speed, convenience, and broader reach through better AI leverage, rather than a standalone technical expertise.

Theme 3: Contesting whether prompting substitutes for, or depends upon, core software engineering skills. Participants did not converge on

a single view of how prompt engineering relates to software engineering expertise. Instead, they framed it as an ongoing tension between two interpretations: prompting as a legitimate shift in where engineering value sits, versus prompting as something that only works well when grounded in traditional competencies like problem definition, design judgement, and code evaluation. This tension was not just about technique; it reflected deeper beliefs about what constitutes durable engineering skill and where accountability remains when AI produces code.

Sub-theme 3A: Prompting as additive and dependent on foundational skills. Many participants described prompt engineering as dependent on strong software engineering fundamentals, particularly the ability to evaluate correctness, manage trade-offs, and remain accountable for the outcome. At Q1, several explicitly rejected substitution, emphasising that oversight requires traditional competence: *"It doesn't replace them because anything AI generated needs to be reviewed and understood which requires those traditional skills"* (P82, Q1) and *"I think engineers still need to know basic coding skills in order to debug the generated code!"* (P102, Q1). Others described effective prompting as constrained by robust engineering skills, *"However to write and refine prompts engineers need to know how to code, the structure they want, the architecture they want"* (P113, Q1). This same stance appears in comparative framings that treat prompting as analogous to "search skill", where it was described as useful but only when paired with discernment. One participant noted this *"will be the difference between an engineer who can leverage AI coding assistants to become more productive and craft quality solutions and those who either get lost in the weeds or end up "copy and pasting" substandard solutions into their code"* (P17, Q1). At Q2, this focus on software engineering fundamentals persisted, with participants describing deliberate retention of engineering responsibility even when using AI for speed, such as: *"The shift has been to rely on the AI only for the speed it brings but to take a little more time on those non-functional requirements of maintainability, testability and ease of enhancement."* (P47, Q2).

Sub-theme 3B: Prompting as substitutive and shifting the locus of value. A contrasting set of responses anticipated that as models and tooling improve, some traditional implementation skills will matter less in day-to-day practice, with value shifting toward intent specification, orchestration, and evaluation. At Q1, some participants saw this shift reshaping roles and team structures: *"Prompt engineering will be a mandatory part of the toolkit moving forward"* (P14, Q1), while another projected that *"seniors will become 5x productivity architects, while the*

incentives to hire juniors at all drops significantly" (P206, Q1). Others suggested that improved models already reduce the need for careful prompting, for instance: *"LLMs are already at the point where I can throw in instructions that are so bad a human would have no idea what I'm asking for, but an LLM can still provide the right answer"* (P19, Q1) and *"I put no effort into prompt engineering because the results are very good even with terrible English"* (P155, Q1).

By Q2, participants described a shift in what they understood "prompt engineering" to involve as AI systems became more capable and better integrated into everyday workflows. Instead of describing prompting as a specialised interaction technique, some participants recast it as a higher-level coordination activity, closer to planning and orchestration than low-level instruction. One participant characterised this shift directly, stating that *"This is less prompt engineering than it is project planning. You'd need to be doing similar task breakdowns to manage your commits, or to share work with multiple developers"* (P8, Q2). In this framing, value was seen to move away from careful phrasing or syntactic precision toward defining scope, sequencing work, and evaluating outcomes. Taken together, these accounts position prompting not as an enduring micro-skill, but as part of a broader reallocation of engineering effort toward higher-level direction-setting and oversight as AI systems absorb more implementation detail.

Theme 4: Scepticism and boundary-setting around "prompt engineering" as a skill. A subset of participants expressed scepticism toward framing prompt engineering as a core or enduring software engineering competency. Instead of describing low AI usage, these accounts reflected boundary-setting around what participants considered legitimate engineering skill, often positioning prompt engineering as transient, hype-driven, or an artefact of current tool limitations. From this perspective, durable value was seen to reside in foundational engineering judgement, accountability, and system understanding rather than in learning tool-specific interaction techniques.

At Q1, scepticism was articulated through rejection of the term itself, with participants questioning its legitimacy or longevity, for example: *""Prompt Engineering" seems like a fad"* (P36, Q1) and *"So, TLDR, forget "prompt engineering" its just an AI-influencer-on-LinkedIn-term not a real thing"* (P19, Q1). Others framed it as a temporary workaround that would fade as tools improve, such as: *"The better AI gets, the less relevant prompt engineering gets"* (P49, Q1). Several participants argued that responsibility should shift toward tools adapting to

users rather than the reverse, stating: *"If it needs to be fancier and more dark art than normal communication, that's a shortcoming of the model"* (P147, Q1). At Q2, this scepticism sometimes extended to outright rejection, such as *"No. I am not learning how to talk to a robot. A robot needs to learn how to understand me"* (P18, Q2). However, this scepticism was more often portrayed as a claim about professional legitimacy than resistance to AI, drawing boundaries between enduring software engineering competence and practices perceived as tool-specific or temporary.

5.5.4 Summary

Participants rated prompt engineering's future importance as moderate-to-high (56–62% very or extremely important), while views on skills replacement were divided, with a small shift toward agreement. The two dimensions were strongly correlated ($\rho = 0.59$, large effect) and bidirectionally associated in mixed-effects models (OR ≈ 3.8). Web developers rated importance significantly higher than compiled or interpreted language users at both time points, and this effect strengthened over time. Replacement belief predictors shifted: role differences at Q1 gave way to programming language and company size effects at Q2. Among matched participants, aggregate stability masked individual movement: 63% changed on at least one dimension. Profile analysis revealed contrasting patterns: the Substitutive profile showed 63% retention, whilst the Additive profile showed complete instability (0% retention), with 42% moving toward Substitutive by Q2. Qualitatively, participants framed prompting as intent specification and a gateway to productivity, but did not converge on whether it substitutes for or depends upon traditional skills, with a subset expressing scepticism about its legitimacy as an enduring competency.

Chapter 6

Discussion

This chapter interprets the findings presented in Chapter 5, addressing each research question before summarising cross-cutting patterns, acknowledging limitations, considering practical implications, and identifying directions for future research.

6.1 RQ1: Task Focus Shifts

How does the introduction of AI coding assistants shift software engineers' perceived focus across various development tasks?

Our results indicate that AI coding assistants are reshaping where software engineers direct their effort across development activities. Approximately two-thirds perceived spending less time overall on the six tasks at both time points. Writing code showed the largest and most consistent reduction, with 82% reporting reduced time on this activity by Q2. Refactoring, testing, designing, and debugging also showed means below neutral, though designing and debugging were less affected. Testing and reviewing both trended upward over the study period, and reviewing was the only task with means above neutral at both time points. These aggregate perceptions remained stable across our two measurement points, though individual participants showed considerable movement in both directions. Importantly, the relative balance between creation-focused tasks (designing, writing, refactoring) and verification-focused tasks (reviewing, testing, debugging) shifted significantly toward verification over the six-month study period ($p_{adj} = 0.006$), even though neither the creation nor the verification group changed significantly on its own. This is consistent with prior research showing AI tools accelerate code generation [74, 25] while increasing verification burden [67].

Our qualitative findings may help explain where these perceived reductions in time came from. Theme 1, *Compressing routine creation work*, captured how AI handles boilerplate, scaffolding, and small fixes that previously consumed substantial effort. This is further supported by Theme 2, *Rerouting information seeking and comprehension into AI interactions*, which revealed how engineers now turn to AI before Google or Stack Overflow, folding what was once a separate information-gathering step into the coding interaction itself, as well as accelerating onboarding onto unfamiliar codebases, languages, and legacy code. Together, these patterns suggest that AI may compress not just the act of writing code but the preparatory work that surrounds it.

Turning to what predicted these shifts, psychological factors significantly improved model fit. Productivity perceptions were the strongest predictor for both testing (OR = 0.44) and writing code (OR = 0.57), while lower self-efficacy predicted increased reviewing time (OR = 0.62). Experience level and company size did not significantly predict task focus shifts. Gender, however, remained significant for reviewing code even after controlling for usage frequency and psychological factors: men were more likely to report increased time reviewing code, suggesting they may be adopting the *creation-to-verification shift* more strongly. Men also reported reduced time writing code, though this effect weakened once psychological factors were controlled for. Recent research has found that women face greater competence penalties for AI use than men [37], which may lead to more selective adoption patterns. Given the small number of women in our sample, this finding warrants replication. With that exception, how engineers engaged with AI tools was a stronger predictor of task focus shifts than who they were.

Some findings were unexpected. Given that verification consumes substantial time in AI-assisted development [67], we expected perceived time spent on testing and code review to sit above the neutral midpoint, reflecting increased verification effort. Instead, reviewing code was the only task above neutral at either time point, and only modestly so. Testing remained below neutral throughout, meaning most engineers perceived spending less time on it even in an AI-assisted context, though it did trend upward between time points. Those who reported spending more time reviewing also reported smaller self-efficacy gains from AI use, perhaps reflecting greater caution in accepting AI-generated code.

One explanation is that engineers may not perceive the verification work they do with AI-generated code as traditional "testing" or "code review". When evaluating whether to accept a suggestion, scanning output for errors, or checking that

generated code integrates correctly, engineers are indeed performing verification, but this may feel qualitatively different from running a test suite or conducting a formal code review. If so, our questions about traditional SDLC tasks may have missed where the effort is actually going.

Additionally, despite expectations that engineers might become "architects" when freed from implementation details, we found no quantitative evidence that perceived design time increases. Only 9% of matched participants showed reduced writing alongside increased design over the study period. Although some participants occasionally framed their experience as involving more "design" or "higher-level thinking" in their open-ended responses, their descriptions more consistently reflected supervision-oriented activities rather than architectural or conceptual design.

These patterns raise an obvious question: if most engineers perceive spending less time overall across these core development activities, and in particular, substantially less time on writing code, and this effort is not flowing into design work and only modestly into verification tasks, then where is it going? Our qualitative findings suggest an answer. Theme 3, *From doing to supervising AI-generated code*, captured participants describing this move, with one noting they now spend time *"Mostly reading the code and directing AI on the right way"* (P54, Q2). This suggests that time is not simply freed up but absorbed by a new category of work that traditional SDLC taxonomies do not capture.

We propose that AI coding assistants may be creating what we term *supervisory engineering work*: a distinct category encompassing the effort required to direct, evaluate, and correct AI output. The proposed components are:

- **Directing**: specifying intent, crafting prompts, providing context, and iterating when output misses the mark
- **Evaluating**: reading AI output and deciding what to accept, modify, or reject
- **Correcting**: fixing errors, integrating output into existing code, and maintaining consistency

Not all engineers experience this supervisory work equally. Theme 4 from our thematic analysis, *Verification and trust calibration moderating perceived benefits*, captured how engineers vary in their willingness to accept AI output, with some restricting AI to low-risk contexts while others accept suggestions more readily.

This calibration directly mediates how much supervision is required. It may also explain our earlier finding that engineers reporting smaller self-efficacy gains from AI also reported spending more time reviewing: lower trust in AI's contribution to one's capability could lead to more careful verification.

It is important to note that this is a proposed interpretive construct, emerging from convergent patterns across our quantitative and qualitative data rather than from direct measurement. We offer it as a conceptual framework for future investigation rather than a confirmed empirical finding, but one that may help explain an important gap: the modest upward trends in reviewing and testing do not fully account for the time freed by reduced creation work, suggesting that engineers are also engaging in a qualitatively different form of work that sits outside conventional task categories.

More broadly, these findings suggest that AI coding assistants are reshaping the nature of software development work itself. If engineers spend less time writing code and more time directing and evaluating AI output, this has implications for which skills remain essential, what engineers practice day-to-day, and how the profession develops its next generation. The emergence of supervisory work, if valid, connects to our other research questions: it may explain the cognitive load increases we observe in RQ2, the prompt engineering importance reported in RQ4, and the complex relationship between AI use and skill development in RQ3.

6.2 RQ2: Developer Experience and Productivity

How does the use of AI coding assistants impact software engineers' perceptions of their developer experience and productivity?

Our results found that software engineers predominantly perceived AI coding assistants as improving their productivity, with 84% reporting improvement at both time points, consistent with industry surveys [90]. These perceptions were highly stable: 77% of matched participants gave identical ratings at both time points, and no one who reported improved productivity at Q1 reported worsened productivity at Q2. Perceived impact on developer experience was also generally positive at Q1, though less uniformly so: 66% reported improved cognitive load, 62% improved feedback loops, and 53% improved flow state. Experience level, gender, and company size showed no significant effects on either construct. Self-efficacy did, with a large effect on productivity perceptions and significant effects on all three developer experience dimensions. The strength of the self-efficacy-productivity association raises questions about construct distinctness, which we

address in Section 6.5.2. Usage frequency was also associated with improved cognitive load and productivity perceptions. Demographic factors showed weaker associations than how engineers engaged with AI tools and how confident they felt in their ability to use them effectively.

Unlike productivity, developer experience showed concerning signs of erosion over the six-month study period. Developer experience here refers to the three DevEx dimensions we measured: cognitive load, feedback loops, and flow state. By erosion we mean falling into what we termed the *Negative* cohort: reporting worsened perceptions (below the neutral midpoint) on at least one of these dimensions. This cohort nearly doubled, from 14% to 27%, while only 9% maintained complete stability across all three dimensions. Our qualitative Theme 1, *Accelerated throughput with uneven day-to-day experience*, helps explain this pattern: participants described impressive time compression alongside persistent friction from repeated prompting and correction. As one noted, *"This does break the flow, but still speeds up the overall process"* (P28, Q2).

In fact, our data showed flow state to be the dimension most vulnerable to erosion: within the Negative cohort, it affected 54% at Q1, rising to 76% at Q2. Cognitive load also showed signs of erosion, with 29% of matched participants rating it lower at Q2 than Q1. Meanwhile, feedback loops showed the opposite trajectory, improving significantly ($p_{adj} = 0.038$). Mobile developers bucked the overall trend, reporting worsening feedback loops but improving cognitive load, while also experiencing the sharpest flow state decline. This may reflect differences in how AI coding assistants integrate with mobile development environments, where IDE support is often less mature than for web or backend development. With only six mobile developers in our sample, this finding warrants caution, but it is unusual enough to merit further study. These patterns were not merely aggregate trends: those who started with positive developer experience showed poor retention, with only 37% maintaining their positive status, and the remainder drifting toward mixed or negative. More concerning, those who started with negative experience never recovered to fully positive; the best outcome was mixed or neutral. What we observed is perceptions of the three developer experience dimensions diverging; although all three dimensions were significantly correlated at Q1, none remained so by Q2. Over time, engineers may be learning to recognise this trade-off, appreciating AI tools for immediate feedback even as they notice the disruption to deeper concentration.

Why might flow state be particularly susceptible? Our literature review identified flow as a state of complete absorption requiring sustained focus free from

disruptive interruptions [24]. AI coding assistants may structurally undermine these conditions. Each suggestion requires evaluation; each generation requires verification. The cycle of prompting, reviewing, accepting or rejecting, and iterating constitutes ongoing interruption built into the workflow itself. Our qualitative Theme 3, *Verification and trust management as ongoing engineering work*, captured this dynamic: participants described needing to maintain vigilance when reviewing AI output, a form of mental effort that persists even when the code itself is generated quickly. Prior research found that context switching and interruption frequency are key factors affecting flow [81]. The delegation model frees developers to turn attention elsewhere while code is being generated, but this shifts work from sustained focus to a rhythm of directing, waiting, and reviewing—precisely the supervisory engineering work we proposed in RQ1.

That productivity perceptions remained stable while developer experience eroded raises questions about the DevEx framework [71], which frames these dimensions as key drivers of productivity. At Q1, our data aligned with this framing: all three dimensions significantly predicted productivity, with flow state the strongest. By Q2, however, feedback loops had become the strongest predictor, cognitive load remained significant, and flow state was no longer significant. More importantly, *changes* in flow state showed essentially zero correlation with *changes* in productivity ($\rho = 0.02$). Our qualitative findings help explain why productivity held steady despite this decoupling. Theme 1, *Accelerated throughput with uneven day-to-day experience*, captured participants describing faster output alongside persistent friction, with perceived productivity and lived experience not always moving together. Theme 2, *Expanded perceived capability and willingness to engage with unfamiliar work*, offers a mechanism: the expanded capability AI provides may compensate for the friction of achieving it, with engineers describing contributions in areas that would previously have required substantial ramp-up. We term this pattern the *productivity-experience paradox*: sustained output perceptions alongside degrading experience, potentially decoupling the established relationship between the two.

This paradox describes coexisting perceptions, not a causal trade-off. Participants reported feeling more productive while also experiencing more friction in flow and cognitive load. These are parallel self-reports; our design cannot establish whether one causes the other. The finding matters for what it reveals about engineers' lived experience of AI-assisted work, not for any causal mechanism.

The productivity-experience paradox suggests that the established relationships between productivity and developer experience may need re-examination

when AI is introduced. Prior research found that uninterrupted focus is central to developers' sense of productivity [62] and that satisfaction and productivity are bidirectionally linked [91]. The DevEx framework treats its three dimensions as complementary drivers of productivity, but AI assistance may reshape their relative contributions, with faster feedback compensating for increased cognitive friction. Whether this reconfiguration is sustainable, or whether the erosion in flow and cognitive load eventually undermines the gains, remains an open question.

6.3 RQ3: Skill Development and Self-Efficacy

How do software engineers perceive the impact of AI coding assistants on their skill development and self-efficacy when building software?

Our results found that both perceived skill development impact and self-efficacy were stable at the aggregate level, though this stability masked substantial individual variation. Self-efficacy remained moderately above neutral at both time points ($M = 3.72$), and most participants (70%) showed no reliable change, with equal proportions improving and declining (15% each). Perceived skill development impact was similarly stable in aggregate, with roughly half perceiving impact at both time points. Yet beneath this stability, over half shifted their perceived skill development impact over six months. These shifts were asymmetric: those who initially perceived no impact were more likely to move toward perceiving it over time, suggesting that extended use tends to surface perceived effects rather than diminish them. Subgroup patterns in these shifts did not suggest systematic demographic differences, though usage frequency did emerge as a significant predictor by Q2, reinforcing that perceiving an impact on skill development may require sustained engagement.

Our qualitative findings suggest that what skill development means in an AI-assisted context may be changing. Theme 2, *Learning shifts from recall to evaluation*, captured how participants described reduced reliance on documentation and syntax recall, instead learning by evaluating AI suggestions: *"I may learn new techniques when reviewing AI generated code"* (P41, Q2). This evaluative mode requires what Theme 4, *Trust calibration as ongoing engineering work*, captures: ongoing judgement about when to accept and when to reject AI output.

An important parallel with RQ2 emerged. Perceived skill development impact and self-efficacy showed significant positive correlations at both time points, yet *changes* in the two constructs showed no significant correlation. In other words,

changes in one did not predict changes in the other. This mirrors the productivity-experience decoupling we observed in RQ2 and suggests that constructs which appear related cross-sectionally may evolve through different mechanisms.

One finding stood out as particularly interesting: beneath the stable self-efficacy composite, the eight items did not move uniformly. "Overcome challenges" showed the largest increase, perhaps because having an AI pair programmer constantly available provides reassurance when facing obstacles. "Comparative ability", the only item explicitly referencing comparison to others, showed the largest decline. If everyone has access to the same tools, perceived differentiation between engineers may decrease even as absolute capability increases. This levelling dynamic may have implications for how engineers perceive their standing relative to peers.

Another intriguing finding was that GitHub Copilot was the only tool significantly associated with self-efficacy, and the association was negative at both time points, meaning Copilot users reported lower self-efficacy than non-users. Across all four research questions, this was the only tool association that persisted at both time points, making it the most robust individual tool finding in the study. Why Copilot specifically showed this pattern is unclear. Although it originated as an inline completion tool, by the time of our study it also offered chat functionality similar to other tools, making modality-based explanations difficult to sustain. The direction of effect is also ambiguous: Copilot use may influence self-efficacy, or engineers with lower self-efficacy may gravitate toward Copilot's more integrated, always-available assistance. Given Copilot's high adoption rate in our sample (70% at Q1), this association warrants further investigation.

Junior engineers reported higher self-efficacy and greater perceived skill development impact, though the patterns differed. For self-efficacy, this effect persisted through full demographic and psychological controls. For perceived skill development impact, the effect was partially explained by self-efficacy: juniors appear to perceive more skill development impact partly because they report higher self-efficacy when using AI. Prior research suggests less experienced developers tend to view AI as a teacher while experienced developers see it as a junior colleague [102], which may explain these different orientations toward perceived impact. Yet other research has cautioned that AI may function as a liability for less experienced developers who lack the knowledge to filter poor suggestions [66]. We cannot distinguish with perception data alone whether this higher self-efficacy reflects genuine adaptation, an illusion of competence where confidence outpaces

underlying skill, or simply different reference points where juniors compare themselves to an AI-augmented baseline while seniors compare to decades of unassisted experience. Prior research with novice programmers found that those who struggled with programming tasks often finished AI-assisted work believing they had performed better than they actually had [78]. Whether similar dynamics apply to professional developers remains an open question.

Productivity and self-efficacy showed a bidirectional relationship, with engineers who perceived greater productivity gains also reporting feeling more capable. Self-efficacy was also strongly associated with anticipated disappointment if tools were removed, which we captured at Q2 ($\rho = 0.48$). This aligns with recent research finding that 80% of regular AI users report they would be sad if they could no longer use them [42]. That this association centred on anticipated loss hints at a growing entanglement between self-efficacy and tool availability.

Together, these patterns suggest what we term *assisted capability*: confidence that is partly constituted by having AI tools available. Our qualitative Theme 1, *Expanded practical reach into unfamiliar domains*, captured this dynamic: participants described working across unfamiliar technologies and domains with greater ease. As one noted, *"Syntax becomes just a dialect, with no prior experience of golang i was able to contribute new functionalities to an existing code base"* (P55, Q1). Theme 3, *Confidence through reduced friction, not mastery*, reinforced this: self-efficacy gains emerged from faster feedback and easier experimentation rather than from deeper expertise. If confidence depends partly on tool availability rather than mastery, questions arise about what this means for how engineers develop and maintain their skills.

Indeed, participants expressed tension about what this means for learning. Theme 5, *The speed-depth trade-off*, captured engineers actively navigating between efficiency and deeper understanding: *"Because I am just copying and pasting, it is faster to get results, but I don't pick up the language as well as when I used to code from scratch"* (P6, Q1). Engineers appear to navigate this trade-off individually, which may explain the substantial between-person variance we observed.

Overall, our findings suggest that engineers feel more capable when using AI coding assistants, while remaining divided on skill development impact. The comparative ability decline hints that AI may be changing how engineers perceive their standing relative to peers, even as it enhances absolute capability. The speed-depth trade-off our participants described connects to the supervisory work

we proposed in RQ1: if engineers spend more time evaluating AI output and less time writing code from scratch, the nature of what they practice daily is shifting. Whether this shift constitutes skill erosion, skill transformation, or simply different skill development may depend on the individual: our participants were themselves divided, and the substantial variance we observed suggests there may be no single answer. The emergence of assisted capability, if valid, has implications for educators preparing the next generation, for practitioners investing in their professional development, and for organisations supporting engineers through this transition.

6.4 RQ4: Prompt Engineering

How do software engineers perceive the role of prompt engineering as a new core competency in the AI-assisted era of software development?

Our results suggest that engineers increasingly view prompt engineering as important for their professional future, with over half rating it highly important at both time points. Views on whether it will replace traditional coding skills were more divided, hovering near neutral initially before shifting slightly toward agreement over the six-month study period. Neither dimension showed significant aggregate change, though as we observed across our research questions, aggregate stability masked substantial individual movement, with 63% of matched participants changing their position on at least one dimension.

Our qualitative findings suggest that engineers value prompt engineering for what it enables rather than as a skill in itself. Theme 2, *Prompting as a gateway to productivity and expanded reach*, captured how participants described prompting as valuable for the efficiency it unlocked rather than as a distinct competency worth developing for its own sake. This pragmatic framing may help explain the patterns we observed in our quantitative data.

Perhaps unsurprisingly, we found that the two dimensions we measured were not independent. The correlation between importance and replacement beliefs was strong and stable across both time points, and in our mixed-effects models each dimension strongly predicted the other. This suggests that engineers are not making separate assessments but forming coherent belief structures: those who see prompt engineering as important also tend to believe it will replace traditional skills, and vice versa. This aligns with early industry findings that roughly a third of developers believe software engineering may evolve into a form of prompt engineering [46].

To explore what these belief structures look like in practice, we defined five perception profiles in Section 4.4.1.5. Two were of particular theoretical interest: the *Additive* profile (high importance, low replacement) and the *Substitutive* profile (high importance, high replacement). Both treat prompt engineering as important, but they differ fundamentally in what this means for the future of traditional coding.

One pattern stood out as particularly interesting: the apparent instability of the Additive profile. Of the twelve matched participants who held this profile at Q1, none retained it by Q2, with 42% moving toward the Substitutive profile and the remainder dispersing across other combinations. By contrast, the Substitutive profile showed 63% retention, making it the most stable combination we observed. This finding warrants caution given the small subgroup sizes, and alternative explanations including selective attrition and regression to the mean cannot be ruled out. Nonetheless, the pattern is suggestive: the Additive profile may be psychologically difficult to sustain.

Why might the Additive profile prove difficult to maintain? One possibility is cognitive tension: if prompt engineering is genuinely important for the future, it becomes harder to simultaneously believe that traditional coding skills will remain equally important. Another is experiential: as engineers gain more experience with AI tools, they may observe prompt engineering absorbing more of what they used to do manually, making the replacement hypothesis feel increasingly plausible. Our qualitative Theme 3, *Contesting whether prompting substitutes for, or depends upon, core software engineering skills*, captured this tension directly. Some participants insisted that traditional skills remain essential for evaluation and accountability, while others reframed the work entirely as project planning rather than a distinct technical skill.

Although the two dimensions were correlated, only importance showed a significant psychological predictor. Productivity perceptions predicted importance beliefs but not replacement, and this relationship was bidirectional: engineers who reported greater productivity gains rated prompt engineering as more important, and vice versa, suggesting a reinforcing cycle in which perceived productivity benefits drive growing importance beliefs. Self-efficacy, however, did not significantly predict either prompt engineering dimension, unlike the experiential outcomes in RQ2 and RQ3. Theme 2's pragmatic framing may help explain this difference. Engineers' confidence in building software with AI assistance shapes how they experience what prompting unlocks, but not their beliefs about whether prompting itself will become a core competency.

Interestingly, programming language and company size also predicted prompt engineering views. Web developers using JavaScript or TypeScript rated prompt engineering importance substantially higher than users of compiled or interpreted languages, and this effect strengthened over time. This may reflect that AI coding assistants are particularly well-suited to web development, which often involves repetitive scaffolding and boilerplate where effective prompting proves especially valuable. By Q2, developers at small companies were nearly five times more likely to agree that prompt engineering will replace traditional skills, perhaps because smaller organisations view AI tools as force multipliers that fundamentally change what individual engineers need to do. Experience level, gender, and specific AI tools, however, did not predict views on either dimension. Unlike RQ1 through RQ3, which concern what engineers observe and experience in their current work, prompt engineering beliefs are predictive—concerning how a competency fits into one's professional future. This may explain why contextual factors matter more here than we observed elsewhere.

Beyond these patterns, our qualitative analysis revealed substantial conceptual diversity in how engineers frame prompt engineering itself. Theme 1, *Recasting prompting as intent specification and communication*, captured participants who framed it as an extension of existing engineering practices rather than a novel skill, treating it as communication and problem decomposition using abilities that have always been central to software engineering. Yet Theme 4, *Scepticism and boundary-setting around "prompt engineering" as a skill*, revealed a subset who questioned whether prompt engineering constitutes a genuine competency at all. Some dismissed it as a fad or expected it to become obsolete as models improve. One participant rejected the framing entirely: *"No. I am not learning how to talk to a robot. A robot needs to learn how to understand me"* (P18, Q2). This scepticism positions prompt engineering as a temporary workaround for current AI limitations rather than a durable professional competency.

Overall, our findings suggest that software engineers increasingly view prompt engineering as important for their professional future, though whether it constitutes a genuinely new competency or an extension of existing skills remains contested. In a field moving this quickly, the positions engineers take are actively evolving, and what constitutes prompt engineering may itself be a moving target. Recent research has shown that effective AI collaboration already extends well beyond crafting individual prompts, encompassing context provision, problem framing, and iterative refinement [58], and industry discourse has begun shifting

from "prompt engineering" toward "context engineering" to reflect this broader view. Indeed, much of the supervisory work we described in RQ1, particularly directing and specifying intent, could be understood as this new competency in practice. The Additive/Substitutive framework offers a lens for understanding how engineers form coherent beliefs about this emerging competency, and future research with larger samples could explore whether the apparent instability of certain profiles holds. Whatever name it ultimately takes, the emergence of this new dimension of professional practice has implications for educators, practitioners, and organisations navigating this transition.

6.5 Cross-Cutting Patterns

Having addressed each research question, we now turn to patterns that emerged across our findings.

6.5.1 The Shifting Tool Landscape

Our study captured a six-month window in a field evolving at unprecedented speed. The tool landscape shifted substantially even during data collection, with 82% of matched participants changing their tool combinations between time points. This means that temporal effects in our data are confounded with a rapidly changing tool landscape; our findings reflect practice-in-the-wild rather than the maturation of any single tool. Changes we observed may stem from tool switching, tool improvement, or accumulated experience, which we cannot fully disentangle.

Two trends emerged. First, *tool diversification*: participants mentioned around 37 distinct tools across the study, and engineers are not settling on a single tool but building toolkits. The average number of tools per engineer increased from 1.9 at Q1 to 2.9 at Q2. Second, *shift toward coding-specialised tools*: across our full samples, ChatGPT declined from 70% to 58% usage, while Cursor grew from 16% to 29%. This suggests some engineers are moving from general-purpose conversational AI toward purpose-built coding environments.

The only concern that changed significantly over the six-month study period was maintainability, which increased. As engineers gained experience with AI-generated code, concerns about long-term maintainability grew. This may reflect accumulated experience recognising that AI optimises for "works now" rather than "maintains later".

6.5.2 Mindset Over Circumstance

Perhaps the most striking finding, however, was not what self-efficacy revealed about RQ3 itself but what it revealed across our other research questions. Self-efficacy emerged not simply as one predictor among many but as the dominant predictor of experiential outcomes: perceived productivity, all three developer experience dimensions, and skill development perceptions. Effect sizes ranged from moderate to very large, as Table 6.1 summarises.

Our qualitative findings help explain why. Participants recognised that their confidence arose from tool-assisted efficiency rather than deeper expertise (RQ3, Theme 3, *Confidence through reduced friction, not mastery*). This is precisely why self-efficacy shapes experiential outcomes so strongly: it reflects how engineers feel about their AI-augmented capabilities.

But self-efficacy did not predict everything. Task focus shifts in RQ1 reflected the structural nature of AI-assisted work, changes driven by how tools are integrated into workflows rather than by individual confidence (Theme 3, *From doing to supervising AI-generated code*). Prompt engineering beliefs in RQ4 were similarly independent: participants framed prompting as pragmatic enablement rather than a core skill in itself (Theme 2, *Prompting as a gateway to productivity and expanded reach*). Confidence shapes how engineers feel about their current work, but does not determine what they observe in their workflows or what they believe about the profession's future.

Meanwhile, across experiential outcomes, demographic factors we expected to matter (experience level, gender, company size) were largely non-significant. The only demographic finding that persisted through full controls was the junior self-efficacy pattern noted above, and even this may reflect psychological rather than purely demographic mechanisms.

The notably large effect size for productivity does raise the question of whether self-efficacy and productivity perceptions are truly distinct constructs. As reported in Section 5.3.2, correlations between the two measures were moderate, with over 60% of the variance in each not explained by the other. Drawing on the principle that distinct constructs should exhibit different patterns of relationships with other variables [6], we observed differential predictive patterns: self-efficacy predicted skill development while productivity did not; conversely, productivity predicted prompt engineering importance while self-efficacy did not. This differentiation suggests the constructs are distinct, though the substantial

Table 6.1: Self-efficacy as predictor across outcomes

Outcome	OR	95% CI
Productivity	**10.68*****	[4.39, 25.95]
Feedback loops	**3.27*****	[2.03, 5.27]
Skill development	**2.06****	[1.28, 3.33]
Cognitive load	**1.73****	[1.17, 2.57]
Flow state	**1.69****	[1.17, 2.44]

OR = Odds Ratio from Model 3 (full) CLMMs; CI = 95% Confidence Interval.
All models control for gender, experience, and usage frequency. RQ2 outcomes additionally control for prompt engineering importance; skill development controls for productivity. Continuous predictors z-scored. Significance: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

shared variance and concurrent self-report measurement suggest effect sizes of this magnitude should be interpreted with caution.

We term this pattern *mindset over circumstance*. Self-efficacy is not a fixed trait but a mediating lens, itself shaped by experience, feedback, organisational context, and tool affordances. How engineers perceive their own capabilities appears to shape their experience of AI tools far more than demographic category or organisational setting. This finding carries an encouraging implication: the strongest predictors of experiential outcomes were factors individuals can influence, such as how they engage with tools and how they perceive their own abilities, rather than fixed characteristics beyond their control. At the same time, it raises questions about what self-efficacy means when capability is partly constituted by tool access, and whether confidence that comes through reduced friction represents genuine professional growth.

6.6 Threats to Validity

Several threats to validity warrant attention when interpreting our findings.

Internal Validity

Correlational Design. Our correlational design means that although self-efficacy predicts productivity, developer experience, and skill development perceptions across our research questions, we cannot claim it *causes* these outcomes. Most predictive relationships examined concurrent associations. Cross-lagged analysis provides a stronger test: in our matched sample ($n = 94$), Q1 self-efficacy predicted Q2 productivity controlling for Q1 productivity ($\beta = 0.23$,

$p = 0.025$), and vice versa ($\beta = 0.27$, $p = 0.007$). This bidirectional pattern is consistent with reciprocal causation but cannot rule out common causes such as general optimism. When we report that psychological factors "matter more than demographics", we mean they are stronger statistical predictors, not that interventions targeting them would produce the predicted effects.

Attrition. 40% of participants who completed Q1 did not complete Q2, reducing our matched longitudinal sample from 158 to 95. As reported in Chapter 4, demographic comparisons showed no significant differences between retained and lost participants, supporting the representativeness of the matched cohort. However, we cannot rule out unmeasured differences in attitudes or experiences. Engineers who became disillusioned with AI tools may have been less motivated to complete a follow-up questionnaire about them, which would cause our longitudinal findings to overestimate stability and underestimate negative trajectories.

External Validity

Sample Limitations. Our study only included engineers currently using AI coding assistants; those who tried these tools and abandoned them are absent from our sample. This selection bias likely causes our findings to overestimate positive outcomes, as engineers who found AI unhelpful would have stopped using it and thus would not appear in our data. The 84% positive productivity rate should be interpreted as "84% of continuing users", not "84% of all who tried". Beyond this survivor bias, 85% of our sample were men and predominantly English-speaking, which limits generalisability to under-represented groups.

Temporal Boundary. Our findings reflect experiences with AI coding assistants as they existed in late 2024 and early 2025. Rapid tool evolution means patterns may not hold with newer agentic tools that operate more autonomously.

Construct Validity

Perception-Based Measures. All of our measures capture perceptions rather than objective performance. As discussed in Chapter 3, perceived and actual productivity can diverge substantially, and several factors may inflate positive perceptions in our data: social desirability, effort justification, and question framing. However, our interpretivist stance treats perceptions as meaningful data in their

own right: they drive adoption decisions, shape professional identity, and influence how engineers approach their work.

Recall Bias. Engineers may overweight memorable events when summarising months of AI use, while recency effects may inflate the influence of experiences closest to each questionnaire. Internal benchmarks may also shift: an engineer who found AI frustrating six months ago may have recalibrated what counts as frustrating, making direct comparisons across time points imprecise. From our interpretivist perspective, however, these distortions are not merely noise to be eliminated but part of what we sought to understand.

Instrument Design. The questionnaires were designed by a practising software engineer, whose familiarity with the domain may have influenced the framing of questions. We mitigated this by seeking feedback on the questionnaire design from an external researcher, and through iterative discussions with the research supervisor.

Statistical Conclusion Validity

Limited Statistical Power. Some demographic cells contained few participants, limiting statistical power for subgroup analyses. Despite applying multiple comparison corrections, testing many relationships increases false positive risk; our findings should be interpreted as patterns requiring replication rather than definitive conclusions.

6.7 Implications

For Individual Engineers. Our findings suggest that attitude and engagement shape outcomes more than background or circumstance. This is encouraging, since these are factors individuals can more readily influence. Practically, engineers might consider experimenting with multiple tools rather than settling on one, since those reporting better outcomes tended to use more tools and engage more frequently. The emergence of supervisory engineering work suggests preparing for a shift in daily practice, with less time writing code and more time directing, evaluating, and correcting AI output. This is not freed-up time but differently allocated effort. The speed-depth trade-off our participants described suggests consciously choosing when efficiency matters versus when deeper learning is the

priority. Perhaps most actionable is developing trust calibration. Knowing when to accept and when to reject AI output emerged across our qualitative findings as a skill engineers are actively building, and one that transfers across the rapidly evolving tool landscape.

For Organisations. Organisations should recognise that AI coding assistants may not deliver pure time savings. Our findings suggest that effort is being reallocated from creation toward verification, with a new form of work, supervisory engineering work, potentially absorbing some of the time that might otherwise appear as productivity gains. This shift has implications for how organisations structure teams, evaluate performance, and prioritise skills when hiring, particularly as verification, judgement, and trust calibration become more central to the role. Output metrics alone may mask experiential erosion, so monitoring developer experience alongside productivity could provide earlier warning of unsustainable patterns. On tooling, enabling experimentation rather than mandating a single tool may better serve adaptation, given how substantially the landscape shifted even during our six-month study.

For Educators. If AI is shifting engineering work from creation toward verification while introducing new responsibilities like supervisory engineering work, educational programmes may need to prepare students accordingly. This means teaching not just traditional technical skills but also judgement, trust calibration, and the components of supervisory work: directing, evaluating, and correcting AI output. On prompting, educators may need to help students develop this capability given its perceived importance, while navigating uncertainty about whether it complements or replaces traditional skills. Teaching it as an extension of existing practices like requirements articulation and problem decomposition, as our participants framed it, may be more effective than treating it as a separate discipline. More broadly, transferable principles of human-AI collaboration may prove more durable than tool-specific training given how rapidly the landscape is evolving. Assessment methods may also need to evolve, evaluating understanding rather than output quality alone, since students can now produce working code without necessarily understanding it.

For Researchers. Our study highlights several methodological considerations. Aggregate stability often masked substantial individual movement, suggesting

value in tracking individual trajectories rather than relying on group-level summaries alone. Tool-agnostic measures capturing human-AI interaction patterns may prove more durable than tool-specific evaluations, and explicit replication efforts would help establish which findings hold as tools evolve. Balancing frequent measurement to capture rapid adaptation with longer-term follow-up to assess sustained impacts remains an open methodological challenge.

6.8 Future Research Directions

Emerging Competencies and Work Categories. AI assistance may be introducing new work categories that existing frameworks do not capture. The supervisory engineering work we proposed raises questions about whether this represents a genuinely new SDLC activity distinct from traditional categories like designing, coding, testing, and reviewing. Prompting likely falls within the directing component, though our participants held divergent views on whether it extends traditional skills or substitutes for them. Future research could examine whether supervisory engineering work is empirically distinct from existing activities, what skills it requires, and how engineers should develop them. If this work category proves real and distinct, established SDLC models may need updating to account for where developer effort now goes.

Developer Experience and Productivity. Prior research has shown that developer experience drives productivity, yet our data show productivity holding steady while two of the three DevEx dimensions declined. This productivity-experience paradox raises a fundamental question: does the established relationship operate differently when AI tools are part of the development process? Future research could examine whether the DevEx dimensions contribute differently in AI-assisted contexts, whether new dimensions are needed, and whether the causal mechanisms have changed. There are also practical questions: if engineers remain productive while flow state and cognitive load erode, does this pattern eventually manifest as burnout or turnover? Longer-term studies tracking both productivity and experience could illuminate whether the divergence we observed is sustainable over longer time horizons.

Satisfaction in Supervisory Work. The shift toward supervisory engineering work raises questions about developer satisfaction that our study could not

fully address. The flow state decline and increased cognitive load we observed suggest some experiential costs may accompany this transition. Future research could examine whether engineers find supervisory work intrinsically satisfying or whether reduced hands-on coding diminishes professional fulfilment. Do these feelings vary systematically across developers, perhaps by experience level or creative orientation? And if supervisory work proves less satisfying for some, what interventions might preserve the joy of craft while capturing the benefits of AI assistance? These questions become increasingly urgent as AI tools mature and supervisory patterns potentially deepen.

Junior Developer Trajectories. Junior developers reported higher self-efficacy than seniors when using AI assistants, a finding that persisted through all controls. Whether this reflects genuine accelerated learning, appropriate confidence calibrated to AI-assisted contexts, or premature confidence that masks skill gaps remains unclear. Longitudinal studies following early-career engineers through their first years of AI-assisted practice could distinguish these possibilities and inform how organisations support junior developers.

Team and Organisational Dynamics. Our study focused on individual engineers, but the creation-to-verification shift and supervisory engineering work we described likely have team-level implications. How do these patterns aggregate when entire teams adopt AI assistance? Do verification responsibilities distribute across team members or concentrate in certain roles? Research examining team dynamics and organisational structures could illuminate how individual-level findings scale.

Chapter 7

Conclusion

This study investigated the impact of AI on software engineering by examining how AI coding assistants affect software engineers' perceptions across the study's four research areas. Through a longitudinal mixed-methods study of professional software engineers who use AI coding assistants, with questionnaires administered at two time points six months apart, we gathered both quantitative and qualitative insights, examining not only cross-sectional patterns but also how perceptions evolved as engineers gained experience with these tools. This longitudinal lens reveals dynamics that cross-sectional snapshots would miss.

Our findings suggest that development work is being reorganised rather than simply accelerated. Most engineers perceived spending less time across traditional development tasks, with the relative balance shifting from creation toward verification; we have termed this the *creation-to-verification shift*. Yet the increase in verification was modest, raising the question of where the effort is going. We propose this effort flows into what we have called *supervisory engineering work*: directing AI, evaluating its output, and correcting its errors. Prompt engineering may be central to this supervisory work, and by the second time point, the balance of opinion had tilted slightly toward seeing it as a replacement for traditional coding skills, though engineers remained divided on whether it represents a fundamentally different way of working or an evolution of existing practices. Meanwhile, the *productivity-experience paradox* we documented suggests these tools may be decoupling dimensions that previously moved together, with productivity perceptions remaining consistently positive even as developer experience showed signs of erosion over time, particularly in flow state and cognitive load. Together, these patterns suggest that AI coding assistants are reshaping the nature of software engineering work, not merely making existing tasks faster.

The overarching finding, however, is how central self-efficacy is. It was the strongest predictor of how engineers rated the impact of AI on their productivity, their developer experience, and their skill development, outweighing demographic and organisational factors. More broadly, malleable factors such as how engineers engage with tools and perceive their abilities mattered more than fixed demographics. This is encouraging, though it raises questions about what self-efficacy means when capability is partly tied to tool access. In practice, engineers appear to be navigating this by developing trust calibration: knowing when to accept and when to reject AI output, a transferable skill that appeared across multiple qualitative themes.

The field of software engineering is in transition. Where much prior research has measured what AI tools achieve, this study examined how engineers experience them over time. For the engineers in our study, AI is not "just another tool"; it is transforming what software engineering involves. The convergence toward substitutive views of prompt engineering, the emergence of supervisory work patterns, and the decoupling of productivity from developer experience all suggest that we may be witnessing a fundamental shift in the nature of software engineering practice, one that our existing frameworks and taxonomies do not yet fully capture.

Whether AI coding assistants will finally close the persistent gap between ambition and achievement that has defined software engineering since its origins remains to be seen. Brooks warned that technology alone has historically failed to deliver dramatic improvements [13], and our findings suggest that unintended consequences are indeed emerging. As software engineering serves as a proving ground for AI-assisted work, these patterns may foreshadow dynamics that other professions will encounter as AI tools spread. For a global community of professionals who have built careers around the craft of building software, understanding how engineers perceive these tools is essential, because perceptions drive behaviour. We hope this study contributes to that understanding, informing educators preparing the next generation, organisations supporting their teams, and engineers navigating their own paths through this period of rapid change.

Appendix A

Study Instrument

A.1 Questionnaire Items

Table A.1: Questionnaire items by construct and research question

Item ID	Item	Q1/Q2	Scale
Screening and Context			
Q1.1	Are you currently working professionally in software engineering?	Q1	Yes/No
Q2.1	Are you currently using an AI coding assistant?	Both	Single choice[a]
Q2.5	In which industry is the company you work for primarily operating?	Both	Single choice
Q3.1	Which AI coding assistants are you currently using the most?	Both	Multi-select[b]
Q3.2	How often are you using AI coding assistants while building software?	Both	Likert (frequency)

Continued on next page

Table A.1 continued from previous page

Item ID	Item	Q1/Q2	Scale
Q3.3*	What was your initial impression of using an AI coding assistant?	Q1	Likert (impression)
Q3.3*	If you could no longer use an AI coding assistant, how would you feel?	Q2	Likert (disappointment)
Q3.5	What concerns do you have about using AI coding assistants in your workflow?	Both	Ranking[c]
RQ1: Task Focus Shifts			
Q3.4	When using an AI coding assistant, do you feel you spend more or less time on the following software engineering tasks?	Both	–
Q3.4_1	Designing	Both	Likert (time)
Q3.4_2	Writing code	Both	Likert (time)
Q3.4_3	Refactoring code	Both	Likert (time)
Q3.4_4	Reviewing code	Both	Likert (time)
Q3.4_5	Testing	Both	Likert (time)
Q3.4_6	Debugging	Both	Likert (time)
Q3.6	In what ways has using an AI coding assistant changed your workflow?	Both	Open-ended
RQ2: Developer Experience and Productivity			
Q4.1	How has your productivity changed since using an AI coding assistant?	Both	Likert (change)
Q4.2	How does using an AI coding assistant affect the following aspects of your Developer Experience?	Both	–
Q4.2_1	Feedback loops[d]	Both	Likert (change)
Q4.2_2	Cognitive load[d]	Both	Likert (change)
Q4.2_3	Flow state[d]	Both	Likert (change)

Continued on next page

Table A.1 continued from previous page

Item ID	Item	Q1/Q2	Scale
Q4.3*	Can you describe a specific instance where an AI coding assistant either significantly helped or hindered your work?	Q1	Open-ended
Q4.3*	Have you experienced any unexpected benefits or challenges from using an AI coding assistant? If so, can you describe a specific instance?	Q2	Open-ended
RQ3: Skill Development and Self-Efficacy			
Q5.1	Has using an AI coding assistant affected your ability to learn and improve your coding skills?	Both	Likert (likelihood)
Q5.2	Please indicate how much more or less you agree with each of the following statements regarding your confidence in building software when using an AI coding assistant:	Both	–
Q5.2_1	I will be able to achieve most of the goals that I have set for myself.[e]	Both	Likert (comparative agreement)
Q5.2_2	When facing difficult tasks, I am certain that I will accomplish them.[e]	Both	Likert (comparative agreement)
Q5.2_3	In general, I think that I can obtain outcomes that are important to me.[e]	Both	Likert (comparative agreement)
Q5.2_4	I believe I can succeed at most any endeavour to which I set my mind.[e]	Both	Likert (comparative agreement)
Q5.2_5	I will be able to successfully overcome many challenges.[e]	Both	Likert (comparative agreement)
Q5.2_6	I am confident that I can perform effectively on many different tasks.[e]	Both	Likert (comparative agreement)

Continued on next page

Table A.1 continued from previous page

Item ID	Item	Q1/Q2	Scale
Q5.2_7	Compared to other people, I can do most tasks very well.[e]	Both	Likert (comparative agreement)
Q5.2_8	Even when things are tough, I can perform quite well.[e]	Both	Likert (comparative agreement)
Q5.3	In what ways has using an AI coding assistant influenced your coding skills and software engineering practices?	Both	Open-ended

Continued on next page

Table A.1 continued from previous page

Item ID	Item	Q1/Q2	Scale
RQ4: Prompt Engineering Perceptions			
Q6.1	How important do you think prompt engineering will be for software engineers in the future?[f]	Both	Likert (importance)
Q6.2	To what extent do you agree that prompt engineering will replace some traditional coding skills?[f]	Both	Likert (agreement)
Q6.3*	How do you think software engineers should develop their prompt engineering skills, and what resources or methods would be most helpful?	Q1	Open-ended
Q6.3*	Has your approach to prompt engineering evolved over the last six months? If so, how and why?	Q2	Open-ended
Demographics			
Q7.1	Gender	Q1	Single choice
Q7.2	Age	Q1	Single choice
Q7.3	Years of software engineering experience	Q1	Single choice
Q7.4	Self-reported expertise level (Dreyfus model)	Q1	Single choice
Q7.5	Primary role	Both[†]	Single choice
Q7.6	Primary programming language	Both[†]	Single choice
Q7.7	Industry	Both[†]	Single choice
Q7.8	Company size	Both[†]	Single choice
Q7.9	Company headquarters location	Both[†]	Single choice
Q7.10	Country of residence	Both[†]	Single choice

Continued on next page

Table A.1 continued from previous page

Item ID	Item	Q1/Q2	Scale
Open Reflections			
Q8.2	What advice would you give to someone just starting their software engineering career in the age of AI?	Both	Open-ended
Q8.3	As the field of software engineering continues to evolve, what should engineers focus on to stay adaptable and thrive?	Both	Open-ended
Q8.4	Is there anything else you think we should know?	Both	Open-ended

Notes:

* Items with different wording between Q1 and Q2.

† In Q2, demographic items were only displayed if participants indicated their details had changed.

[a] Options: Yes (only at work); Yes (only on personal projects); Yes (both at work and on personal projects); No.

[b] Q1 offered 9 tools; Q2 offered 23 tools reflecting market evolution. Both included an "Other" option.

[c] Concerns: Quality; Readability; Maintainability; Security; Dependency; Learning curve; Other.

[d] Developer experience dimensions from the DevEx framework [71]. Definitions provided to participants in Section A.3.1.

[e] Self-efficacy items adapted from the New General Self-Efficacy Scale (NGSE) [19]. Participants indicated how much *more or less* they agreed with each statement when using an AI coding assistant.

[f] Definition of prompt engineering provided to participants in Section A.3.2.

A.2 Response Scales

Table A.2: Response scales used in the questionnaire

Scale Type	Response Options
Likert (time)	Much less; Somewhat less; About the same; Somewhat more; Much more
Likert (change)	Much worse; Somewhat worse; About the same; Somewhat better; Much better
Likert (agreement)	Strongly disagree; Somewhat disagree; Neither agree nor disagree; Somewhat agree; Strongly agree
Likert (comparative agreement)	Much less; Somewhat less; About the same; Somewhat more; Much more
Likert (importance)	Not at all important; Slightly important; Moderately important; Very important; Extremely important
Likert (likelihood)	Definitely not; Probably not; Might or might not; Probably yes; Definitely yes
Likert (impression)	Extremely negative; Somewhat negative; Neither positive nor negative; Somewhat positive; Extremely positive
Likert (disappointment)	Not at all disappointed; Slightly disappointed; Moderately disappointed; Very disappointed; Extremely disappointed
Likert (frequency)	Never; Once a week; 2–3 times a week; 4–6 times a week; Daily

A.3 Construct Definitions Provided to Participants

A.3.1 Developer Experience Dimensions

- **Feedback loops:** The speed and effectiveness with which you receive and act on feedback during development.

- **Cognitive load:** The mental effort required to complete your development tasks.
- **Flow state:** The extent to which you can work seamlessly and maintain concentration without interruptions.

A.3.2 Prompt Engineering

Prompt engineering, in the context of AI coding assistants, involves crafting precise and effective instructions or prompts to guide the AI in generating accurate code or providing helpful suggestions. It requires understanding how to communicate with the AI to optimise its responses and improve coding efficiency.

A.4 Key Differences Between Questionnaires

Table A.3: Key differences between Q1 and Q2

Item	**Questionnaire 1**	**Questionnaire 2**
Q3.1	9 AI tools listed	23 AI tools listed
Q3.3	"What was your initial impression of using an AI coding assistant?"	"If you could no longer use an AI coding assistant, how would you feel?"
Q4.3	"...significantly helped or hindered your work?"	"...unexpected benefits or challenges..."
Q6.3	"How do you think software engineers should develop their prompt engineering skills..."	"Has your approach to prompt engineering evolved over the last six months?"
Demographics	Full demographic questions	Only displayed if participant indicated changes

Appendix B

Standardised Variable Groupings

Table B.1: Standardised variable groupings used in analysis

Variable	**Standardised group**	**Cleaned categories included**
Experience Level	Junior	Less than 1 year; 1–2 years; 3–5 years
	Mid-Career	6–10 years; 11–15 years
	Senior	16–20 years; More than 20 years
	Other	Missing
Role	Backend	Backend
	Frontend	Frontend
	Fullstack	Fullstack

Continued on next page

Table B.1 continued from previous page

Variable	**Standardised group**	**Cleaned categories included**
	Mobile	Mobile
	Manager	Engineering Manager; Manager; Software Architect
	Specialised	Data/ML Engineer; Platform Engineer; Quality Engineer
	Other	Desktop Applications; Generalist; Missing
Primary Programming Language	Web	JavaScript; TypeScript
	Compiled	Java; C#; Go; Rust; Swift; Kotlin
	Interpreted	Python; Ruby; PHP
	Other	C++; Scala; Terraform; Missing
Company Size	Small	Self-employed; 1–50 employees
	Medium	51–500 employees
	Large	501–5,000 employees
	Very Large	5,001+ employees
	Other	Missing

Appendix C

Software and Packages

Table C.1: R packages used for data analysis (R version 4.4.2)

Category	**Package (Version)**	**Primary Purpose**
Data Processing	tidyverse (2.0.0)	Data manipulation, joining Q1/Q2 datasets, and visualisation (includes ggplot2).
Statistical Modelling	lme4 (1.1.37)	Mixed-effects models for paired Q1/Q2 responses.
	lmerTest (3.1.3)	Significance tests for mixed model comparisons.
	ordinal (2023.12.4.1)	Cumulative link models for ordinal Likert responses.
	effectsize (1.0.1)	Cohen's d and rank biserial correlation effect sizes.
	psych (2.5.6)	Cronbach's alpha for NGSE self-efficacy scale.

Continued on next page

Table C.1 continued from previous page

Category	**Package (Version)**	**Primary Purpose**
Model Diagnostics	broom (1.0.8)	Tidying model outputs for result tables.
	broom.mixed (0.2.9.6)	Tidying mixed-effects model outputs.
Visualisation	likert (1.3.5)	Diverging stacked bar charts for Likert responses.
	ggalluvial (0.12.5)	Sankey diagrams for tool adoption changes.
	patchwork (1.3.0)	Combining multiple plots into single figures.
	gridExtra (2.3)	Additional plot arrangement utilities.

All packages available from CRAN (`https://cran.r-project.org`).

Appendix D

AI Coding Assistants

Table D.1: AI coding assistants used by participants at Q1 ($n = 158$) and Q2 ($n = 101$)

Tool	Q1	Q2	Note
GitHub Copilot	110	61	
ChatGPT	111	59	
Claude	23	36	
Cursor	26	29	
Gemini	6	29	
Microsoft Copilot	7	20	
v0.dev	3	8	
Claude Code	0	11	New at Q2
Perplexity	3	5	
JetBrains AI Assistant	1	6	
Windsurf	0	7	New at Q2
AWS CodeWhisperer	4	2	
Sourcegraph Cody	2	2	
Codeium	3	0	Dropped at Q2
Tabnine	3	0	Dropped at Q2
Continue.dev	2	1	
Bolt.new	1	2	
Gemini Code Assist	2	0	Dropped at Q2
Databricks Assistant	1	1	
Aria	1	1	
Custom CLI Assistant	1	1	
Cline	0	2	New at Q2

Continued on next page

Table D.1 continued from previous page

Tool	Q1	Q2	Note
Lovable	0	2	New at Q2
Roo Code	0	2	New at Q2
Aider	1	0	Dropped at Q2
Groq	1	0	Dropped at Q2
Google Vertex AI	1	0	Dropped at Q2
Fitten Code	1	0	Dropped at Q2
phind.com	1	0	Dropped at Q2
OpenAI Codex	1	0	Dropped at Q2
Glean	1	0	Dropped at Q2
Supermaven	1	0	Dropped at Q2
OpenWebUI	1	0	Dropped at Q2
GitLab Duo	1	0	Dropped at Q2
Grok	0	1	New at Q2
Ollama	0	1	New at Q2
Trae	0	1	New at Q2

Works Cited

[1] Adam Alami and Neil Ernst. "Human and Machine: How Software Engineers Perceive and Engage with AI-Assisted Code Reviews Compared to Their Peers". In: *2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*. Apr. 2025, pp. 63–74. DOI: 10.1109/CHASE66643.2025.00016. URL: https://ieeexplore.ieee.org/document/11024268.

[2] J. W. Backus et al. "The FORTRAN automatic coding system". In: *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability*. IRE-AIEE-ACM '57 (Western). New York, NY, USA: Association for Computing Machinery, Feb. 1957, pp. 188–198. DOI: 10.1145/1455567.1455599. URL: https://dl.acm.org/doi/10.1145/1455567.1455599.

[3] Sebastian Baltes and Paul Ralph. "Sampling in software engineering research: a critical review and guidelines". In: *Empirical Software Engineering* 27.4 (Apr. 2022). DOI: 10.1007/s10664-021-10072-8. URL: https://doi.org/10.1007/s10664-021-10072-8.

[4] Albert Bandura. *Self-efficacy: The exercise of control*. New York: W.H. Freeman, 1997.

[5] Joel Becker et al. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. METR. 2025. URL: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.

[6] Zakariya Belkhamza and Geoffrey Hubona. "Nomological Networks in IS Research". In: *AMCIS 2018 Proceedings*. 2018.

[7] M.M. Beller. "An Empirical Evaluation of Feedback-Driven Software Development". PhD thesis. Delft University of Technology, 2018. DOI: 10.4233/UUID:B2946104-2092-42BB-A1EE-3B085D110466. URL: http://resolver.tudelft.nl/uuid:b2946104-2092-42bb-a1ee-3b085d110466 (visited on 01/09/2026).

[8] Mattan S. Ben-Shachar, Daniel Lüdecke, and Dominique Makowski. "effectsize: Estimation of Effect Size Indices and Standardized Parameters". In: *Journal of Open Source Software* 5.56 (2020), p. 2815. DOI: 10.21105/joss.02815. URL: https://joss.theoj.org/papers/10.21105/joss.02815.

[9] Alan W. Biermann. "Automatic programming : A tutorial on formal methodologies". In: *Journal of Symbolic Computation* 1.2 (June 1985), pp. 119–142. DOI: 10.1016/S0747-7171(85)80010-9. URL: https://linkinghub.elsevier.com/retrieve/pii/S0747717185800109.

[10] Christian Bird et al. "Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools". In: *Queue* 20.6 (Dec. 2022), pp. 35–57. DOI: 10.1145/3582083. URL: https://dl.acm.org/doi/10.1145/3582083.

[11] Virginia Braun and Victoria Clarke. "Reflecting on reflexive thematic analysis". In: *Qualitative Research in Sport, Exercise and Health* 11.4 (Aug. 2019), pp. 589–597. DOI: 10.1080/2159676x.2019.1628806. URL: https://www.tandfonline.com/doi/full/10.1080/2159676X.2019.1628806.

[12] Virginia Braun and Victoria Clarke. "Using thematic analysis in psychology". In: *Qualitative Research in Psychology* 3.2 (Jan. 2006), pp. 77–101. DOI: 10.1191/1478088706qp063oa. URL: http://www.tandfonline.com/doi/abs/10.1191/1478088706qp063oa.

[13] Brooks. "No Silver Bullet Essence and Accidents of Software Engineering". In: *Computer* 20.4 (Apr. 1987), pp. 10–19. DOI: 10.1109/mc.1987.1663532. URL: http://ieeexplore.ieee.org/document/1663532/.

[14] Adam Brown et al. "Identifying the Factors That Influence Trust in AI Code Completion". In: *Proceedings of the 1st ACM International Conference on AI-Powered Software*. July 2024, pp. 1–9. DOI: 10.1145/3664646.3664757. URL: https://doi.org/10.1145/3664646.3664757.

[15] Business Insider. *Anthropic CEO says AI will be writing 90% of code in 3 to 6 months*. Reporting remarks by Dario Amodei, CEO of Anthropic. 2025. URL: https://www.businessinsider.com/anthropic-ceo-ai-90-percent-code-3-to-6-months-2025-3.

[16] Business Insider. *Microsoft CTO says AI will generate 95% of code within 5 years*. Reporting remarks by Kevin Scott, CTO of Microsoft. 2025. URL: https://www.businessinsider.com/microsoft-cto-ai-generated-code-software-developer-job-change-2025-4.

[17] Jenna Butler et al. "Dear Diary: A Randomized Controlled Trial of Generative AI Coding Tools in the Workplace". In: *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. Apr. 2025, pp. 319–329. DOI: 10.1109/ICSE-SEIP66354.2025.00034. URL: https://ieeexplore.ieee.org/abstract/document/11121735.

[18] Sayan Chatterjee et al. *The Impact of AI Tool on Engineering at ANZ Bank: An Empirical Study on GitHub Copilot within Corporate Environment*. Feb. 2024. DOI: 10.48550/arXiv.2402.05636. URL: http://arxiv.org/abs/2402.05636.

[19] Gilad Chen, Stanley M. Gully, and Dov Eden. "Validation of a New General Self-Efficacy Scale". In: *Organizational Research Methods* 4.1 (Jan. 2001), pp. 62–83. DOI: 10.1177/109442810141004. URL: https://doi.org/10.1177/109442810141004.

[20] Tianyi Chen. "The Impact of AI-Pair Programmers on Code Quality and Developer Satisfaction: Evidence from TiMi studio". In: *2024 International Conference on Generative Artificial Intelligence and Information Security (GAIIS)*. May 2024, pp. 201–205. DOI: 10.1145/3665348.3665383. URL: https://dl.acm.org/doi/10.1145/3665348.3665383.

[21] Zhenpeng Chen et al. *Promptware Engineering: Software Engineering for LLM Prompt Development*. Mar. 2025. DOI: 10.48550/arXiv.2503.02400. URL: http://arxiv.org/abs/2503.02400.

[22] Lee Anna Clark and David Watson. "Constructing Validity: Basic Issues in Objective Scale Development". In: *Psychological Assessment* 7.3 (1995), pp. 309–319.

[23] John W. Creswell and Vicki L. Plano Clark. *Designing and Conducting Mixed Methods Research*. Third. Thousand Oaks, California: SAGE, 2018.

[24] Mihaly Csikszentmihalyi. *Flow and the Foundations of Positive Psychology: The Collected Works of Mihaly Csikszentmihalyi*. Dordrecht: Springer Netherlands, 2014. DOI: 10.1007/978-94-017-9088-8. URL: https://link.springer.com/10.1007/978-94-017-9088-8.

[25] Zheyuan (Kevin) Cui et al. *The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers*. SSRN Scholarly Paper. Rochester, NY, Feb. 2025. DOI: 10.2139/ssrn.4945566. URL: https://papers.ssrn.com/abstract=4945566.

[26] Sarah D'Angelo et al. "What Do Developers Want From AI?" In: *IEEE Software* 41.3 (May 2024), pp. 11–15. DOI: 10.1109/MS.2024.3363538. URL: https://ieeexplore.ieee.org/document/10493171.

[27] Paul Denny, Viraj Kumar, and Nasser Giacaman. "Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language". In: *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*. Association for Computing Machinery, 2023, pp. 1136–1142. DOI: 10.1145/3545945.3569823.

[28] Kayla DePalma et al. "Exploring ChatGPT's code refactoring capabilities: An empirical study". In: *Expert Systems with Applications* 249 (Sept. 2024), p. 123602. DOI: 10.1016/j.eswa.2024.123602. URL: https://www.sciencedirect.com/science/article/pii/S0957417424004676.

[29] Arthur W DeTore. "An Introduction to Expert Systems". In: *Journal of Insurance Medicine* 21.4 (1989).

[30] Edsger W. Dijkstra. "Letters to the editor: go to statement considered harmful". In: *Communications of the ACM* 11.3 (Mar. 1968), pp. 147–148. DOI: 10.1145/362929.362947. URL: https://dl.acm.org/doi/10.1145/362929.362947.

[31] DORA. *State of AI-assisted Software Development*. Tech. rep. Google, 2025. URL: https://dora.dev/research/2025/dora-report/.

[32] Tom Dotan. *Over 25% of Google's code is written by AI*. Reporting remarks by Sundar Pichai, CEO of Google. 2024. URL: https://fortune.com/2024/10/30/googles-code-ai-sundar-pichai/.

[33] Stuart E. Dreyfus. "The Five-Stage Model of Adult Skill Acquisition". In: *Bulletin of Science, Technology & Society* 24.3 (June 2004), pp. 177–181. DOI: 10.1177/0270467604264992. URL: http://journals.sagepub.com/doi/10.1177/0270467604264992.

[34] Tore Dybå and Torgeir Dingsøyr. "Empirical studies of agile software development: A systematic review". In: *Information and Software Technology* 50.9 (Aug. 2008), pp. 833–859. DOI: 10.1016/j.infsof.2008.01.006. URL: https://www.sciencedirect.com/science/article/pii/S0950584908000256.

[35] Christof Ebert and Panos Louridas. "Generative AI for Software Practitioners". In: *IEEE Software* 40.4 (July 2023), pp. 30–38. DOI: 10.1109/MS.2023.3265877. URL: https://ieeexplore.ieee.org/document/10176168.

[36] Denis Federiakin et al. "Prompt engineering as a new 21st century skill". In: *Frontiers in Education* 9 (2024), p. 1366434. DOI: 10.3389/feduc.2024.1366434.

[37] Phyliss Jia Gai, Jiayi Hou, and Yanping Tu. *Competence Penalty Is a Barrier to the Adoption of New Technology*. SSRN Scholarly Paper. Rochester, NY, May 2025. DOI: 10.2139/ssrn.5255039. URL: https://papers.ssrn.com/abstract=5255039.

[38] Nicholas Gardella, Raymond Pettit, and Sara L. Riggs. "Performance, Workload, Emotion, and Self-Efficacy of Novice Programmers Using AI Code Generation". In: *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. ITiCSE 2024. New York, NY, USA: Association for Computing Machinery, July 2024, pp. 290–296. DOI: 10.1145/3649217.3653615. URL: https://doi.org/10.1145/3649217.3653615.

[39] W. Wayt Gibbs. "Software's Chronic Crisis". In: *Scientific American* 271.3 (Sept. 1994), pp. 86–95. DOI: 10.1038/scientificamerican0994-86. URL: https://www.scientificamerican.com/article/softwares-chronic-crisis.

[40] Omer Gibreel and Ibrahim Arpaci. "Development and validation of the prompt engineering competence scale (PECS)". In: *Information Development* (Apr. 2025). DOI: 10.1177/02666669251336455. URL: https://journals.sagepub.com/doi/10.1177/02666669251336455.

[41] Manuel Hoffmann et al. *Generative AI and the Nature of Work*. SSRN Scholarly Paper. Rochester, NY, Apr. 2025. DOI: 10.2139/ssrn.5007084. URL: https://papers.ssrn.com/abstract=5007084.

[42] Brian Houck et al. *The SPACE of AI: Real-World Lessons on AI's Impact on Developers*. July 2025. DOI: 10.48550/arXiv.2508.00178. URL: http://arxiv.org/abs/2508.00178.

[43] Anna Y. Q. Huang et al. "The impact of GenAI-enabled coding hints on students' programming performance and cognitive load in an SRL-based Python course". In: *British Journal of Educational Technology* 56.5 (2025), pp. 1942–1972. DOI: 10.1111/bjet.13589. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/bjet.13589.

[44] Sarah Inman et al. "Seamful AI for Creative Software Engineering". In: *IEEE Software* 42 (2025). DOI: 10.1109/MS.2025.3534085. URL: https://ieeexplore.ieee.org/document/10857384.

[45] Neil S. Jacobson and Paula Truax. "Clinical significance: a statistical approach to defining meaningful change in psychotherapy research". In: *Journal of Consulting and Clinical Psychology* 59.1 (1991), pp. 12–19. DOI: 10.1037/0022-006X.59.1.12.

[46] JetBrains. *The State of Developer Ecosystem 2023: AI*. 2023. URL: https://www.jetbrains.com/lp/devecosystem-2023/ai/.

[47] JetBrains. *The State of Developer Experience and Developer Productivity*. 2025. URL: https://lp.jetbrains.com/devex-productivity-report-full-2025-dataviz/.

[48] Carlos E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* Oct. 2023. DOI: 10.48550/arXiv.2310.06770. URL: http://arxiv.org/abs/2310.06770.

[49] Mansi Khemka and Brian Houck. "Toward Effective AI Support for Developers: A survey of desires and concerns". In: *Queue* 22.3 (July 2024), Pages 60:53–Pages 60:78. DOI: 10.1145/3675416. URL: https://dl.acm.org/doi/10.1145/3675416.

[50] Nils Knoth et al. "AI literacy and its implications for prompt engineering strategies". In: *Computers and Education: Artificial Intelligence* 6 (2024), p. 100225. DOI: 10.1016/j.caeai.2024.100225.

[51] Mohammad Amin Kuhail et al. ""Will I be replaced?" Assessing ChatGPT's effect on software development and programmer perceptions of AI tools". In: *Science of Computer Programming* 235 (July 2024), p. 103111. DOI: 10.1016/j.scico.2024.103111. URL: https://www.sciencedirect.com/science/article/pii/S0167642324000340.

[52] Anand Kumar et al. *Intuition to Evidence: Measuring AI's True Impact on Developer Productivity*. Sept. 2025. DOI: 10.48550/arXiv.2509.19708. URL: http://arxiv.org/abs/2509.19708.

[53] Chuck Lance, Marcus Butts, and Lawrence Michels. "The Sources of Four Commonly Reported Cutoff Criteria: What Did They Really Say?" In: *Organizational Research Methods* 9 (Apr. 2006), pp. 202–220. DOI: 10.1177/1094428105284919.

[54] Eve Martina Lange, Åsa Cajander, and Maria Normark. "Exploring Flow in IT Professionals' Use of AI-Integrated Tools: Insights from Interviews". In: *Artificial Intelligence in HCI*. Ed. by Helmut Degen and Stavroula Ntoa. Cham: Springer Nature Switzerland, 2025, pp. 44–58. DOI: 10.1007/978-3-031-93429-2_3.

[55] C. Larman and V.R. Basili. "Iterative and incremental developments. a brief history". In: *Computer* 36.6 (June 2003), pp. 47–56. DOI: 10.1109/MC.2003.1204375. URL: http://ieeexplore.ieee.org/document/1204375/.

[56] Ze Shi Li et al. *AI Tool Use and Adoption in Software Development by Individuals and Organizations: A Grounded Theory Study*. June 2024. DOI: 10.48550/arXiv.2406.17325. URL: http://arxiv.org/abs/2406.17325.

[57] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. "A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges". In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ICSE '24. New York, NY, USA: Association for Computing Machinery, Feb. 2024, pp. 1–13. DOI: 10.1145/3597503.3608128. URL: https://dl.acm.org/doi/10.1145/3597503.3608128.

[58] Jenny T. Liang et al. *Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts*. Apr. 2025. DOI: 10.48550/arXiv.2409.12447. URL: http://arxiv.org/abs/2409.12447.

[59] Zohar Manna and Richard Waldinger. "A Deductive Approach to Program Synthesis". In: *ACM Trans. Program. Lang. Syst.* 2.1 (Jan. 1980), pp. 90–121. DOI: 10.1145/357084.357090. URL: https://dl.acm.org/doi/10.1145/357084.357090.

[60] Gloria Mark, Daniela Gudith, and Ulrich Klocke. "The cost of interrupted work: more speed and stress". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '08. New York, NY, USA: Association for Computing Machinery, Apr. 2008, pp. 107–110. DOI: 10.1145/1357054.1357072. URL: https://doi.org/10.1145/1357054.1357072.

[61] Antonio Mastropaolo et al. "On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot". In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Melbourne, Australia: IEEE, May 2023, pp. 2149–2160. DOI: 10.1109/ICSE48619.2023.00181. URL: https://ieeexplore.ieee.org/document/10172792/.

[62] André N. Meyer et al. "Software developers' perceptions of productivity". In: *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. FSE 2014. New York, NY, USA: Association for Computing Machinery, Nov. 2014, pp. 19–29. DOI: 10.1145/2635868.2635892. URL: https://doi.org/10.1145/2635868.2635892.

[63] André N. Meyer et al. "The Work Life of Developers: Activities, Switches, and Perceived Productivity". In: *IEEE Transactions on Software Engineering* 43.12 (2017), pp. 1178–1193. DOI: 10.1109/TSE.2017.2656886.

[64] Amr Mohamed, Maram Assi, and Mariam Guizani. *The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review*. July 2025. DOI: 10.48550/arXiv.2507.03156. URL: http://arxiv.org/abs/2507.03156.

[65] Hal Mooz and Kevin Forsberg. "10.2.1 The Dual Vee - Illuminating the Management of Complexity". In: *INCOSE International Symposium* 16.1 (2006), pp. 1368–1381. DOI: 10.1002/j.2334-5837.2006.tb02819.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/j.2334-5837.2006.tb02819.x.

[66] Arghavan Moradi Dakhel et al. "GitHub Copilot AI pair programmer: Asset or Liability?" In: *Journal of Systems and Software* 203 (Sept. 2023), p. 111734. DOI: 10.1016/j.jss.2023.111734. URL: https://www.sciencedirect.com/science/article/pii/S0164121223001292.

[67] Hussein Mozannar et al. "Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming". In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. CHI '24. New York, NY, USA: Association for Computing Machinery, May 2024, pp. 1–16. DOI: 10.1145/3613904.3641936. URL: https://doi.org/10.1145/3613904.3641936.

[68] Emerson Murphy-Hill et al. "What Predicts Software Developers' Productivity?" In: *IEEE Transactions on Software Engineering* 47.3 (Mar. 2021), pp. 582–594. DOI: 10.1109/TSE.2019.2900308. URL: https://ieeexplore.ieee.org/document/8643844.

[69] P. Naur and B. Randell. "Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th-11th October 1968". In: *https://eprints.ncl.ac.uk* (1969). URL: https://eprints.ncl.ac.uk.

[70] Nhan Nguyen and Sarah Nadi. "An empirical evaluation of GitHub copilot's code suggestions". In: *Proceedings of the 19th International Conference on Mining Software Repositories*. Pittsburgh Pennsylvania: ACM, May 2022, pp. 1–5. DOI: 10.1145/3524842.3528470. URL: https://dl.acm.org/doi/10.1145/3524842.3528470.

[71] Abi Noda et al. "DevEx: What Actually Drives Productivity: The developer-centric approach to measuring and improving productivity". In: *Queue* 21.2 (May 2023), Pages 20:35–Pages 20:53. DOI: 10.1145/3595878. URL: https://dl.acm.org/doi/10.1145/3595878.

[72] Ruchika Pandey et al. *Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects*. 2024. arXiv: 2406.17910 [cs.SE]. URL: https://arxiv.org/abs/2406.17910.

[73] Hammond Pearce et al. "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions". In: *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, May 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571. URL: https://ieeexplore.ieee.org/document/9833571/.

[74] Sida Peng et al. *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. 2023. arXiv: 2302.06590 [cs.SE]. URL: https://arxiv.org/abs/2302.06590.

[75] Neil Perry et al. "Do Users Write More Insecure Code with AI Assistants?" In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS '23. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 2785–2799. DOI: 10.1145/3576915.3623157. URL: https://doi.org/10.1145/3576915.3623157.

[76] Gustavo Pinto et al. "Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes". In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*. CAIN '24. New York, NY, USA: Association for Computing Machinery, June 2024, pp. 81–91. DOI: 10.1145/3644815.3644949. URL: https://dl.acm.org/doi/10.1145/3644815.3644949.

[77] Julien Pollack, Jane Helm, and Daniel Adler. "What is the Iron Triangle, and how has it changed?" In: *International Journal of Managing Projects in Business* 11.2 (May 2018), pp. 527–547. DOI: 10.1108/ijmpb-09-2017-0107. URL: https://www.emerald.com/insight/content/doi/10.1108/IJMPB-09-2017-0107/full/html.

[78] James Prather et al. "The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers". In: *Proceedings of the 2024 ACM Conference on International Computing Education Research*. Vol. 1. ACM, Aug. 2024, pp. 469–486. DOI: 10.1145/3632620.3671116. URL: https://doi.org/10.1145/3632620.3671116.

[79] Brian Randell. "Software Engineering: As it was in 1968." In: *Proceedings of the 4th International Conference on Software Engineering*. Jan. 1979, pp. 1–10.

[80] Sanka Rasnayaka et al. *An Empirical Study on Usage and Perceptions of LLMs in a Software Engineering Project*. Jan. 2024. DOI: 10.48550/arXiv.2401.16186. URL: http://arxiv.org/abs/2401.16186.

[81] Abdul Razzaq et al. "A Systematic Literature Review on the Influence of Enhanced Developer Experience on Developers' Productivity: Factors, Practices, and Recommendations". In: *ACM Comput. Surv.* 57.1 (Oct. 2024), 13:1–13:46. DOI: 10.1145/3687299. URL: https://doi.org/10.1145/3687299.

[82] C. Rich and R.C. Waters. "Automatic programming: myths and prospects". In: *Computer* 21.8 (Aug. 1988), pp. 40–51. DOI: 10.1109/2.75. URL: https://ieeexplore.ieee.org/document/75.

[83] Saima Ritonummi et al. "Flow Experience in Software Engineering". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 618–630. DOI: 10.1145/3611643.3616263. URL: https://doi.org/10.1145/3611643.3616263.

[84] W. W. Royce. "Managing the development of large software systems: concepts and techniques". In: *Proceedings of the 9th international conference on Software Engineering*. ICSE '87. Washington, DC, USA: IEEE Computer Society Press, Mar. 1987, pp. 328–338.

[85] Pranab Sahoo et al. "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications". In: *IEEE Open Journal of the Computer Society* 7 (2025), pp. 231–260. DOI: 10.1109/OJCS.2025.3531291.

[86] Yuya Sasaki et al. "Systematic Literature Review of Prompt Engineering Patterns in Software Engineering". In: *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. 2024, pp. 670–675. DOI: 10.1109/COMPSAC61105.2024.00096.

[87] Mali Senapathi, Jim Buchan, and Hady Osman. "DevOps Capabilities, Practices, and Challenges: Insights from a Case Study". In: *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*. EASE '18. New York, NY, USA: Association for Computing Machinery, June 2018, pp. 57–67. DOI: 10.1145/3210459.3210465. URL: https://dl.acm.org/doi/10.1145/3210459.3210465.

[88] Agnia Sergeyuk et al. "Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward". In: *Information and Software Technology* 178 (Feb. 2025), p. 107610. DOI: 10.1016/j.infsof.2024.107610. URL: https://www.sciencedirect.com/science/article/pii/S0950584924002155.

[89] Stack Overflow. *2025 Stack Overflow Developer Survey*. 2025. URL: https://survey.stackoverflow.co/2025/.

[90] Stack Overflow. *AI — 2024 Stack Overflow Developer Survey*. 2024. URL: https://survey.stackoverflow.co/2024/ai.

[91] Margaret-Anne Storey et al. "Towards a Theory of Software Developer Job Satisfaction and Perceived Productivity". In: *IEEE Transactions on Software Engineering* 47.10 (Oct. 2021), pp. 2125–2142. DOI: 10.1109/TSE.2019.2944354. URL: https://ieeexplore.ieee.org/document/8851296.

[92] Viktoria Stray et al. *Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study*. Sept. 2025. DOI: 10.48550/arXiv.2509.20353. URL: http://arxiv.org/abs/2509.20353.

[93] Thiago S. Vaillant et al. *Developers' Perceptions on the Impact of ChatGPT in Software Development: A Survey*. May 2024. URL: http://arxiv.org/abs/2405.12195.

[94] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. "Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models". In: *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. New Orleans LA USA: ACM, Apr. 2022, pp. 1–7. DOI: 10.1145/3491101.3519665. URL: https://dl.acm.org/doi/10.1145/3491101.3519665.

[95] An Vu and Jonas Oppenlaender. *Prompt Engineer: Analyzing Skill Requirements in the AI Job Market*. 2025. DOI: 10.48550/arXiv.2506.00058.

[96] Yoshija Walter. "Embracing the future of Artificial Intelligence in the classroom: the relevance of AI literacy, prompt engineering, and critical thinking in modern education". In: *International Journal of Educational Technology in Higher Education* 21.1 (2024), p. 15. DOI: 10.1186/s41239-024-00448-3.

[97] Thomas Weber et al. "Significant Productivity Gains through Programming with Large Language Models". In: *Proc. ACM Hum.-Comput. Interact.* 8.EICS (June 2024), 256:1–256:29. DOI: 10.1145/3661145. URL: https://doi.org/10.1145/3661145.

[98] Justin D. Weisz et al. *Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise*. Dec. 2024. DOI: 10.48550/arXiv.2412.06603. URL: http://arxiv.org/abs/2412.06603.

[99] Feiyang Xu et al. *AI-assisted Programming May Decrease the Productivity of Experienced Developers by Increasing Maintenance Burden*. Oct. 2025. DOI: 10.48550/arXiv.2510.10165. URL: http://arxiv.org/abs/2510.10165.

[100] Burak Yetiştiren et al. *Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT*. 2023. arXiv: 2304.10778 [cs.SE]. URL: https://arxiv.org/abs/2304.10778.

[101] Ramazan Yilmaz and Fatma Gizem Karaoglan Yilmaz. "The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation". In: *Computers and Education: Artificial Intelligence* 4 (2023), p. 100147. DOI: 10.1016/j.caeai.2023.100147. URL: https://linkinghub.elsevier.com/retrieve/pii/S2666920X23000267.

[102] Ilya Zakharov, Ekaterina Koshchenko, and Agnia Sergeyuk. *From Teacher to Colleague: How Coding Experience Shapes Developer Perceptions of AI Tools*. Apr. 2025. DOI: 10.48550/arXiv.2504.13903. URL: http://arxiv.org/abs/2504.13903.

[103] Han Zhang, Shiyi Wang, and Zijian Li. "The Neurophysiological Paradox of AI-Induced Frustration: A Multimodal Study of Heart Rate Variability, Affective Responses, and Creative Output". In: *Brain Sciences* 15.6 (May 2025), p. 565. DOI: 10.3390/brainsci15060565. URL: https://www.mdpi.com/2076-3425/15/6/565.